

\$author-display-name

09/11/2026



MIPI CSI-2 Receiver Subsystem Product Guide (PG232)

\$author-display-name

Introduction

Features

IP Facts

Overview

Core Overview

Sub-Core Details

Applications

Unsupported Features

Licensing and Ordering

Product Specification

Standards

Performance

Resource Utilization

Port Descriptions

Register Space

Designing with the Subsystem

General Design Guidelines

Shared Logic

I/O Planning

Clocking

Resets

Protocol Description

Design Flow Steps

Customizing and Generating the Subsystem

Constraining the Subsystem

Simulation

Synthesis and Implementation

Application Example Design

ZCU102 Application Example Design Overview

SP701 Application Example Design Overview

VCK190 Application Example Design Overview

VEK385 D-PHY Application Example Design Overview

VEK385 C-PHY Application Example Design Overview

VEK280 Application Example Design Overview

SCU200 XP5IO Application Example Design Overview

SCU200 HPIO Application Example Design Overview

Implementing the Example Design

Running the Design on Hardware

Vitis Debugger Flow to Program FPGA Instead of XSDB Console

Verification, Compliance, and Interoperability

Hardware Validation

Debugging

Debug Tools

Hardware Debug

Interface Debug

Additional Resources and Legal Notices

Support Resources

References

Revision History

Please Read: Important Legal Notices

MIPI CSI-2 Receiver Subsystem Product Guide (PG232)

Introduction

The Mobile Industry Processor Interface (MIPI) Camera Serial Interface (CSI-2) RX subsystem implements a CSI-2 receive interface according to the [MIPI CSI-2 standard v3.0](#) with underlying MIPI D-PHY standard v2.0/MIPI C-PHY Standard v2.0. The subsystem captures images from MIPI CSI-2 camera sensors and outputs AXI4-Stream video data ready for image processing. The subsystem allows fast selection of the top level parameters and automates most of the lower level parameterization. The AXI4-Stream video interface allows a seamless interface to other AXI4-Stream-based subsystems.

Features

- Support for 1–4 D-PHY lane and 1–3 C-PHY lane
- Line rates ranging from 80–4500 Mb/s

 **Note:** Refer to the specific data sheet for each device to determine the line rate support for that particular device.

- Multiple data type support:
 - RAW6
 - RAW7
 - RAW8
 - RAW10
 - RAW12
 - RAW14
 - RAW16
 - RAW20
 - RGB444
 - RGB555
 - RGB565
 - RGB666
 - RGB888
 - YUV420_8bit
 - YUV422_8bit
 - YUV422_10bit
- Filtering based on Virtual Channel Identifier
- Support for zero to 15 Virtual Channel
- Support for one, two, four, and eight pixels per clock at the output as defined in AXI4–Stream Video IP and System Design Guide ([UG934](#)) format
- AXI4–Lite interface for register access to configure different subsystem options
- Dynamic selection of active lanes within the configured lanes during subsystem generation
- Interrupt generation to indicate subsystem status information
- Enable dynamic line rate switching for Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices
- Internal D-PHY/C-PHY allows direct connection to image sources
- Support for Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices for the MIPI CSI 2 RX 3.0 standard
- Support for MIPI CSI–2 standard v2.0/3.0 features such as VCX, RAW16, and RAW20

IP Facts

AMD LogiCORE™ IP Facts Table	
Subsystem Specifics	
Supported Device Family ¹	AMD Versal™ adaptive SoCs, AMD UltraScale+™ , AMD Zynq™ UltraScale+™ MPSoC, AMD Zynq™ UltraScale+™ RFSoc, AMD Zynq™ 7000 SoC, 7 series FPGAs, AMD Spartan™ UltraScale+™
Supported User Interfaces	AXI4-Lite, AXI4-Stream
Resources	Performance and Resource Utilization web page
Provided with Subsystem	
Design Files	Encrypted RTL
Example Design	AMD Vivado™ IP integrator
Test Bench	Not Provided
Constraints File	XDC
Simulation Model	Not Provided
Supported S/W Driver ²	Standalone and Linux
Tested Design Flows ³	
Design Entry	AMD Vivado™ Design Suite
Simulation	For supported simulators, refer to the Vivado Design Suite User Guide: Release Notes, Installation, and Licensing (UG973).
Synthesis	AMD Vivado™ Synthesis
Support	
Release Notes and Known Issues	Master Answer Record: 65242

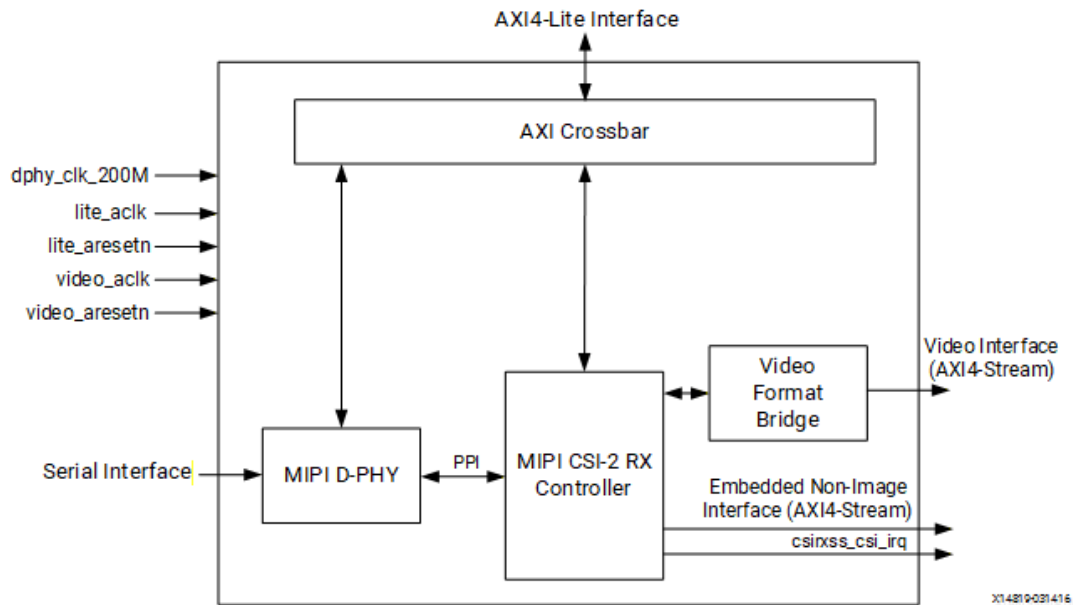
AMD LogiCORE™ IP Facts Table	
All Vivado IP Change Logs	Master AMD Vivado™ IP Change Logs: 72775
Support web page	
1. For a complete list of supported devices, refer to the AMD Vivado™ IP catalog. 2. Standalone driver details can be found in <install_directory>/Vitis/ <release>/data/embeddedsw/doc/xilinx_drivers_api_toc.htm. 3. For the supported versions of third-party tools, refer to the Vivado Design Suite User Guide: Release Notes, Installation, and Licensing (UG973).	

Overview

Core Overview

The MIPI CSI-2 RX subsystem allows you to quickly create systems based on the MIPI protocol. It interfaces between MIPI-based image sensors and an image sensor pipe. An internal high speed physical layer design, D-PHY/C-PHY, is provided that allows direct connection to image sources. The top-level customization parameters select the required hardware blocks needed to build the subsystem. The following figure shows the subsystem architecture.

Figure: Subsystem Architecture



The subsystem consists of the following sub-cores:

- MIPI D-PHY/MIPI RX-PHY
- MIPI CSI-2 RX Controller
- AXI Crossbar/Smart Connect
- Video Format Bridge

Sub-Core Details

MIPI D-PHY

For non-Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices, the MIPI D-PHY IP core implements a D-PHY RX interface and provides PHY protocol layer support compatible with the CSI-2 RX interface. The MIPI D-PHY IP core also supports the deskew pattern detection for line rates greater than 1500 Mb/s. Refer to the MIPI D-PHY LogiCORE IP Product Guide ([PG202](#)) for details. MIPI D-PHY implementation differs for the AMD UltraScale+™ devices and the 7 series devices concerning I/O.

For UltraScale+ devices, the AMD Vivado™ IDE provides a Pin Assignment Tab to select the required I/O. However, for the 7 series devices, you must select the clock-capable I/O manually. In addition, the 7 series devices do not have native

MIPI IOB support. You must target either HR bank I/O or HP bank I/O for the MIPI IP implementation. For more information on MIPI IOB compliant solution and guidance, refer to D-PHY Solutions ([XAPP894](#)).

MIPI RX C-PHY/D-PHY

For Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices, the MIPI RX-PHY IP core implements a D-PHY/C-PHY RX interface and provides PHY protocol layer support compatible with the CSI-2 RX interface. Refer to the MIPI RX C-PHY/D-PHY LogiCORE IP Product Guide ([PG435](#)) for more details.

MIPI CSI-2 RX Controller

The MIPI CSI-2 RX Controller core consists of multiple layers defined in the MIPI CSI-2 RX, such as the lane management layer, low-level protocol, and byte to pixel conversion.

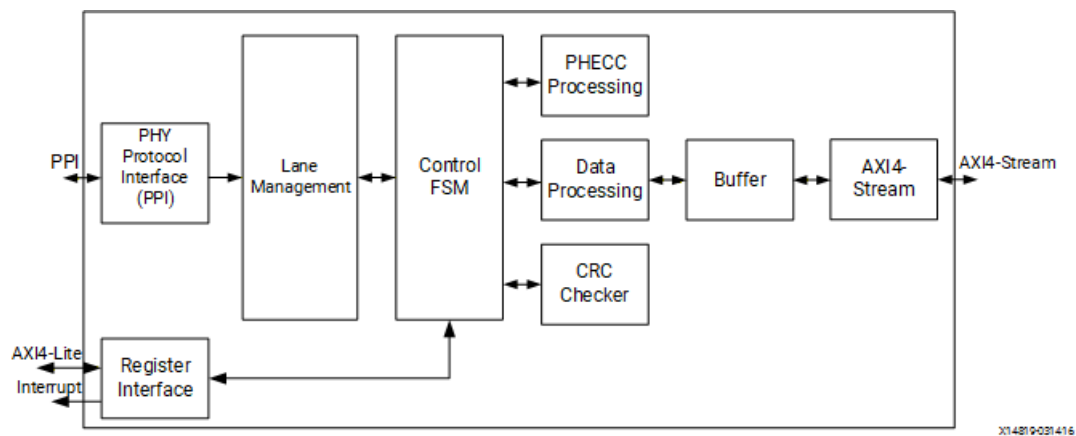
For non-Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices, the MIPI CSI-2 RX controller supports 8-bit data per lane, with support for up to four lanes from the MIPI D-PHY core through the PPI. As shown in the following figure, the byte data received on the PPI is processed by the low-level protocol module to extract the real image information. The final extracted image is made available to the user/processor interface using the AXI4-Stream protocol.

For Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices, whether it is D-PHY or C-PHY with 16-bit PPI, the lane management block consistently processes 64-bit data collected from PPI, regardless of the lane count. However, when dealing with Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 C-PHY 32-bit PPI, the lane management block consistently processes 128-bit data obtained from the PPI, independent of the number of lanes. As for non-Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices configurations, the lane management block operates uniformly on 64/128-bit data obtained from the PPI, regardless of the total number of lanes.

For Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices, the MIPI CSI-2 RX controller supports 16 (C-PHY/D-PHY)/32 (C-PHY)-bit data per lane and accommodates up to 4 lanes for D-PHY and 3 lanes for C-PHY with support for up to four lanes from the MIPI D-PHY core through the PPI. As shown in the following figure, the byte data received on the PPI is processed by the low-level protocol

module to extract the real image information. The final extracted image is made available to the user/processor interface using the AXI4-Stream protocol, D-PHY/C-PHY with 16-bit PPI, the lane management block consistently processes 64-bit data collected from PPI, regardless of lane count. However, when dealing with Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 C-PHY 32-bit PPI, the lane management block consistently processes 128-bit data obtained from PPI, independent of the number of lanes.

Figure: MIPI CSI-2 RX Controller Core



Features of this core include:

- 1–4 lane D-PHY support and 1–3 lane C-PHY support, with register support to select active lanes (the actual number of available lanes to be used)
- Short and long packets with all word count values supported
- Primary and many secondary video formats supported
- Data type (DT) interleaving
- Virtual Channel Identifier (VC) interleaving
- Combination of Data Type and VC interleaving
- Multi-lane interoperability
- For D-PHY, error correction code (ECC) for 1-bit error correction and 2-bit error detection in packet header
- CRC check for payload data
- Long packet ECC/CRC forwarding capability for downstream IPs
- Maximum data rate of 4500 Mb/s pixel byte packing based on data format
- AXI4-Lite interface to access core registers
- Low power state detection
- Error detection (PHY level Errors, Packet Level Errors, and Protocol Decoding Level Errors)
- AXI4-Stream interface with 32/64/192-bit TDATA width support to offload pixel information externally
- Interrupt support for indicating internal status/error information.

The following tables show image AXI4-Stream interface data width selection based on the selected data type, VFB enabled, and pixel per clock (PPC).

Table: Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 Image AXI4-Stream Interface TDATA Widths (VFB Enabled)

Data Type	AXI4-Stream Interface TDATA Width (PPC=1)	AXI4-Stream Interface TDATA Width (PPC=2)	AXI4-Stream Interface TDATA Width (PPC=4)	AXI4-Stream Interface TDATA Width (PPC=8)
RAW6	8	16	32	64
RAW7	8	16	32	64
RAW8	8	16	32	64
RAW10	16	24	40	80
RAW12	16	24	48	96
RAW14	16	32	56	112

Data Type	AXI4–Stream Interface TDATA Width (PPC=1)	AXI4–Stream Interface TDATA Width (PPC=2)	AXI4–Stream Interface TDATA Width (PPC=4)	AXI4–Stream Interface TDATA Width (PPC=8)
RAW16	16	32	64	128
RAW20	24	40	80	160
RGB444	24	48	96	192
RGB555	24	48	96	192
RGB565	24	48	96	192
RGB666	24	48	96	192
RGB888	24	48	96	192
YUV420_8bit	16	32	64	128
YUV422_8bit	16	32	64	128
YUV422_10bit	24	40	80	

Table: VersalAdaptive SoC and UltraScale+ Image AXI4–Stream Interface TDATA Widths (VFB Enabled)

Data Type	AXI4–Stream Interface TDATA Width (PPC=1)	AXI4–Stream Interface TDATA Width (PPC=2)	AXI4–Stream Interface TDATA Width (PPC=4)
RAW6	8	16	32
RAW7	8	16	32
RAW8	8	16	32
RAW10	16	24	40
RAW12	16	24	48
RAW14	16	32	56
RAW16	16	32	64
RAW20	24	40	80

Data Type	AXI4-Stream Interface TDATA Width (PPC=1)	AXI4-Stream Interface TDATA Width (PPC=2)	AXI4-Stream Interface TDATA Width (PPC=4)
RGB444	16	24	48
RGB555	16	32	64
RGB565	16	32	64
RGB666	24	40	72
RGB888	24	48	96
YUV420_8bit	16	32	64
YUV422_8bit	16	32	64
YUV422_10bit	24	40	80

The following table shows the embedded non-image (with data type code 0x12) AXI4-Stream interface data width selection based on the selected data type.

Table: Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 Embedded Non-Image AXI4-Stream Interface TDATA Widths

Data Type (DT)	AXI4-Stream Interface TDATA Width
RAW6	96
RAW7	96
RAW8	96
RAW10	96
RAW12	96
RAW14	192
RAW16	192
RAW20	192
All RGB	192
YUV420_8bit	192

Data Type (DT)	AXI4–Stream Interface TDATA Width
YUV422_8bit	192
YUV422_10bit	192

Table: Versal Adaptive SoC and UltraScale+ Embedded Non–Image AXI4–Stream Interface TDATA Widths

Data Type (DT)	AXI4–Stream Interface TDATA Width
RAW6	32
RAW7	32
RAW8	32
RAW10	64
RAW12	64
RAW14	64
RAW16	64
RAW20	64
All RGB	64
YUV420_8bit	64
YUV422_8bit	64
YUV422_10bit	64

Abrupt termination events such as a soft reset, disabling a core while a packet is being written to the line buffer, or a line buffer full condition results in early termination. The termination is implemented by the assertion of EOL on the video interface or TLAST and TUSER [1] on the embedded non–image interface, based on the current long packet being processed.

Note: In the case of D–PHY with a non–continuous clock, if a reset leads to an ISR error, it is likely because the reset has caused stale data to remain in the FIFO. To resolve this, you have two options:

1. Clear the ISR before sending new data.

2. Ensure that an adequate number of PPI clock cycles are present after the reset to allow the FIFO to exit the reset cleanly.

Null/blanking packets are ignored by the MIPI CSI-2 RX Controller. No errors are reported in such cases, and the core continues with the next packet processing.

ECC/CRC Forwarding

Sideband signals of AXI4-Stream interface [Include/Exclude Video Format Bridge and Embedded non-image interface] report ECC and CRC data received from the source [sensor] to downstream IPs. This allows re-calculating ECC/CRC by the downstream IPs in specific functional safety applications. Refer to Port Descriptions for details on signal mapping.

In error scenarios, such as abrupt termination due to a soft reset, disabling the core while a packet transfer is in progress, a line buffer in a full condition, or when the word count of a received packet exceeds the actual payload, these sideband signals do not accurately report the ECC and CRC.

Related Information

[Port Descriptions](#)

VCX Support

The MIPI CSI-2 standard v2.0 specific VCX support feature extends the maximum available virtual channels to 16. When enabled, the virtual channel is deduced by combining the 2-bit VC field (LSB) and the 2-bit VCX field (MSB) from the packet header.

 **Note:** The IP reports ECC errors if the source is sending VC>3 and VCX Support is not selected. Hence, the source does not send VC>3 when VCX Support is not selected.

The MIPI CSI-2 RX 3.0 standard inherently supports the VCX feature by default, with no options available to enable or disable it.

Tclk-Post Requirement

MIPI CSI-2 RX Subsystem needs Tclk-POST to be a minimum of $18 \times \text{rxbyteclkhs}$.
MIPI D-PHY v2.0 recommends the minimum value of Tclk-POST as $60 \text{ ns} + 52 \text{ UI}$.
For non-Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices, consider the maximum of these two Tclk-POST values for the MIPI source.

For example:

1. For a line rate of 896 Mb/s, rxbyteclkhs is 8.9285 ns. The required Tclk-POST is $18 \times 8.9285 \text{ ns} = 160 \text{ ns}$
2. The following tables show examples for specific line rates.

Table: Line Rate Specific Example

Line Rate	EQ1: Tclk-POST minimum per D-PHY 2.0 specification ($60 \text{ ns} + 52 \text{ UI}$)	EQ2: Required Tclk-POST minimum by MIPI CSI-2 RX Subsystem	Final Tclk-POST MAX (EQ1, EQ2)
896	118 ns	160 ns	160 ns
1440	96 ns	100 ns	100 ns
2000	86 ns	72 ns	86 ns

Tclk-POST is the Transmitter (D-PHY or CSI-2) setting you need to set based on the receiver requirements.

AXI Crossbar

The subsystem uses the AXI Crossbar core or SmartConnect core (for Versal adaptive SoCs) to route AXI4-Lite requests to corresponding sub-cores based on the address. Refer to the following documents for details:

- AXI Interconnect LogiCORE IP Product Guide ([PG059](#))
- SmartConnect LogiCORE IP Product Guide ([PG247](#))

Video Format Bridge

The Video Format Bridge core uses the user-selected VC and Data Type information to filter only the required AXI4-Stream data beats. This AXI4-Stream data is further processed based on the Data Type information, and the output is based on the requested number of pixels per clock. The output interface adheres to the protocol defined in the AXI4-Stream Video IP and System Design Guide ([UG934](#)).

The Video Format Bridge core processes the data type selected in the Vivado Integrated Design Environment (IDE) and filters out all other data types except for RAW8 and User Defined Byte-based Data types (0x30 to 0x37) received from the CSI-2 RX Controller.

Irrespective of the Vivado IDE selection, RAW8 and User Defined Byte-based Data types are always processed by the Video Format Bridge core. This allows multiple data-type support, one main data type from the Vivado IDE for pixel data and a user-defined byte-based data type for metadata. When multiple data types are transferred (for example, RAW10 and User Defined Byte-based Data), the actual placement pixel data bits are defined in the AXI4-Stream Video IP and System Design Guide ([UG934](#)).

For unaligned transfers, there is no way to specify the partial final output (TKEEP) for the output interface. Ensure that you consider this and discard the unintended bytes in the last beats when there are unaligned transfers.

YUV420 8-Bit Support

Ensure the following guidelines to use the YUV420 8-bit (0x18) feature correctly:

- Ensure that you maintain at least 1 Y-line gap between two successive YUV420 8-bit frames.
- To use virtual channel interleaving, you must maintain at least 1 Y-line gap while switching to a different virtual channel.
- To use data type interleaving or the embedded non-image feature, you must ensure that at least 1 Y-line gap is maintained while switching the data types.
- Consider the pixel width as eight when calculating the `video_aclk` value. Refer to Clocking for details on calculating `video_aclk`.
- The maximum Y-line word count of the incoming data has to be at least eight bytes less than the 'YUV420 word count' value configured in the GUI. For example, if the buffer depth selected in GUI is 1024 bytes, the source/sensor can send Y-lines with a word count of up to 1016 bytes.
- For Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2, D-PHY and C-PHY (≤ 2250) if the buffer depth selected in GUI is 1024 bytes, the source/sensor can send Y-lines with a word count of up to 1008 bytes.
- C-PHY (> 2250) if the buffer depth selected in GUI is 1024 bytes, the source or the sensor can send Y-lines with a word count of up to 992 bytes.

 Note:

1. For the YUV420 data format, the abrupt packet end is not indicated by `TUSER[1]` accompanied by the `tlast` assertion. Instead, discard the current frame and make sure the TX sends a new frame, indicated by a frame start (`tuser[0]`).
2. Core disable and re-enable is required when ECC 2-bit errors or SoT Sync errors (`ErrSotSyncHS`) occur in the YUV420 8-bit reception because this data format works in pairs.

Related Information

[Clocking](#)

video_out Port Width

The width of the data port in the video_out interface depends on the data type selected and the number of pixels per clock selected. The width is a maximum of the RAW8, and the data type selected in the Vivado IDE is multiplied by the number of pixels per clock. This is then rounded to the nearest byte boundary per the AXI4-Stream protocol.

Example 1: RAW10 and Two Pixels per Clock Selected in the Vivado IDE

- Single pixel width of RAW10 = 10
- Single pixel width of RAW8 = 8

For the selected two pixels per clock, the effective pixel widths are 20 and 16 for RAW10 and RAW8, respectively. The `video_out` port width is configured as the maximum of the individual pixel widths and rounded to the nearest byte boundary. This results in a `video_out` port width of 24.

Example 2: RAW7 and Four Pixels per Clock Selected in the Vivado IDE

- Single pixel width of RAW7 = 7
- Single pixel width of RAW8 = 8

With four pixels per clock selected, the effective pixel widths are 28 and 32 for RAW7 and RAW8, respectively. The `video_out` port width is configured as the maximum of the individual pixel widths and rounded to nearest byte boundary. This results in a `video_out` port width of 32.

Pixel Packing for Multiple Data Types

When multiple pixels are transferred with different pixel width, the pixels with lower width are justified to most significant bits.

Example 1

When RAW12 and RAW8 are transferred with two pixels per clock, the data port width of the `video_out` interface is 24 bits. Within the 24 bits, the RAW8 pixels are aligned to the most significant bits as shown in the following table.

 **Important:** In a multi-pixel scenario, pixel width varies, and pixels with lower width are justified to the most significant bit.

Table: Pixel Packing for RAW12 and RAW8 Data Types

Bit Positions	RAW12																								RAW8							
	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
RAW12	q1	q1	q9	q8	q7	q6	q5	q4	q3	q2	q1	q0	p1	p1	p9	p8	p7	p6	p5	p4	p3	p2	p1	p0								
RAW8																									q7	q6	q5	q4	q3	q2	q1	q0

1. p0 to p11 is the first pixel bits of RAW12; q0 to q11 is the second pixel bits of RAW12.
2. p0 to p7 is the first pixel bits of RAW8; q0 to q7 is the second pixel bits of RAW8.

Example 2

When the core is configured with RAW6 and two pixels per clock, the video_out port width is set to 16 bits. Within the 16 bits, the RAW6 and RAW8 pixels are aligned to the most significant bits as shown in the following table.

Table: Pixel Packing for RAW8 and RAW6 Data Types

Bit Position	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RAW8	q7	q6	q5	q4	q3	q2	q1	q0	p7	p6	p5	p4	p3	p2	p1	p0
RAW6	q5	q4	q3	q2	q1	q0			p5	p4	p3	p2	p1	p0		

1. p0 to p7 is the first pixel bits of RAW8; q0 to q7 is the second pixel bits of RAW8.

2. p0 to p5 is the first pixel bits of RAW6; q0 to q5 is the second pixel bits of RAW6.

Pixel Packing for Embedded Non-Image Data Types

AXI4-Stream TDATA width is based on main data type selected from the AMD Vivado™ IDE. The position of embedded non-image data type bytes on `emb_nonimg_tdata` are listed as follows:

- First byte on `emb_nonimg_tdata` [7:0]
- Second byte on `emb_nonimg_tdata` [15:8] and so on.

Pixel Packing When Video Format Bridge is Not Present

The width of the data port in the `video_out` can be selected from Vivado IDE, under CSI-2 Options TDATA width. MIPI CSI-2 RX Subsystem follows the Recommended Memory Storage section of the MIPI CSI-2 specifications to output pixels when a video format bridge is not present.

For more information on the data type packing, refer to the [MIPI Alliance Standard for Camera Serial Interface CSI-2 Specification](#).

Example

The following tables show the pixel mapping for different data types.

Pixel Packing for RAW8 Data Type

Table: Pixel Packing for RAW8 Data Type

B i t P o s i t i o n	3	3	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	1	9	8	7	6	5	4	3	2	1	0		
	1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0											
R A W 8	s	s	s	s	s	s	s	s	s	r	r	r	r	r	r	r	r	q	q	q	q	q	q	q	q	p	p	p	p	p	p	p	
	7	6	5	4	3	2	1	0		7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
1. p0 to p7 is the first pixel bits of RAW8; q0 to q7 is the second pixel bits of RAW8.																																	

Pixel Packing for RAW10 Data Type

Table: Pixel Packing for RAW10 Data Type

B i t P o s i t i o n	3	3	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	1	9	8	7	6	5	4	3	2	1	0
	1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0										
R A W 1 0	s	s	s	s	s	s	s	s	r	r	r	r	r	r	r	r	q	q	q	q	q	q	q	q	p	p	p	p	p	p	p	p
R A W 1 0	v	v	v	v	v	v	v	v	u	u	u	u	u	u	u	u	t	t	t	t	t	t	t	s	s	r	r	q	q	p	p	p
R A W 1 0	y	y	y	y	y	y	y	y	x	x	x	x	x	x	x	x	w	w	v	v	u	u	t	t	w	w	w	w	w	w	w	w

1. In RAW10, MSB 8 bits of 4 pixels are transferred first, followed by LSB 2 bits of each pixel.

Pixel Packing for RGB888 Data Type

Table: Pixel Packing for RGB888 Data Type

B i t P o s i t i o n	B	3	3	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	1	9	8	7	6	5	4	3	2	1	0
	i	1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0									
	t																															
	P																															
	o																															
	s																															
	i																															
	t																															
	i																															
	o																															
	n																															
R G B 8 8 8	R	d	d	d	d	d	d	d	c	c	c	c	c	c	c	c	b	b	b	b	b	b	b	b	a	a	a	a	a	a	a	
	G	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1
	B																															
	8																															
	8																															
	8																															
	R	h	h	h	h	h	h	h	g	g	g	g	g	g	g	g	f	f	f	f	f	f	f	f	e	e	e	e	e	e	e	
	G	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1
	B																															
	8																															
	8																															
	8																															

1. In RGB888, a0 to a7 represents the B component, b0 to b7 represents the G component, and c0 to c7 represents the R component.

Applications

The AMD MIPI CSI-2 RX controller implements a Camera Serial Interface between a camera sensor and a programmable device performing baseband processing. The bandwidth requirement for the camera sensor interface has increased due to the development of higher-resolution cameras. Traditional parallel interfaces require an increasing number of signal lines, resulting in higher power consumption. The new high-speed serial interfaces, such as MIPI CSI specifications, address these expanding bandwidth requirements without sacrificing power. MIPI is a group of protocols defined by the mobile industry group to standardize all interfaces within mobile platforms such as mobile phones and tablets. However, the large volumes

and the economies of scale of the mobile industry are forcing other applications to adopt these standards. As such, MIPI-based camera sensors are increasingly used in applications such as driver assistance technologies in automotive applications, video security surveillance cameras, video conferencing and emerging applications such as virtual and augmented reality.

Unsupported Features

- When the Video Format Bridge is included, some YUV data types (YUV420 (10-bit))
- Escape mode processing
- Eight lanes support
- Data scramble feature
- Latency reduction and transport efficiency features
- Null data type and blanking data type

Licensing and Ordering

This AMD LogiCORE™ IP module is provided at no additional cost with the AMD Vivado™ Design Suite under the terms of the [End User License](#).

For more information, visit the MIPI CSI-2 RX Subsystem [product web page](#)

Information about other AMD LogiCORE™ IP modules is available at the [Intellectual Property](#) page. For information about pricing and availability of other AMD LogiCORE IP modules and tools, contact your [local sales representative](#).

Product Specification

Standards

- MIPI Alliance Standard for Camera Serial Interface CSI-2 v2.0/v3.0
- MIPI Alliance Physical Layer Specifications, D-PHY Specification v1.2, D-PHY specification v2.0/C-PHY specification v2.0
- Processor Interface, AXI4-Lite: Refer to the Vivado Design Suite: AXI Reference Guide ([UG1037](#))
- Output Pixel Interface: Refer to the AXI4-Stream Video IP and System Design Guide ([UG934](#))

Performance

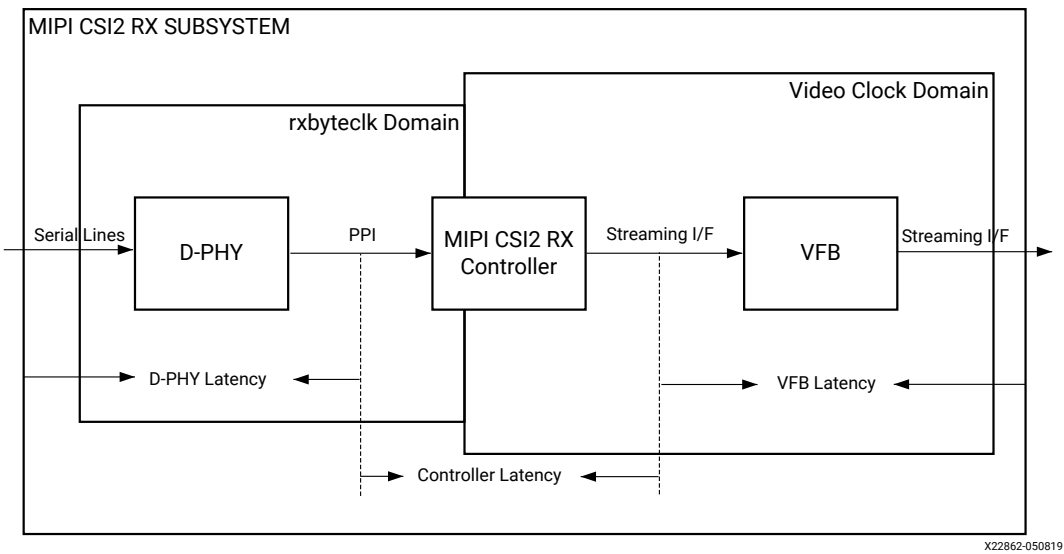
This section details the performance information for various core configurations.

Latency

The CSI-2 RX Subsystem core latency is the time from the start-of-transmission (SoT) pattern on the serial lines to the `tvalid` signal assertion at CSI-2 RX Subsystem output. This includes the D-PHY latency, MIPI RX Controller latency and VFB latency (if the Video Format Bridge is included in the subsystem).

The following figure represents the latency calculation for the subsystem.

Figure: MIPI CSI-2 RX Subsystem Latency Calculation



D-PHY Latency

The MIPI D-PHY RX core latency is the time from the start-of-transmission (SoT) pattern on the serial lines to the active `hs` signal assertion on the PPI. The `HS_SETTLE` period contributes significantly in the D-PHY latency calculation.

The following table provides the latency numbers for various core configurations.

Table: D-PHY Latency

Data Type	Pixel Mode	Line Rate	Latency in rxbyteclk (HS_SETTLE + Internal latency)
RAW20	Single	1000	26(23+3)
RAW8	Single	1000	26(23+3)
RAW8	Dual	1000	26(23+3)
RAW8	Quad	1000	26(23+3)
RAW10	Single	1000	26(23+3)
RAW10	Dual	1200	30(26+4)
RAW10	Quad	800	22(20+2)
YUV422_8bit	Quad	2000	44
1. All the calculations are made for a single lane design with a fixed video clock of 148 MHz.			

MIPI CSI-2 RX Controller Latency

The MIPI CSI-2 RX Controller core latency is the time from the active hs assertion on the PPI interface to valid signal assertion on the controller output.

The following table provides the latency numbers for various core configurations.

Table: MIPI CSI-2 RX Controller Latency

Data Type	Pixel Mode	Line Rate	Latency in rxbyteclk	Latency in Video Clock
RAW20	Single	1000	25	60
RAW8	Single	1000	21	49
RAW8	Dual	1000	21	49
RAW8	Quad	1000	21	49
RAW10	Single	1000	25	60

Data Type	Pixel Mode	Line Rate	Latency in rxbyteclk	Latency in Video Clock
RAW10	Dual	1200	26	63
RAW10	Quad	800	24	56
YUV422_8bit	Quad	2000	31	18
1. All the calculations are made for a single lane design with a fixed video clock of 148 MHz.				

Video Format Bridge (VFB) Latency

The VFB core latency is the time from the VFB input stream interface `tvalid` to VFB output stream interface `tvalid`.

The following table provides the latency numbers for various core configurations.

Table: VFB Latency

Data Type	Pixel Mode	Line Rate	Latency in rxbyteclk	Latency in Video Clock
RAW20	Single	1000	10	24
RAW8	Single	1000	1	3
RAW8	Dual	1000	1	3
RAW8	Quad	1000	1	4
RAW10	Single	1000	2	6
RAW10	Dual	1200	3	6
RAW10	Quad	800	1	3
YUV422_8bit	Quad	2000	16	9
1. All the calculations are made for a fixed video clock of 148 MHz.				

The following table provides the overall latency numbers of MIPI CSI-2 RX

Subsystem for various core configurations.

Table: MIPI CSI-2 RX Subsystem Latency

Data Type	Pixel Mode	Line Rate	Latency in rxbyteclk
RAW20	Single	1000	61
RAW8	Single	1000	48
RAW8	Dual	1000	48
RAW8	Quad	1000	48
RAW10	Single	1000	53
RAW10	Dual	1200	59
RAW10	Quad	800	47
YUV422_8bit	Quad	2000	92

1. All the calculations are made for a single lane design with a fixed video clock of 148 MHz.
2. The latency is improved by increasing the number of lanes.
3. Overall latency in micro seconds (μs) can be calculated from the data provided in this table.
For RAW8 Single pixel mode with a line rate of 1000 Mb/s, rxbyteclk in MHz
 $(\text{Line rate}/8) = 1000/8 = 125 \text{ MHz}$
Latency in μs (Latency in rxbyteclk * rxbyteclk period) = $48 * (1/125 \text{ MHz}) = 0.384 \mu s$.

Resource Utilization

For full details about performance and resource utilization, visit the [Performance and Resource Utilization web page](#).

Note: Refer to the CSI-2 Optimization feature (CSI-2 Controller Register Interface) for additional resource optimization.

Also, refer to the MIPI D-PHY core resource utilization to understand the resource requirements for line rates greater than 1500 Mb/s.

Port Descriptions

The following table describes the MIPI CSI-2 RX Subsystem I/O signals.

Table: Port Descriptions

Signal Name	Direction	Description
lite_aclk	I	AXI clock.
lite_aresetn	I	AXI reset. Active-Low. This signal needs to be synchronous to lite_aclk.
S00_AXI*	–	AXI4-Lite interface, defined in the Vivado Design Suite: AXI Reference Guide (UG1037)
dphy_clk_200M	I	Clock for PHY core. Must be 200 MHz.
video_aclk	I	Subsystem clock.
video_aresetn	I	Subsystem reset. Active-Low. This signal needs to be synchronous to video_aclk.
AXI4-Stream Video Interface when Video Format Bridge is Present		
video_out_tvalid	O	Data valid.
video_out_tready	I	Slave ready to accept the data.

Signal Name	Direction	Description
video_out_tuser[n-1:0]	O	n is based on TUSER width selected in the Vivado IDE. • 112-127: CPHY Header CRC • 96-111: Reserved • 95-80: CRC • 79-72: ECC • 71-70: Reserved • 69-64: Data Type • 63-48 Word Count • 47-32: Line Number • 31-16: Frame Number • 15-2: Reserved • 1: Packet Error • 0: Start of Frame
video_out_tlast	O	End of line.
video_out_tdata[n-1:0]	O	Data n is based on the Data type and number of pixels selected in the Vivado IDE (refer to video_out Port Width).
video_out_tdest[0-10] ¹	O	• 5-10: Data Type • 4: Reserved • 3-0: Virtual Channel Identifier (VC)
AXI4-Stream Interface when Embedded Non-image Interface is Selected		

Signal Name	Direction	Description
emb_nonimg_tdata[n-1:0]	O	Data. n is based on Data type selected in the Vivado IDE (refer to Embedded Non-Image AXI4-Stream Interface TDATA Widths).
emb_nonimg_tdest[3:0]	O	Specifies the Virtual Channel Identifier (VC) value of the embedded non-image packet.
emb_nonimg_tkeep[n/8-1:0]	O	Specifies valid bytes.
emb_nonimg_tlast	O	End of line.
emb_nonimg_tready	I	Slave ready to accept data.
emb_nonimg_tuser[112:0]	O	112-96: Reserved 95-80: CRC 79-72: ECC 71-70: Reserved 69-64: Data Type 63-48: Word Count 47-32: Line Number 31-16: Frame Number 15-2: Reserved 1: Packet Error 0: Start of frame
emb_nonimg_tvalid	O	Data valid.
MIPI ISP Bridge option is selected When DPHY selected		

Signal Name	Direction	Description
header_data[31:0]	O	Header Data • 31–30: VCX • 29–24: ECC • 23–8: Word count • 7–6: VC • 5–0: Data Type
header_valid	O	Header valid
MIPI ISP Bridge option is selected When CPHY selected		
header_data[47:0]	O	• Header Data • 47–42: Reserved • 41–40: VCX • 39–24: Header CRC • 23–8: Word count • 7–6: VC • 5–0: Data Type
header_valid	O	Header valid
AXI4–Stream Interface when Video Format Bridge is Not Present		
video_out_tdata[n–1:0]	O	Data. n is based on TDATA width selected in the Vivado IDE.
video_out_tdest[n–1:0]	O	n is based on TDEST width selected in the Vivado IDE: • 5–10: Data type • 4: Reserved • 3–0: Virtual Channel Identifier (VC)

Signal Name	Direction	Description
video_out_tkeep[n/8-1:0]	O	Specifies valid bytes.
video_out_tlast	O	End of line.
video_out_tready	I	Slave ready to accept data.
video_out_tuser[n-1:0]	O	n is based on TUSER width selected in the Vivado IDE • 112-96: Reserved • 95-80: CRC • 79-72: ECC • 71-70: Reserved • 69-64: Data Type • 63-48: Word Count • 47-32: Line Number • 31-16: Frame Number • 15-2: Reserved • 1: Packet Error • 0: Start of frame
video_out_tvalid	O	Data valid.
Other Signals		
csirxss_csi_irq	O	Interrupt (active-High) from CSI-2 RX Controller.
frame_rcvd_pulse_out ²	O	High when the Frame End (FE) short packet is received for the frame_rcvd_pulse_out O current frame.
AMD 7 series FPGA		

Signal Name	Direction	Description
mipi_dphy_if	O	D-PHY interface.
rxbyteclkhs	O	PPI high-speed receive byte clock.
system_rst_out	O	Reset indication due to PLL reset (active-High).
dlyctrl_rdy_out	O	Ready signal output from IDELAYCTRL, stating delay values are adjusted as per VTC changes.
dlyctrl_rdy_in	I	Ready signal input to IDELAYCTRL. Refer to “Include IDELAYCTRL in Core” in the Configuration Tab for more details.
clk_300m	I	300 MHz clock for IDELAYCTRL.
UltraScale+ and Versal Adaptive SoC Shared Logic Outside the Subsystem		
mipi_phy_if	O	D-PHY interface.

Signal Name	Direction	Description
bg<x>_pin<y>_nc	I	Inferred bitslice ports. The core infers bitslice0 of a nibble for strobe propagation within the byte group; <x> indicates byte group (0,1,2,3); <y> indicates bitslice0 position (0 for the lower nibble, 6 for the upper nibble) • RTL Design: There is no need to drive any data on these ports. • IP integrator: These ports must be brought to the top level of the design to apply the constraints correctly. • Pins are available only for AMD UltraScale+ families. • BITSlice defines a single I/O for UltraScale+, NIBBLESlice defines a single I/O for Versal devices.
clkoutphy_in	I	PHY serial clock.
pll_lock_in	I	PLL Lock indication.
rxbyteclkhs	O	PPI high-speed receive byte clock.

Signal Name	Direction	Description
rxbyteclkhs_cnts_in	I	Continuous PPI high-speed receive byte clock for line rates > 1500 Mb/s.
ext_mmcm_clk_in	I	External MMCM clock when selected External MMCM for line rates >1500.
ext_mmcm_lock_in	I	External MMCM lock in when selected External MMCM for line rates > 1500.
ext_mmcm_clk_out	O	MMCM clock out when selected to use MMCM inside subsystem for line rates >1500.
ext_mmcm_lock_out	O	MMCM lock out when selected to use MMCM inside subsystem for line rates > 1500.
Versal Premium Series Gen 2 Outside Subsystem		
mipi_phy_if/mipi_c_phy_if	O	PHY (D-PHY/C-PHY) interface.
shared_rxwordlckhs_in	I	PPI high-speed receive word clock.
cnts_rxworkclkhs_in	I	PPI high-speed receive continuous word clock.
shared_plllocked_in	I	PLL lock indication (active-High).
shared_pll_clkoutphy_in	I	PHY serial clock.
shared_pll_clkoutphy_90_in	I	90 shift of the pll_clkoutphy.

Signal Name	Direction	Description
iophy_intf_rdy	O	Indication of IO PHY reset sequence completion.
Versal Premium Series Gen 2 Inside Subsystem		
mipi_phy_if/mipi_c_phy_if	O	PHY (D-PHY/C-PHY) interface.
clkoutphy_out	O	PHY serial clock.
rxwordlckhs_outt	O	PPI high-speed receive word clock.
cnts_rxworkclkhs_outt	O	PPI high-speed receive continuous word clock.
system_rst_out	O	Reset indication due to PLL reset (active-High).
pll_lock_out	O	PLL lock indication (active-High).
pll_clkoutphy_90_out	O	90 shift of the pll_clkoutphy.
iophy_intf_rdy	O	Indication of IO PHY reset sequence completion.
CSI-2 Controller Register Interface Disabled		
ctrl_core_en	I	Enable the core to receive and process packets.
active_lanes[1:0]	I	Active lanes.
ctrl_dis_in_prgs	O	Indicates the CSI-2 RX Controller core disable is in progress.

Signal Name	Direction	Description
errsotsynchs_intr	O	Interrupt output indicating SoT synchronization completely failed.
errsoths_intr	O	Interrupt output indicating SoT error detected.
cl_stopstate_intr	O	High when clock lane is currently in stop state.
dl<n>_stopstate_intr	O	High when the lane module is currently in stop state.
crc_status_intr	O	High when the computed CRC code is different from the received CRC code.
ecc_status_intr[1:0]	O	Bit 1 – High when ECC syndrome is computed and two bits error is detected in the received packet header. Bit 0: High when ECC syndrome was computed and a single bit error in the packet header was detected and corrected
linebuffer_full	O	High when line buffer is full.

Signal Name	Direction	Description
1. The TDEST value stays constant throughout the line. 2. When the CSI2 Controller Register Interface is not selected, the port is present. When the CSI2 Controller Register Interface is selected, the port is not present. 3. The active-High reset for the MIPI D-PHY core is generated internally by setting the external active-Low reset (<code>video_aresetn</code>) to 0. 4. Each frame start packet with Virtual Channel (VC) identifier is mapped to the first image packet and the first embedded non-image with the corresponding VC. 5. As CRC appears at the end of the MIPI packet, ECC, and CRC are reported ONLY during the last beat of the stream transfer when TLAST and TVALID are asserted. You need to ignore the ECC/CRC reported during other transfer beats. Refer to Interface Debug for more details.		

Register Space

This section details registers available in the MIPI CSI-2 RX Subsystem. The address map is split into following regions:

- MIPI CSI-2 RX Controller core
- MIPI D-PHY/RX C-PHY/D-PHY core

Each IP core is given an address space of 4K. Example offset addresses from the system base address when the MIPI D-PHY/ MIPI PHY registers are enabled are shown in the following table.

Table: Sub-Core Address Offsets

IP Cores	Offset
MIPI CSI-2 RX Controller	0x0_0000
MIPI D-PHY	0x0_1000

MIPI CSI-2 RX Controller Core Registers

The following table specifies the name, address, and description of each firmware addressable register within the MIPI CSI-2 RX controller core.

Table: MIPI CSI-2 RX Controller Core Registers

Address Offset	Register Name	Description
0x00	Refer to Core Configuration Register	Core configuration options
0x04	Protocol Configuration Register	Protocol configuration options
0x08	Reserved	
0x0C	Reserved	
0x10	Core Status Register	Internal status of the core
0x14	Reserved	
0x18	Reserved	
0x1C	Reserved	
0x20	Global Interrupt Enable Register	Global interrupt enable registers
0x24	Interrupt Status Register	Interrupt status register
0x28	Interrupt Enable Register	Interrupt enable register
0x2C	Dynamic VC Selection Register	VC selection register
0x30	Generic Short Packet Register	Short packet data
0x34	VCX Frame Error Register	VCX Frame Error Register
0x38	Reserved	

Address Offset	Register Name	Description
0x3C	Clock Lane Information Register	Clock lane status information
Lane<n> Information Registers		
0x40	Lane0 Information	Lane 0 status information
0x44	Lane1 Information	Lane 1 status information
0x48	Lane2 Information	Lane 2 status information
0x4C	Lane3 Information	Lane 3 status information
0x50	Reserved	
0x54	Reserved	
0x58	Reserved	
0x5C	Reserved	
Image Information 1 Registers (VC0 to VC15) and Image Information 2 Registers (VC0 to VC15)		
0x60	Image Information 1 for VC0	Image information 1 of the current processing packet with VC of 0
0x64	Image Information 2 for VC0	Image information 2 of the current processing packet with VC of 0
0x68	Image Information 1 for VC1	Image information 1 of the current processing packet with VC of 1
0x6C	Image Information 2 for VC1	Image information 2 of the current processing packet with VC of 1

Address Offset	Register Name	Description
0x70	Image Information 1 for VC2	Image information 1 of the current processing packet with VC of 2
0x74	Image Information 2 for VC2	Image information 2 of the current processing packet with VC of 2
0x78	Image Information 1 for VC3	Image information 1 of the current processing packet with VC of 3
0x7C	Image Information 2 for VC3	Image information 2 of the current processing packet with VC of 3
0x80	Image Information 1 for VC4	Image information 1 of the current processing packet with VC of 4
0x84	Image Information 2 for VC4	Image information 2 of the current processing packet with VC of 4
0x88	Image Information 1 for VC5	Image information 1 of the current processing packet with VC of 5
0x8C	Image Information 2 for VC5	Image information 2 of the current processing packet with VC of 5
0x90	Image Information 1 for VC6	Image information 1 of the current processing packet with VC of 6
0x94	Image Information 2 for VC6	Image information 2 of the current processing packet with VC of 6
0x98	Image Information 1 for VC7	Image information 1 of the current processing packet with VC of 7

Address Offset	Register Name	Description
0x9C	Image Information 2 for VC7	Image information 2 of the current processing packet with VC of 7
0xA0	Image Information 1 for VC8	Image information 1 of the current processing packet with VC of 8
0xA4	Image Information 2 for VC8	Image information 2 of the current processing packet with VC of 8
0xA8	Image Information 1 for VC9	Image information 1 of the current processing packet with VC of 9
0xAC	Image Information 2 for VC9	Image information 2 of the current processing packet with VC of 9
0xB0	Image Information 1 for VC10	Image information 1 of the current processing packet with VC of 10
0xB4	Image Information 2 for VC10	Image information 2 of the current processing packet with VC of 10
0xB8	Image Information 1 for VC11	Image information 1 of the current processing packet with VC of 11
0xBC	Image Information 2 for VC11	Image information 2 of the current processing packet with VC of 11
0xC0	Image Information 1 for VC12	Image information 1 of the current processing packet with VC of 12
0xC4	Image Information 2 for VC12	Image information 2 of the current processing packet with VC of 12

Address Offset	Register Name	Description
0xC8	Image Information 1 for VC13	Image information 1 of the current processing packet with VC of 13
0xCC	Image Information 2 for VC13	Image information 2 of the current processing packet with VC of 13
0xD0	Image Information 1 for VC14	Image information 1 of the current processing packet with VC of 14
0xD4	Image Information 2 for VC14	Image information 2 of the current processing packet with VC of 14
0xD8	Image Information 1 for VC15	Image information 1 of the current processing packet with VC of 15
0xDC	Image Information 2 for VC15	Image information 2 of the current processing packet with VC of 15
1. Access type and reset value for all the reserved bits in the registers is read-only with value 0. 2. Register accesses need to be word aligned and there is no support for a write strobe. WSTRB is not used internally. 3. Only the lower 7-bits (6:0) of the read and write address of the AXI4-Lite interface are decoded. This means that accessing address 0x00 and 0x80 results in reading the same address of 0x00. 4. Reads and writes to addresses outside this table do not return an error.		

Core Configuration Register

The Core Configuration register is described in the following table and allows you to enable and disable the MIPI CSI-2 RX Controller core and apply a soft reset during core operation.

Table: Core Configuration Register (0x00)

Bits	Name	Reset Value	Access	Description
31-3	Reserved	N/A	N/A	Reserved

Bits	Name	Reset Value	Access	Description
1	Soft Reset	0x0	R/W	1: Resets the core 0: Takes core out of soft reset All registers reset to their default value except the following:

Bits	Name	Reset Value	Access	Description
				• Soft Reset bit (offset 0x00 bit[1]) • Core Enable bit (offset 0x00 bit[0]) • Active Lanes Configuration bit (offset 0x04 bit[1:0]) • Global Interrupt Enable Register (offset 0x20) • Interrupt Enable Register (offset 0x28) When this bit is set to 1, the following also occurs:

Bits	Name	Reset Value	Access	Description
				• Shut down port is not asserted on the PPI lanes • Internal FIFOs (PPI, Packet, Generic Short Packet) are flushed • Control Finite State Machine (FSM) stops processing current packet. Any partially written packet to memory is marked as errored. This packet, when made

Bits	Name	Reset Value	Access	Description
				available through the AXI4-Stream interface , reports the error on TUSER[1] .

Bits	Name	Reset Value	Access	Description
2 ^{2 3}	Full Reset	0x0	R/W	Full Reset, that is, when asserted, it resets the PHY controller and the CSI2 Controller. 1: Resets the core 0: Takes core out of full reset All registers reset to their default value except the following:

Bits	Name	Reset Value	Access	Description
				• Soft Reset bit (offset 0x00 bit[1]) • Core Enable bit (offset 0x00 bit[0]) • Active Lanes Configuration bit (offset 0x04 bit[1:0]) • Global Interrupt Enable Register (offset 0x20) • Interrupt Enable Register (offset 0x28) When this bit is set to 1, the following also occurs:

Bits	Name	Reset Value	Access	Description
				• Shut down port is not asserted on the PPI lanes • Internal FIFOs (PPI, Packet, Generic Short Packet) are flushed • Control finite state machine (FSM) stops processing current packet. Any partially written packet to memory is marked as errored. This packet, when made

Bits	Name	Reset Value	Access	Description
				available through the AXI4-Stream interface , reports the error on TUSER[1] .

Bits	Name	Reset Value	Access	Description
0	Core Enable	0x1	R/W	1: Enables the core to receive and process packets 0: Disables the core for operation When disabled:

Bits	Name	Reset Value	Access	Description
				- Shuts down port assertion on the PPI lanes - Internal FIFOs (PPI, Packet, Generic Short Packet) are flushed - Control FSM stops processing current packet Any partially written packet to memory is marked as errored. This packet, when made available through the AXI4–

Bits	Name	Reset Value	Access	Description
				Stream interface, reports the error on TUSER[1].
1. The short packet and line buffer FIFO full conditions take a few clocks to reflect in the register clock domain from the core clock domain due to Clock Domain Crossing (CDC) blocks. 2. This bit applies only for Versal Prime, Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2. 3. On the system bring up, if ISR is not clean, assert this bit.				

Protocol Configuration Register

The following table describes the Protocol Configuration register and allows you to configure protocol specific options such as the number of lanes to be used.

Table: Protocol Configuration Register (0x04)

Bits	Name	Reset Value	Access	Description
31–5	Reserved	N/A	N/A	Reserved
4–3	Maximum Lanes	Number of lanes configured during core generation	R	Maximum lanes of the core 0x0—1 Lane 0x1—2 Lanes 0x2—3 Lanes 0x3—4 Lanes
2	Reserved	N/A		Reserved

Bits	Name	Reset Value	Access	Description
1–0	Active Lanes	Number of lanes configured during core generation	R/W	Active lanes in the core 0x0—1 Lane 0x1—2 Lanes 0x2 —3 Lanes 0x3—4 Lanes
1. Maximum Lanes cannot exceed the number of lanes as set by the Serial Data Lanes parameter at generation time. 2. A read from this register reflects the current number of lanes being used by core. This is useful when dynamically updating the active lanes during core operation to ensure that the core is using the new active lanes information. See Designing with the Subsystem for more information. 3. Active Lanes cannot exceed the Maximum Lanes as set in the Protocol Configuration register setting of bits 4–3.				

Core Status Register

The following table describes the Core Status register.

Table: Core Status Register (0x10)

Bits	Name	Reset Value	Access	Description
------	------	-------------	--------	-------------

Bits	Name	Reset Value	Access	Description
31-16	Packet Count	0x0	R	Counts number of long packets written to the line buffer • Packet count rolls over from 0xFFFF to 0x0000. The roll over of this counter is not reported . • Count includes error packets (if any)
15-4	Reserved	N/A	N/A	N/A
3	Short packet FIFO Full	0x0	R	Indicates the current status of short packet FIFO full condition

Bits	Name	Reset Value	Access	Description
2	Short packet FIFO not empty	0x0	R	FIFO not empty: Indicates the current status of short packet FIFO not empty condition
1	Stream Line buffer Full	0x0	R	Indicates the current status of line buffer full condition
0	Soft reset/Core disable in progress	0x0	R	Set to 1 by the core to indicate that internal soft reset/core disable activities are in progress

Global Interrupt Enable Register

The following table describes the Global Interrupt Enable register.

Table: Global Interrupt Enable Register (0x20)

Bits	Name	Reset Value	Access	Description
31-1	Reserved	N/A	N/A	Reserved

Bits	Name	Reset Value	Access	Description
0	Global Interrupt enable	0x0	R/W	Master enable for the device interrupt output to the system 1: Enabled—the corresponding Interrupt Enable register (IER) bits are used to generate interrupts 0: Disabled—Interrupt generation blocked irrespective of IER bits

Interrupt Status Register

The following table describes the Interrupt Status register (ISR) and captures the error and status information for the core.

Table: Interrupt Status Register (0x24)

Bits	Name	Reset Value	Access	Description
			1	

Bits	Name	Reset Value	Access 1	Description
31	Frame Received	0x0	R/W1C	Asserts when the Frame End (FE) short packet is received for the current frame.
30	VCX Frame Error	0x0	R/W1C	Asserts when the VCX Frame error is detected.
29	RX_Skewcalhs	0x0	R/W1C	Asserts when rxskewcalhs is detected.
28	YUV420 WC Error	0x0	R/W1C	Asserts when the user-configured YUV420 word count value is less than the actual Y-line word count of the incoming data, which results in an internal buffer full condition. 2

Bits	Name	Reset Value	Access 1	Description
27	Pending write in internal FIFO	0x0	R/W1C	Indicates internal FIFO has insufficient clocks to process the packet. This happens in non-continuous clock mode if there are insufficient clocks to completely write the PPI data to internal FIFO. To resolve this, the user needs to increase the Tclk-post parameter in the transmitter. In case this bit is seen set by default, write 1 to clear this bit after the rxbyteclkhs starts toggling.
26-23	Reserved	N/A	N/A	N/A

Bits	Name	Reset Value	Access 1	Description
22	Word Count (WC) corruption	0x0	R/W1C	Asserts when WC field of packet header corrupted and core receives less bytes than indicated in WC field. Such a case can occur only where more than 2-bits of header are corrupted which ECC algorithm cannot report and the corruption is such that the ECC algorithm reports a higher Word Count (WC) value as part of ECC correction. In this case, the core limits processing of the packet on reduced number of bytes

Bits	Name	Reset Value	Access 1	Description
				received through PPI interface.
21	Incorrect lane configuration	0x0	R/W1C	Asserts when Active lanes is greater than Maximum lanes in the protocol configuration register
20	Short packet FIFO full	0x0	R/W1C	Active-High signal asserted when the short packet FIFO full condition detected
19	Short packet FIFO not empty	0x0	R/W1C	Active-High signal asserted when short packet FIFO not empty condition detected.
18	Stream line buffer full	0x0	R/W1C	Asserts when the line buffer is full ³

Bits	Name	Reset Value	Access 1	Description
17	Stop state	0x0	R/W1C	Active-High signal indicates that the lane module is currently in Stop state. ⁴
16	Reserved	N/A	N/A	N/A
15	Reserved	N/A	N/A	N/A
14	InValidcodehs	0x0	R/W1C	In case of C-PHY Indicate High-Speed Invalid Code Word Detected.
13	SoT error (ErrSoTHS)	0x0	R/W1C	Indicates Start-of-Transmission (SoT) error detected. ⁴
12	SoT sync error (ErrSotSyncHS)	0x0	R/W1C	Indicates SoT synchronization completely failed. ⁴

Bits	Name	Reset Value	Access 1	Description
11	ECC-2 bit Error/CRC Header Error all	0x0	R/W1C	Asserts when an ECC syndrome is computed and two bit errors detected in the received packet header in C-PHY. In case of C-PHY, CRC header check fails for all header copy.
10	ECC 1-bit error/CRC Header Error (Detected and Corrected) (ErrEccCorrected)	0x0	R/W1C	Asserts when an ECC syndrome was computed and a single bit error in the packet header was detected and corrected in D-PHY. In case of C-PHY, the CRC header check passes for at least one header copy.

Bits	Name	Reset Value	Access 1	Description
9	CRC error (ErrCrc)	0x0	R/W1C	Asserts when the computed CRC code is different from the received CRC code.
8	Unsupported Data Type (ErrID)	0x0	R/W1C	Asserts when a packet header is decoded with an unrecognized or not implemented data ID.
7	Frame synchronization error for VC3 (ErrFrameSync)	0x0	R/W1C	Asserts when an FE is not paired with a Frame Start (FS) on the same virtual channel. ⁴

Bits	Name	Reset Value	Access 1	Description
6	Frame level error for VC3 (ErrFrameData)	0x0	R/W1C	Asserts after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.
5	Frame synchronization error for VC2 (ErrFrameSync)	0x0	R/W1C	Asserts when an FE is not paired with an FS on the same virtual channel. ⁴
4	Frame level error for VC2 (ErrFrameData)	0x0	R/W1C	Asserts after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.

Bits	Name	Reset Value	Access 1	Description
3	Frame synchronizati on error for VC1 (ErrFrameSyn c)	0x0	R/W1C	Asserts when an FE is not paired with an FS on the same virtual channel. ⁴
2	Frame level error for VC1 (ErrFrameDat a)	0x0	R/W1C	Asserts after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.
1	Frame synchronizati on error for VC0 (ErrFrameSyn c)	0x0	R/W1C	Asserts when a FE is not paired with an FS on the same virtual channel. ⁴

Bits	Name	Reset Value	Access 1	Description
0	Frame level error for VC0 (ErrFrameData)	0x0	R/W1C	Asserts after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.

Bits	Name	Reset Value	Access 1	Description
1. W1C = Write 1 to clear. 2. Ensure the Y-line word count of the incoming data is at least 8 bytes less than the value configured in GUI to avoid this flag. 3. In a line buffer full condition, reset the core using the external reset, video_aresetn. 4. Reported through the PPI. 5. An ErrSotSynchS error also generates this error signal. 6. Short packet and line buffer FIFO full conditions take a few clock periods to reflect in the register clock domain from the core clock domain due to Clock Domain Crossing (CDC) blocks. 7. All PPI signals captured in the ISR take a few clock periods to reflect in the register clock domain from the PPI clock domain due to CDC blocks. 8. Frame level errors due to ErrSotSynchS are mapped to the recent VC processed by the ECC block of the core. 9. Set conditions take priority over the reset conditions for the ISR bits. 10. Signal names in brackets are defined in the MIPI Alliance Standard for Camera Serial Interface CSI-2. 11. ISR ECC still is reported in scenarios where the packet is dropped. 12. For YUV420 data, if any Virtual Channel (VC) experiences corruption in either the even or odd line, the corrupted data propagates to the video output. To mitigate this, monitor the word count at the TUSER signal, and apply a reset to reinitialize and receive a fresh data packet. An ECC (Error-Correcting Code) 2-bit error in the even or odd line is not recoverable. This unrecoverable error can lead to the corruption of critical metadata, such as the data type, word count, or VC identifier. Consequently, such corruption can cause a cascading (domino) effect, resulting in all subsequent packets associated with the affected VC being corrupted.				

The following table provides detailed information about the bits in the previous table.

Table: Incorrect Lane Configuration

Field	Description
-------	-------------

Field	Description
Set Condition(s)	Set by the core when incorrect lane configuration is programmed. For example, maximum available lanes =2 and “Active lanes” configured as 3.
Reset Sequence	Write 1 to clear this bit.
Priority	Set condition takes priority over reset sequence.
Impact	This is a core configuration error and the core cannot function as required. Correct this error before proceeding further.

Table: Stream Line Buffer Full

Field	Description
Set Condition(s)	Set by the core when the line buffer storing pixel data is full.
Reset Sequence	Write 1 to clear this bit.
Priority	Set condition takes priority over reset sequence.
Impact	Core reports this condition on stream interface using an error indication on the TUSER[1] port if a partial packet is being written to line buffer. Because PPI does not allow back pressure, you need to ensure that this condition does not occur.

Table: Control Error, Escape Entry Error, Escape Ultra Low Power Mode, Stopstate

Field	Description
-------	-------------

Field	Description
Set Condition(s)	Set by the core when the condition for the corresponding signal as defined in the MIPI CSI-2 specification See MIPI Alliance Standard for Camera Serial Interface CSI-2: https://www.mipi.org/specifications/csi-2 is seen, reported through the PPI interface.
Reset Sequence	Write 1 to clear this bit.
Priority	Set condition takes priority over reset sequence.
Impact	Current packet being processed does not have any impact.

Table: Invalidcodehs Error

Field	Description
Set Condition(s)	Set by the core when the current packet being processed has High-Speed Invalid Code Word Detection Error reported through PPI Interface. This indicates that the data word being output has a low confidence of being correct.
Reset Sequence	Write 1 to clear this bit.
Priority	Set condition takes priority over reset sequence.
Impact	The current packet being processed is not processed further. The core waits for the next packet to process. The current packet results in a CRC mismatch.

Table: SoT Error

Field	Description
-------	-------------

Field	Description
Set Condition(s)	Set by the core when the current packet being processed has Start-of-Transmission (SoT) Error reported through PPI Interface.
Reset Sequence	Write 1 to clear this bit.
Priority	Set condition takes priority over reset sequence.
Impact	Current packet under process does not have any impact as synchronization is still achieved. This is considered a “soft error” in the leader sequence and confidence in the payload data is reduced.

Table: SoT Sync Error

Field	Description
Set Condition(s)	Set by the core when current packet being processed has Start-of-Transmission Synchronization Error reported through PPI interface.
Reset Sequence	Write 1 to clear this bit.
Priority	Set condition takes priority over reset sequence.
Impact	The current packet being processed is not processed further. The core waits for the next packet to process.

Table: ECC 2-Bit Error

Field	Description
Set Condition(s)	Set by the core when an ECC syndrome was computed and two bit-errors are detected in the received Packet Header.

Field	Description
Reset Sequence	Write 1 to clear this bit.
Priority	Set condition takes priority over reset sequence.
Impact	Current packet being processed is not processed further as WC is not usable, and thus the packet end cannot be estimated. The core waits for the next packet to process.

Table: CRC Header Error All

Field	Description
Set Condition(s)	Set by the core when an all C-PHY CRC header syndrome was computed and CRC error is detected in the received Packet Header.
Reset Sequence	Write 1 to clear this bit.
Priority	Set condition takes priority over reset sequence.
Impact	Current packet being processed is not processed further as WC is not usable, and thus the packet end cannot be estimated. The core waits for the next packet to process.

Table: ECC 1-Bit Error

Field	Description
Set Condition(s)	Set by the core when an ECC syndrome was computed and a single bit-error in the Packet Header was detected and corrected.
Reset Sequence	Write 1 to clear this bit.

Field	Description
Priority	Set condition takes priority over reset sequence.
Impact	Current packet being processed does not have any impact.

Table: CRC Header Error

Field	Description
Set Condition(s)	Set by the core when at least one CPHY Header CRC syndrome was computed and the error in the Packet Header was detected and at least one copy detected no error.
Reset Sequence	Write 1 to clear this bit.
Priority	Priority Set condition takes priority over reset sequence.
Impact	Current packet being processed does not have any impact.

Table: CRC Error

Field	Description
Set Condition(s)	Set by the core when the computed CRC code is different than the received CRC code.
Reset Sequence	Write 1 to clear this bit.
Priority	Set condition takes priority over reset sequence.
Impact	Current packet being processed does not have any impact, but the payload might be corrupted.

Table: Unsupported Data Type

Field	Description
-------	-------------

Field	Description
Set Condition(s)	Set by the core when a Packet Header is decoded with an unrecognized or unimplemented data ID.
Reset Sequence	Write 1 to clear this bit.
Priority	Set condition takes priority over reset sequence.
Impact	Current packet being processed is not processed further. The core waits for the next packet to process.

Table: Frame Synchronization Error

Field	Description
Set Condition(s)	Set by the core when a Frame End (FE) is not paired with a Frame Start (FS) on the same virtual channel. An ErrSotSyncHS also generates this error signal.
Reset Sequence	Write 1 to clear this bit.
Priority	Set condition takes priority over reset sequence.
Impact	Based on the different sources for this error packet might or might not be processed (that is, stored in the line buffer).

Table: ErrFrameSync Sources

Source	Impact on Packet Processing
Source	Impact on Packet Processing
FS followed by FS	Processed
ErrEccDouble	Not processed
FE followed FE	Processed

Source	Impact on Packet Processing
ErrSotSyncHS	Not processed

Table: Header Error All

Field	Description
Set Condition(s)	Set by the core when an CRC syndrome was computed for all CRC Headercopy and CRC check failed for all the copies in the received Packet Header.
Reset Sequence	Write 1 to clear this bit.
Priority Set Condition	Takes priority over reset sequence. Impact Current packet being processed is not processed further as WC is not usable, and thus the packet end cannot be estimated. The core waits for the next packet to process.

Table: CRC Header 1

Field	Description
Set Condition(s)	When an CRC syndrome was computed for all CRC Header copy and CRC check is passed at-least for one the copy in the received Packet Header.
Reset Sequence	Write 1 to clear this bit.
Priority Set Condition	Takes priority over reset sequence. Impact Current packet being processed does not have any impact.

VC Mapping

In the event of an `ErrEccDouble` error, the VC is mapped to the VC reported in the current packet header (even if corrupted).

In the event of an `ErrSotSyncHS` error, the VC is mapped to the previous VC

processed because, in this case, the packet header is not available.

Table: Frame Level Error

Field	Description
Set Condition(s)	Set by the core after an FE when the data payload received between FS and FE contains errors.
Reset Sequence	Write 1 to clear this bit.
Priority	Set condition takes priority over reset sequence.
Impact	Current packet being processed does not have any impact but the payload might be corrupted.

Interrupt Enable Register

The following table describes the Interrupt Enable register (IER) and allows you to selectively generate an interrupt at the output port for each error/status bit in the ISR. An IER bit set to 0 does not inhibit an error/status condition from being captured, but inhibits it from generating an interrupt.

Table: Interrupt Enable Register (0x28)

Bits	Name	Reset Value	Access	Description
------	------	-------------	--------	-------------

Bits	Name	Reset Value	Access	Description
31	Frame Received	0x0	R/W	Set bits in this register to 1 to generate the required interrupts. Set to 0 to disable the interrupt. For a description of the specific interrupt you are enabling/disabling in this register, refer to the ISR descriptions in Interrupt Status Register .
30	VCX Frame Error	0x0	R/W	Set to 1 to generate the VCX Frame Error interrupt.
29	RX_Skewcalhs	0x0	R/W	Set to 1 to generate the rxskecalhs interrupt.
28	YUV420 WC Error	0x0	R/W	Set to 1 to generate the YUV420 WC Error interrupt.

Bits	Name	Reset Value	Access	Description
27	Pending write in internal FIFO	0x0	R/W	Set bits in this register to 1 to generate the required interrupts. Set to 0 to disable the interrupt. For a description of the specific interrupt you are enabling/disabling in this register, refer to the ISR descriptions in Interrupt Status Register .
26-23	Reserved	N/A	N/A	
22	Word Count (WC) corruption	0x0	R/W	
21	Incorrect lane configuration	0x0	R/W	
20	Short packet FIFO full	0x0	R/W	
19	Short packet FIFO empty	0x0	R/W	
18	Stream line buffer full	0x0	R/W	
17	Stop state	0x0	R/W	
16	Reserved	N/A	N/A	
15	Reserved	N/A	N/A	
14	Invalidcodehs	0x0	R/W	
13	SoT error	0x0	R/W	
12	SoT Sync error	0x0	R/W	
11	ECC 2-bit error/CPHY Header Error All	0x0	R/W	
10	ECC 1-bit Error/CPHY Header Error	0x0	R/W	
9	CRC error	0x0	R/W	

Bits	Name	Reset Value	Access	Description
8	Unsupported Data Type	0x0	R/W	
7	Frame synchronization error for VC3	0x0	R/W	
6	Frame level error for VC3	0x0	R/W	
5	Frame synchronization error for VC2	0x0	R/W	
4	Frame level error for VC2	0x0	R/W	
3	Frame synchronization error for VC1	0x0	R/W	
2	Frame level error for VC1	0x0	R/W	
1	Frame synchronization error for VC0	0x0	R/W	
0	Frame level error for VC0	0x0	R/W	

Dynamic VC Selection Register

The following table describes the VC Selection register which allows you to dynamically select the VC packets to be processed by the core. You need to set the Allowed VC GUI setting to All to use this feature. Packets that are filtered as part of this VC selection is not counted in Core status register field- “Packet Count.”

“Frame received” bit in Interrupt Status Register is not set if VC disabling occurs before the “Frame End” packet is received.

Table: VC Selection Register (0x2C)

Bits	Name	Reset Value	Access	Description
31:16	Reserved	N/A	N/A	Reserved
15:0	VC Selection	0xFFFF	R/W	1: Core processes packets with this VC. 0: Core filters packets with this VC. For example: VC Selection = 0x000E means all packets with VC=3,2,1 are processed. Other packets are filtered out. Bits 15:4 are used only when "Support VCX Feature" is enabled. Writing 1 or 0 to 15:4 when "Support VCX Feature" is not enabled does not have any impact.

Generic Short Packet Register

The following table describes the Generic Short Packet register. Packets received with generic short packet codes are stored in a 31–deep internal FIFO and are made available through this register. The following conditions reset the FIFO:

- External reset on `video_aresetn`
- Core disable or soft reset through register settings.

Note:

1. If one-bit error occurs during data-transmission, the MIPI CSI-2 controller fixes the error-bit and stores generic short packet data into the FIFO.
 2. When a short packet is received with a 2-bit error, the MIPI CSI-2 controller discards the data without pushing the data to the FIFO.
 3. Because the data field of the register is only 16 bits wide, the ECC information is not stored.
-

Table: Generic Short Packet Register (0x30)

Bits	Name	Reset Value	Access	Description
24–29	Reserved	N/A	N/A	Reserved
30–31	Reserved	N/A	N/A	Extra 1 bit VC support (VC >4)
23–8	Data	0x0	R	16-bit short packet data
7–6	Virtual Channel	0x0	R	Virtual channel number
5–0	Data Type	0x0	R	Generic short packet code

VCX Frame Error Register

The following table describes the VCX Frame Error register. It captures the frame level and frame synchronization errors for the VC extension channels.

Table: VCX Frame Error (0x34)

Bits	Name	Reset Value	Access	Description
31-24	Reserved	N/A	N/A	Reserved
23	Frame synchronizati on error for VC15	0x0	R/W1C	Asserted when an FE is not paired with a Frame Start (FS) on the same virtual channel.
22	Frame level error for VC15	0x0	R/W1C	Asserted after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.
21	Frame synchronizati on error for VC14	0x0	R/W1C	Asserted when an FE is not paired with a Frame Start (FS) on the same virtual channel.

Bits	Name	Reset Value	Access	Description
20	Frame level error for VC14	0x0	R/W1C	Asserted after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.
19	Frame synchronization error for VC13	0x0	R/W1C	Asserted when an FE is not paired with a Frame Start (FS) on the same virtual channel.
18	Frame level error for VC13	0x0	R/W1C	Asserted after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.

Bits	Name	Reset Value	Access	Description
17	Frame synchronizati on error for VC12	0x0	R/W1C	Asserted when an FE is not paired with a Frame Start (FS) on the same virtual channel.
16	Frame level error for VC12	0x0	R/W1C	Asserted after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.
15	Frame synchronizati on error for VC11	0x0	R/W1C	Asserted when an FE is not paired with a Frame Start (FS) on the same virtual channel.

Bits	Name	Reset Value	Access	Description
14	Frame level error for VC11	0x0	R/W1C	Asserted after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.
13	Frame synchronization error for VC10	0x0	R/W1C	Asserted when an FE is not paired with a Frame Start (FS) on the same virtual channel.
12	Frame level error for VC10	0x0	R/W1C	Asserted after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.

Bits	Name	Reset Value	Access	Description
11	Frame synchronization error for VC9	0x0	R/W1C	Asserted when an FE is not paired with a Frame Start (FS) on the same virtual channel.
10	Frame level error for VC9	0x0	R/W1C	Asserted after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.
9	Frame synchronization error for VC8	0x0	R/W1C	Asserted when an FE is not paired with a Frame Start (FS) on the same virtual channel.

Bits	Name	Reset Value	Access	Description
8	Frame level error for VC8	0x0	R/W1C	Asserted after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.
7	Frame synchronization error for VC7	0x0	R/W1C	Asserted when an FE is not paired with a Frame Start (FS) on the same virtual channel.
6	Frame level error for VC7	0x0	R/W1C	Asserted after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.

Bits	Name	Reset Value	Access	Description
5	Frame synchronization error for VC6	0x0	R/W1C	Asserted when an FE is not paired with a Frame Start (FS) on the same virtual channel.
4	Frame level error for VC6	0x0	R/W1C	Asserted after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.
3	Frame synchronization error for VC5	0x0	R/W1C	Asserted when an FE is not paired with a Frame Start (FS) on the same virtual channel.

Bits	Name	Reset Value	Access	Description
2	Frame level error for VC5	0x0	R/W1C	Asserted after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.
1	Frame synchronization error for VC4	0x0	R/W1C	Asserted when an FE is not paired with a Frame Start (FS) on the same virtual channel.
0	Frame level error for VC4	0x0	R/W1C	Asserted after an FE when the data payload received between FS and FE contains errors. The data payload errors are CRC errors.

Clock Lane Information Register

The following table describes the Clock Lane Information register. The Stop state is captured in this register.

Table: Clock Lane Information Register (0x3C)

Bits	Name	Reset Value	Access	Description
31–2	Reserved	N/A	N/A	Reserved
1	Stop state	0x0	R	Stop state on clock lane
0	Reserved	N/A	N/A	Reserved

Lane<n> Information Registers

The following table describes Lane<n> Information register, where n is 0, 1, 2, or 3 and provides the status of the <n> data lane. This register resets when any write to the Protocol Configuration register is detected, irrespective of whether the Protocol Configuration register contents are updated or not.

Table: Lane 0, 1, 2, 3 Information Register (0x40, 0x44, 0x48, 0x4C)

Bits	Name	Reset Value	Access	Description 2
31–6	Reserved	N/A	N/A	Reserved
5	Stop state	0x0	RO	Detection of stop state
4	Reserved	N/A	N/A	Reserved
3	Invalidcodehs	0x0	R	Indicates the invalidcode reception
2	skewcalhs	0x0	R	Indicates the deskew reception
1	SoT error	0x0	R	Detection of SoT Error
0	SoT Sync error	0x0	R	Detection of SoT Synchronization Error

Bits	Name	Reset Value	Access	Description
1. Lane Information registers are present only for the maximum defined number of lanes. Reads to others registers gives 0x0. 2. All bits are reported through the PPI.				

Image Information 1 Registers (VC0 to VC15)

The following table describes the Image Information 1 registers and provide image information for line count and byte count per VC. The byte count gets updated whenever a long packet (from Data Types 0x18 and above) for the corresponding virtual channel is processed by the control FSM. The line count is updated whenever the packet is written into the line buffer.

Table: Image Information 1 Registers (0x60, 0x68, 0x70, 0x78)

Bits	Name	Reset Value	Access	Description
31-16	Line count	0x0	R	Number of long packets written to line buffer
15-0	Byte count	0x0	R	Byte count of current packet being processed by the control FSM

Image Information 2 Registers (VC0 to VC15)

The following table describes the Image Information 2 registers and provide the image information Data Type. The Data Type is updated whenever a long packet (Data Types 0x18 and above) for the corresponding virtual channel is processed by the control FSM.

Table: Image Information 2 Registers (0x64, 0x6C, 0x74, 0x7C)

Bits	Name	Reset Value	Access	Description
------	------	-------------	--------	-------------

Bits	Name	Reset Value	Access	Description
31–6	Reserved	N/A	N/A	Reserved
5–0	Data Type	0x0	R	Data Type of current packet being processed by control FSM

MIPI D-PHY Registers

The MIPI D-PHY registers are available when the D-PHY Register Interface is selected in Vivado IDE. For details about MIPI D-PHY registers, refer to the MIPI D-PHY LogiCORE IP Product Guide ([PG202](#)).

MIPI RX PHY Registers

The MIPI RX PHY registers are available when the PHY Register Interface is selected in Vivado IDE. For details about MIPI RX PHY registers, refer to the MIPI RX C-PHY/D-PHY LogiCORE IP Product Guide ([PG435](#)).

Designing with the Subsystem

This chapter includes guidelines and additional information to facilitate designing with the subsystem.

General Design Guidelines

The subsystem fits into an image sensor pipe receive path. The input to the subsystem must be connected to an image source such as an image sensor transmitting data that adheres to the MIPI protocol. The output of the subsystem is image data in AXI4-Stream format. Based on the throughput requirement the output interface can be tuned using customization parameters available for the subsystem.

Because the MIPI protocol does not allow throttling on the input interface, the module connected to the output of this subsystem needs to have sufficient

bandwidth for the data generated by the image sensor.

The Protocol Configuration Register [1:0] can be used to dynamically configure the active lanes used by the subsystem using the following guidelines:

1. Program the required lanes in the Protocol Configuration register (only allowed when “Enable Active Lanes” is set in the AMD Vivado™ IDE).
2. The subsystem internally updates the new lanes information after the current packet complete indication is seen (that is, when the current active lanes indicate a Stop state condition) and a subsequent `RxByteClkHS` signal is seen on the PPI.
3. A read from the Protocol Configuration register reflects the new value after the subsystem has successfully updated the new lanes information internally.
4. Do not send the new updated lanes traffic until the read from Protocol Configuration registers reflects the new value.
5. Initialize all MIPI interfaces in the same HP I/O bank at the same time. For example, multiple CSI-2 or D-PHY instances in a system. For more information on implementing multiple interfaces in the same HP I/O bank, refer to the UltraScale Architecture SelectIO Resources User Guide ([UG571](#)).

Note: When multiple D-PHY IPs are placed in the same bank, they need to be reset at the same time even if they are independent.

Shared Logic

Shared Logic provides a flexible architecture that works both as a stand-alone subsystem and as part of a larger design with one of more subsystem instances. This minimizes the amount of HDL modifications required, but at the same time retains the flexibility of the subsystem.

Shared logic in the CSI-2 RX Subsystem allows you to share PLLs with multiple instances of the CSI-2 RX Subsystem within the same I/O bank.

There is a level of hierarchy called `<component_name>_support`. The following figures show two hierarchies where the shared logic is either contained in the subsystem or in the example design. In these figures, `<component_name>` is the name of the generated subsystem. The difference between the two hierarchies is the boundary of the subsystem. It is controlled using the Shared Logic option in the Vivado IDE Shared Logic tab for the MIPI CSI-2 RX Subsystem. The shared logic comprises a PLL and some BUFGs (maximum of four).

Figure: Shared Logic Included in the Subsystem

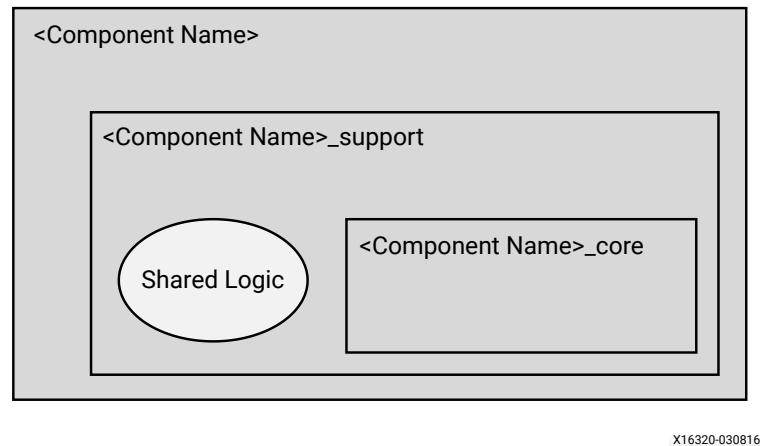
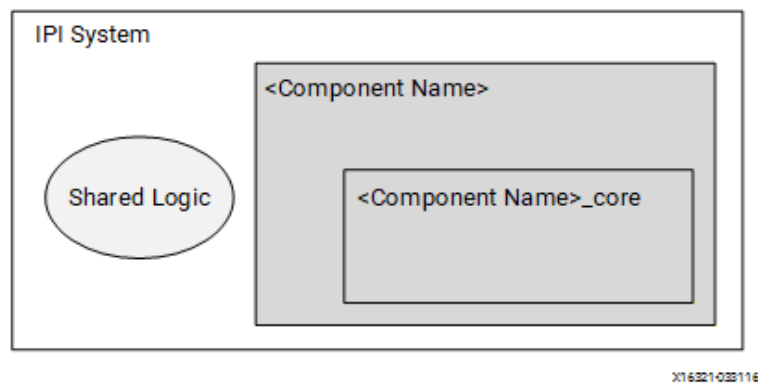


Figure: Shared Logic Outside the Subsystem



Shared Logic in the Subsystem

Selecting Shared Logic in the Core implements the subsystem with the PLL inside the subsystem to generate all the clocking requirement of the PHY layer.

Select Include Shared Logic in Core if:

- You do not require direct control over the PLL generated clocks
- You want to manage multiple customizations of the subsystem for multi-subsystem designs
- This is the first MIPI CSI-2 RX Subsystem in a multi-subsystem system

These components are included in the subsystem, and their output ports are also provided as subsystem outputs.

Shared logic in subsystem is active and running at all times if the clocks are shared by other CSI subsystems in the system.

MMCM Sharing

For Line Rates > 1500 and when the Include Shared logic in the core is selected, there is a provision given to have the MMCM outside the core (required clock inputs must be driven to the core) by selecting the External MMCM in GUI for MMCM selection. You can use the same MMCM across different banks.

Selecting Internal MMCM implements MMCM+PLL combination in the core and the clocks can be share within the Bank only.

Shared Logic Outside the Subsystem

The PLLs are outside this subsystem instance.

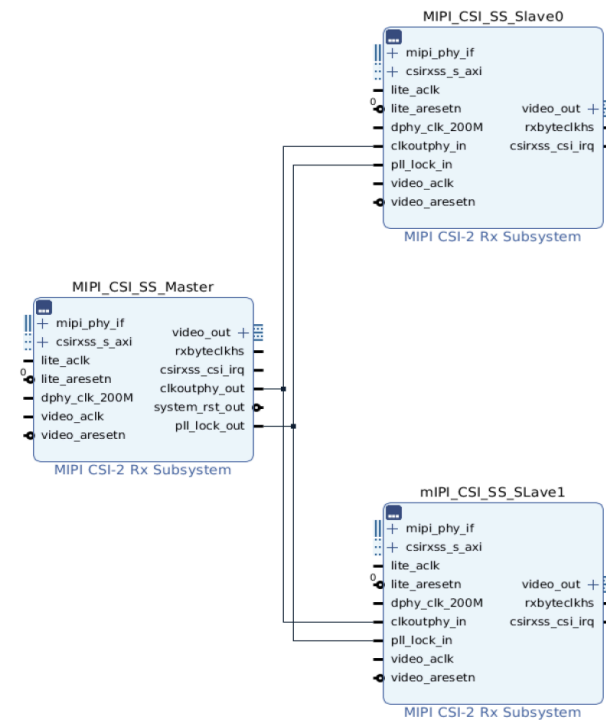
Select Include Shared Logic in example design if this is not the first MIPI CSI-2 RX Subsystem instance in a multi-subsystem design that shares PLLs generated from other MIPI CSI-2 RX Subsystem that is configured with shared logic in the Core mode.

To fully use the PLL, customize one MIPI CSI-2 RX Subsystem with shared logic in the subsystem and one with shared logic in the example design. You can connect the PLL outputs from the first MIPI CSI-2 RX Subsystem to the second subsystem.

There needs to be at least one MIPI CSI-2 RX Subsystem with Include shared Logic in the Core mode whose outputs for shared resources can be used in other MIPI CSI-2 RX Subsystem generated with Include shared logic in example design mode.

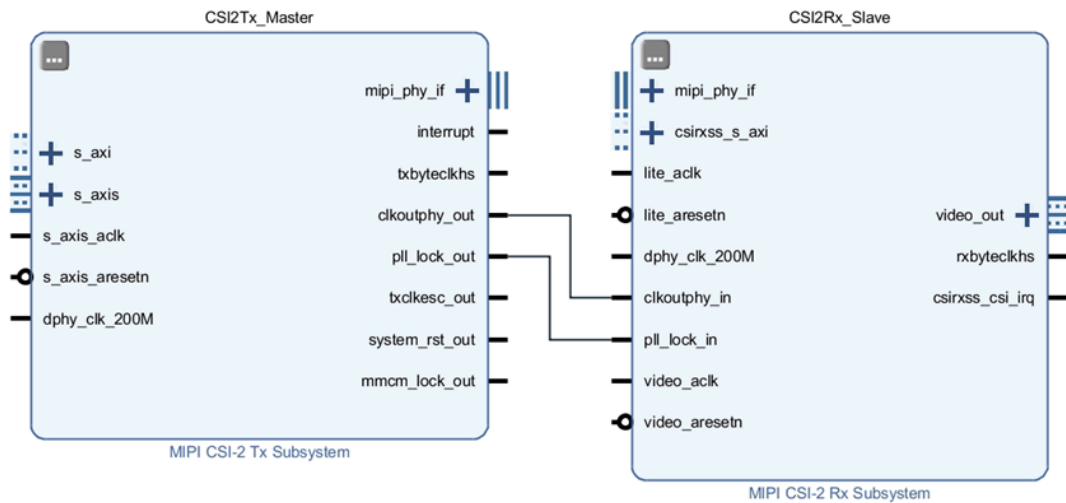
The following figure shows the sharable resource connections from the MIPI CSI-2 RX Subsystem with shared logic included (MIPI_CSI_SS_Master) to the instance of another MIPI CSI-2 RX Subsystem without shared logic (MIPI_CSI_SS_Slave00 and MIPI_CSI_SS_Slave01) for AMD UltraScale+™ devices.

Figure: Shared Logic in the Example Design



Note: The master and slave cores can be configured with different line rates when operating at less than 1500 Mb/s. However, when operating at greater than 1500 Mb/s with the Deskew detection feature enabled, the master and slave cores need to be configured with the same line rate when sharing `clkoutphy` within the IO bank. There must be at least one core in master mode in a system whose clocks can be shared with slave mode cores.

Figure: Clock Sharing in MIPI CSI-2 TX and MIPI CSI-2 RX Subsystems



Important: MIPI CSI-2 TX Subsystem and MIPI CSI-2 RX Subsystem share clocking resources, in such scenario MIPI CSI-2 TX Subsystem need to be configured using Include Shared Logic in Core option under Shared Logic tab. However, for line rates > 1500, MIPI CSI-2 RX Subsystem has additional ports called "rxbyteclkhs_cnts_in/rxbyteclkhs_cnts_out." Hence, MIPI CSI-2 TX Subsystem and MIPI CSI-2 RX Subsystem cannot share clocking resources for cases of line rates > 1500.

Important: For line rates ≤ 1500, the master and slave can be configured with the different line rate when sharing `clkoutphy` within IO bank.

Important: For line rates > 1500, the controller and the target needs to be configured with the same line rate when sharing `clkoutphy` within the IO bank.

I/O Planning

The MIPI CSI-2 RX Subsystem provides an I/O planner feature for I/O selection. In the Pin Assignment tab, dedicated byte clocks (DBC) or quad byte clocks (QBC) are listed for the clock lane for the selected HP I/O bank. For the QBC clock lane, all the I/O pins are listed for data lane I/O selection, but for the DBC clock lane only, the byte group I/O pins are listed for data lane I/O selection.

Eight MIPI CSI-2 RX Subsystem IP cores can be implemented per IO bank based on BITSlice and BITSlice_CONTROL instances in the UltraScale+ devices.

Note: If the RX data lane I/O pins are selected non-contiguously then an additional one, two, or three I/O pins (RX_BITSLICE) are automatically used for clock/Strobe propagation. Therefore, it is recommended that you select adjacent I/O pins for the RX configuration to make efficient use of the I/O. The propagation of strobes to the RX data pins follows the inter-byte and inter-nibble clocking rules given in the UltraScale Architecture SelectIO Resources User Guide ([UG571](#)). All lanes of a particular MIPI CSI-2 RX Subsystem instance need to be in the same HP I/O bank which is automatically controlled by the Pin Assignment tab of the XGUI for UltraScale+ devices. Multiple MIPI CSI-2 RX Subsystem instances sharing clocking resources also need to be in the same HP IO bank.

The MIPI CSI-2 RX Subsystem GUI does not have an I/O Assignment tab for AMD Versal™ adaptive SoCs. Instead, use the consolidated I/O planning in the main Vivado IDE Planning, that is, nibble planner. You can select any I/O for the data lanes for the selected XPIO bank. For the clock lane, select the clock capable pin that is the 0th pin of a nibble for the selected XPIO bank. Detailed steps on how to perform the Vivado IDE planning is detailed under section I/O Planning for Versal Adaptive SoC Advanced IO Wizard in the Advanced IO Wizard LogiCORE IP Product Guide ([PG320](#)).

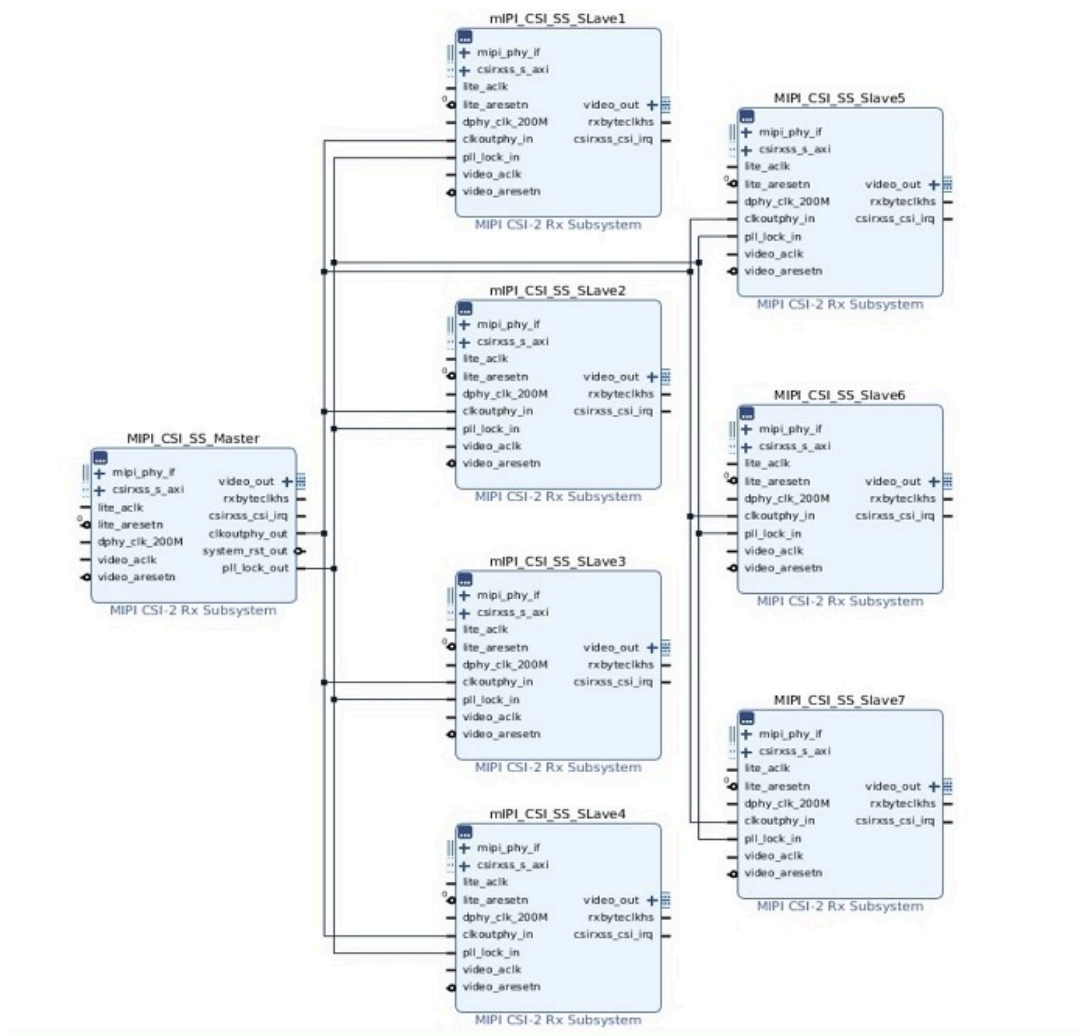
While selecting I/Os in a bank across nibbles, you need to ensure the Inter-nibble, Inter-byte clock guidelines are followed. Refer to the “Clocking” section in the Versal Adaptive SoC SelectIO Resources Architecture Manual ([AM010](#)). For more details on Versal adaptive SoC I/O planning, refer to MIPI D-PHY LogiCORE IP Product Guide ([PG202](#)).

The following figure shows the eight MIPI CSI-2 RX Subsystem IP cores configured with one clock lane and two data lanes and implemented in a single HP/XP I/O bank.

The `csi2_rx_master` is configured with Include Shared Logic in core option and the remaining cores are configured with Include Shared Logic in example design. The constant `clkoutphy` signal is generated within the PLL of the `csi2_rx_master` core irrespective of line rate and is shared with all other slave IP cores (`csi2_rx_slave1` to `csi2_rx_slave7`) with different line rates.

The controller and target cores can be configured with a different line rate when sharing `clkoutphy` within the I/O bank.

Figure: MIPI CSI-2 RX Subsystem Core Shared Logic Use Case for Single I/O Bank



The maximum number of MIPI CSI-2 interfaces which can be connected to a single HP IO (UltraScale) or XPIO (Versal adaptive SoC) bank depends on the number of clock capable pins available on that bank.

Figure: Subsystem Customization Screen – Pin Assignment

Name	Pin Loc	Pin Name
Clock Lane	W12	IO_L1P_T0L_N0_DBC_67
Data Lane0	W12	IO_L2P_T0L_N2_67
Data Lane1	T12	IO_L3P_T0L_N4_AD15P_67
Data Lane2	L16	IO_L4P_T0U_N6_DBC_AD7P_67
Data Lane3	N13	IO_L5P_T0U_N8_AD14P_67
	V8	
	T7	
	N9	
	P11	

Clocking

The following table describes the subsystem clocks. Select clock frequencies to match the throughput requirement of the downstream video pipe IP cores.

Table: Subsystem Clocks for Non-Versal Premium Series Gen 2 Devices

Clock Name	Description
lite_aclk	AXI4-Lite clock used by the register interface of all IP cores in the subsystem.
video_aclk	Clock used as core clock for all IP cores in the subsystem.
dphy_clk_200M	Refer to the MIPI D-PHY LogiCORE IP Product Guide (PG202) for information on this clock.

Clock Name	Description
clkoutphy_out	Generates within the PLL with 2500 Mb/s line rate when the Include Shared logic in core option is selected. When Deskew detection is enabled, the clkoutphy_out signal is generated with the same line rate same as the subsystem line rate.
clkoutphy_in	Connect to the clkoutphy_out signal generated when the Include Shared logic in core option is selected.
rxbyteclkhs_cnts_out	Continuous clock signal generated within the PLL with the same frequency as rxbyteclkhs when the Include Shared logic in core option is selected and line rates are greater than 1500 Mb/s.
rxbyteclkhs_cnts_in	Connect to the rxbyteclkhs_cnts_out signal generated when the Include Shared logic in example design option is selected and line rates are greater than 1500 Mb/s.
1. The <code>lite_aclk</code> clock needs to be less than or equal to <code>video_aclk</code> .	

Table: Subsystem Clocks for Versal Premium Series Gen 2 Devices

Clock Name	Description
lite_aclk	AXI4-Lite clock used by the register interface of all IP cores in the subsystem.
video_aclk	Used as core clock for all IP cores in the subsystem.

Clock Name	Description
dphy_clk_200M	Refer to the MIPI D-PHY LogiCORE IP Product Guide (PG202) for information on this clock.
rxwordclkhs_out	Recovered clock in case of C-PHY mode and clock from RX clock lane in case of D-PHY mode. • For D-PHY Mode: 25.000–281.25 MHz. Derived from Line Rate/16. • For C-PHY Mode: 57.14–321.4 MHz. Derived from line rate: ◦ For line rates ≤ 2250: Line Rate/7. ◦ For line rates from >2250: Line Rate/14
clkoutphy_out	Generates within the PLL when the Include Shared logic in Example Design option is selected. When Deskew detection is enabled, the clkoutphy_out signal is generated with the same line rate same as the subsystem line rate
shared_pll_clkoutphy_in	Connect to the clkoutphy_out signal generated when the Include Shared logic in core option is selected.
shared_pll_clkoutphy_90_in	Connect to the 90 degree shift of shared_pll_clkoutphy_in signal generated when the Include Shared logic in core option is selected.
shared_pll_clkoutphy_in	Generates when the Include Shared logic in core option is selected.
cnts_rxwordclkhs_out	Continuous clock signal generated within the PLL with the same frequency as rxwordclkhs_out when the Include Shared logic in core option is selected.

video_aclk Calculation

The read path of the lane management block in the CSI-2 RX Controller operates on a 32-bit data path (irrespective of the number of lanes) and uses the video clock for processing the data. Therefore, the minimum required video clock for line rates ≤ 1500 Mb/s needs to be greater than or equal to the effective PPI clock divided by four.

The following examples show this:

- For a MIPI interface with 1000 Mb/s per lane, 1 lane design, the effective rate at which the lane management block is written is 125 MHz. Because the lane management block read path operates on a 32-bit (4-byte) data path, the minimum required video clock is $125 \text{ MHz} / 4$ or higher.
- For a MIPI interface with 800 Mb/s, 2 lane design, the effective rate at which the lane management block is written is $100 \text{ MHz} * 2$. Because the lane management block read path operates on a 32-bit (4-byte) data path, the minimum required video clock is $(100 \text{ MHz} * 2) / 4$ or higher.
- For a MIPI interface with 700 Mb/s per lane, 3 lane design, the effective rate at which the lane management block is written is $87.5 \text{ MHz} * 3$. Because the lane management block read path operates on a 32-bit (4-byte) data path, the minimum required video clock is $(87.5 \text{ MHz} * 3) / 4$ or higher.
- For a MIPI interface with 1200 Mb/s per lane, 4 lane design, the effective rate at which the lane management block is written is $150 \text{ MHz} * 4$. Because the lane management block read path operates on 32-bit (4-byte) data path, the minimum required video clock is $(150 \text{ MHz} * 4) / 4$ or higher.

Apart from the minimum video clock requirement mentioned above, it is equally important to ensure that at any given time, the output bandwidth of the subsystem is greater than or equal to the input bandwidth.

The following examples show this:

- For a MIPI interface with 1000 Mb/s per lane, 1 lane, 2 pixel per clock design, processing RAW10 data, the minimum required video clock is $(1000 * 1) / (2 * 10)$ or higher, where 10 is the number of bits in one RAW10 pixel.
- For a MIPI interface with 600 Mb/s per lane, 4 lanes, single pixel mode design, processing YUV420 8-bit data, the minimum required video clock is $(600 * 4) / (1 * 8)$ or higher, where 8 is the number of bits in one 8-bit YUV420 pixel.
- Similarly, for a MIPI interface with 1200 Mb/s per lane, 4 lane, dual pixel mode design, processing YUV420 8-bit data, the minimum required video clock is $(1200 * 4) / (2 * 8)$ or higher, where 8 is the number of bits in one pixel.

The following equation is used to calculate the minimum required video clock:

$$\text{video_aclk1 (MHz)} = \text{Line Rate (Mb/s)} * \text{Data Lanes} / (8 * 4) \quad \text{video_aclk2 (MHz)} = (\text{Line Rate (Mb/s)} * \text{Data Lanes}) / (\text{Pixels per Clock} * \text{Number of Bits Per Pixel})$$

The effective minimum required video clock is:

$$\text{video_aclk (MHz)} = \text{Max} \{ \text{video_aclk1}, \text{video_aclk2} \}$$

For line rates greater than 1500 Mb/s, there is no limitation to the read path of the lane management block in the MIPI CSI-2 RX Controller. So, the effective minimum required clock for line rates >1500 Mb/s is:

$$\text{video_aclk (MHz)} = \text{video_aclk2}$$

When the Video Format Bridge is not included and line-rate ≤ 1500 Mb/s, the `video_aclk` calculation does not depend on the Pixel Format and Pixels Per Clock selection.

$$\text{video_aclk (MHz)} = \text{video_aclk1}$$

When the Video Format Bridge is not included and line-rate > 1500 Mb/s, the `video_aclk` calculation does not depend on Pixel Format and Pixels Per Clock selection.

$$\text{video_aclk (MHz)} = \text{rxbyteclkhs}$$

Note: For Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices, ensure you add an additional 20 MHz margin to the `video_aclk`.

Video Clock Calculation Example

For a MIPI interface with 600 Mb/s per lane, 4 lanes, single pixel mode design, processing YUV420 8-bit data, the minimum required video clock is $(600 * 4) / (1 * 8)$ or higher, where 8 is the number of bits in one 8-bit YUV420 pixel.

$\text{video_aclk1(MHz)} = 600 * 4 / (4 * 8) = 75 \text{ MHz}$
 $\text{video_aclk2(MHz)} = 600 * 4 / (1 * 8) = 300 \text{ MHz}$

The final video clock is:

$\text{video_aclk(MHz)} = \max\{75 \text{ MHz}, 300 \text{ MHz}\} = 300 \text{ MHz}$

Note: Because the internal data path architecture of the pixel processing the minimum supported data type for line rates greater than 1500 Mb/s is RAW8. RAW6 and RAW7 data types are not supported for line rates greater than 1500 Mb/s.

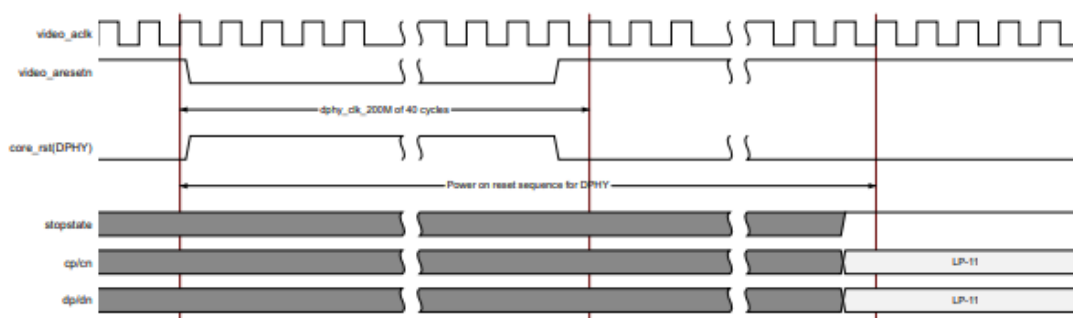
Resets

The subsystem has two reset ports:

- lite_aresetn**
 Active-Low reset for the AXI4-Lite register interface.
- video_aresetn**
 Active-Low reset for the subsystem blocks.

The duration of **video_aresetn** needs to be a minimum of 40 **dphy_clk_200M** cycles to propagate the reset throughout the system.

Figure: MIPI CSI-2 RX Reset



The following table summarizes all resets available to the MIPI CSI-2 RX Subsystem and the components affected by them.

Table: Resets

Sub-core	Lite_aresetn	Video_aresetn
----------	--------------	---------------

Sub-core	Lite_aresetn	Video_aresetn
MIPI CSI-2 RX Controller	Connected to s_axi_aresetn core port	Connected to m_axis_aresetn core port
MIPI D-PHY	Connected to s_axi_aresetn core port	Inverted signal connected to core_rst core port
Video Format Bridge	N/A	Connected to s_axis_aresetn core port
AXI Crossbar	Connected to aresetn core port	N/A

Note: The effect of each reset (`lite_aresetn`, `video_aresetn`) is determined by the ports of the sub-cores to which they are connected. Refer to the individual sub-core product guides for the effect of each reset signal.

Protocol Description

Programming Sequence

This section contains the programming sequence for the subsystem. Program and enable the components of subsystem in the following order:

1. MIPI CSI-2 RX Controller
2. MIPI D-PHY/MIPI RX C-PHY/D-PHY (if register interface is enabled)

Address Map Example

The following table shows an example based on a subsystem base address of 0x44A0_0000 (32-bits) when the MIPI D-PHY register interface is enabled.

Table: Address Map Example

Core	Base Address
------	--------------

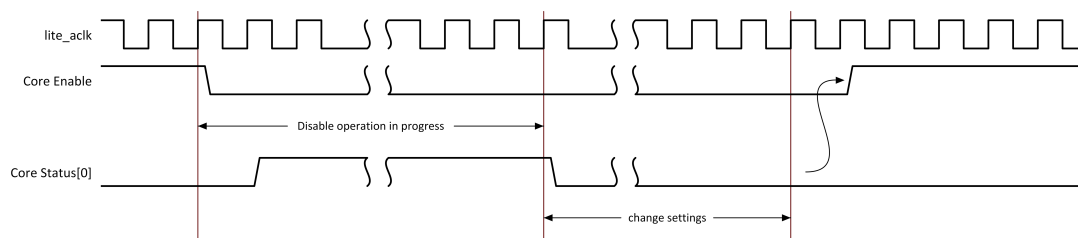
Core	Base Address
MIPI CSI-2 RX Controller	0x44A0_0000
MIPI D-PHY/MIPI RX C-PHY/D-PHY	0x44A0_1000

MIPI CSI-2 RX Controller Core Programming

The MIPI CSI-2 RX Controller programming sequence is as follows and the following figure shows a graphical representation of the sequence:

1. After power on reset (`video_aresetn`), the core enable bit is, by default, set to 1 so the core starts processing packets sent on the PPI. The Active Lanes parameter is set to Maximum Lanes (configured in the Vivado IDE using the `Serial Data Lanes` parameter).
2. Disabling and re-enabling the core
3. Disable the core using the Core Configuration Register (set the Core Enable bit to 0 or the Soft reset bit to 1).
4. Poll the Core Status Register until the Soft reset/Core enable in progress bit is cleared.
5. Change the required core settings (for example, enabling interrupts).
6. Re-enable the core (set the Core Enable bit to 1 or the Soft Reset bit to 0).

Figure: Core Programming Sequence



Note: The MIPI CSI-2 RX Subsystem is initialized only when the required duration of LP-11 is observed as per the specification.

Related Information

[Core Configuration Register](#)

Active Lanes Configuration

The Protocol Configuration Register [1:0] can be used to dynamically configure the active lanes used by the subsystem using the following guidelines:

1. Program the required lanes in the Protocol Configuration register (only allowed when Enable Active Lanes is set in the Vivado IDE).
2. The subsystem internally updates the new lanes information after the current packet complete indication is seen (for example, when the current active lanes indicate a Stop state condition) and a subsequent `RxByteClkHS` signal is seen on the PPI.
3. A read from the Protocol Configuration register reflects the new value after the subsystem has successfully updated the new lanes information internally.
4. Do not send the new updated lanes traffic until the read from Protocol Configuration registers reflects the new value.

The Active Lane bit field is not be updated if the `RxByteClkHS` is absent. This is indicated by the MIPI D-PHY RX Clock lane being in stop state. After updating the active lanes field, if the MIPI D-PHY RX Clock lane is in the stop state, you can continue without waiting for the Active Lane bit field getting updated. When the D-PHY RX Clock Lane is out of the stop state, you can check for this field to get updated with programmed value.

MIPI D-PHY IP Core Programming

Refer to the MIPI D-PHY LogiCORE IP Product Guide ([PG202](#)) for MIPI D-PHY IP core programming details.

MIPI RX PHY IP Core Programming

Refer to the MIPI RX C-PHY/D-PHY LogiCORE IP Product Guide ([PG435](#)) for MIPI RX PHY IP core programming details.

Design Flow Steps

This section describes customizing and generating the subsystem, constraining the subsystem, and the simulation, synthesis, and implementation steps that are specific to this IP subsystem. More detailed information about the standard AMD Vivado™ design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator ([UG994](#))
- Vivado Design Suite User Guide: Designing with IP ([UG896](#))
- Vivado Design Suite User Guide: Getting Started ([UG910](#))
- Vivado Design Suite User Guide: Logic Simulation ([UG900](#))

Customizing and Generating the Subsystem

This section includes information about using AMD tools to customize and generate the subsystem in the Vivado Design Suite.

If you are customizing and generating the subsystem in the Vivado IP integrator, see the Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator ([UG994](#)) for detailed information. IP integrator might auto-compute certain configuration values when validating or generating the design. To check whether the values do change, see the description of the parameter in this chapter. To view the parameter value, run the `validate_bd_design` command in the Tcl Console.

You can customize the IP for use in your design by specifying values for the various parameters associated with the subsystem using the following steps:

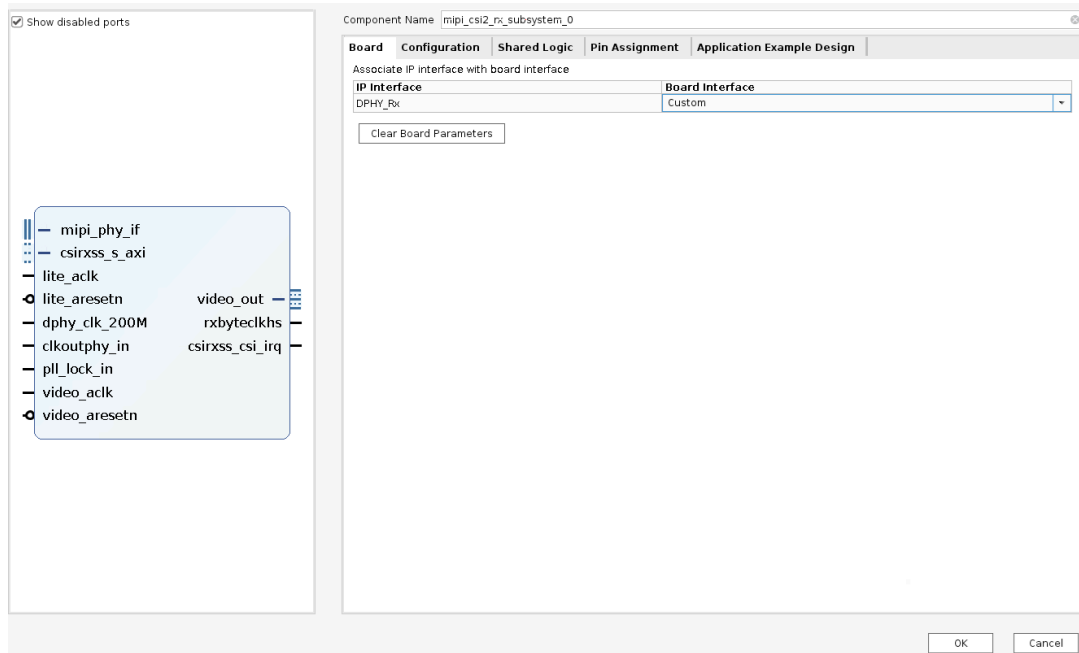
1. Select the IP from the Vivado IP catalog
2. Double-click the selected IP or select the Customize IP command from the toolbar or right-click menu.

For details, see the Vivado Design Suite User Guide: Designing with IP ([UG896](#)) and the Vivado Design Suite User Guide: Getting Started ([UG910](#)).

 **Note:** Figures in this chapter are illustrations of the Vivado Integrated Design Environment (IDE). The layout depicted here might vary from the current version.

The subsystem configuration screen for AMD UltraScale+™ devices is shown in the following figure.

Figure: Subsystem Customization Screen – Board



- **Component Name**

The Component Name is used as the name of the top-level wrapper file for the subsystem. The underlying netlist still retains its original name. Names must begin with a letter and must be composed from the following characters: a through z, 0 through 9, and “_.” The default is mipi_csi2_rx_subsystem_0.

Board Tab

The Board tab page provides board automation related parameters. The preceding figure shows the subsystem board configuration screen.

- **Board Interface**

Select the board parameters.

- **Custom**

Enables you to configure custom values (no board automation support).

- **fmc_hpc0_connector_EV_CSI2Rx_I2**

Applicable only for ZCU102 board with FMC_HPC0 connector selected as LI-IMX274MIPI-FMC V1.0 during board selection for a 2-lane MIPI CSI-2 RX subsystem.

- **fmc_hpc0_connector_EV_CSI2Rx_I4**

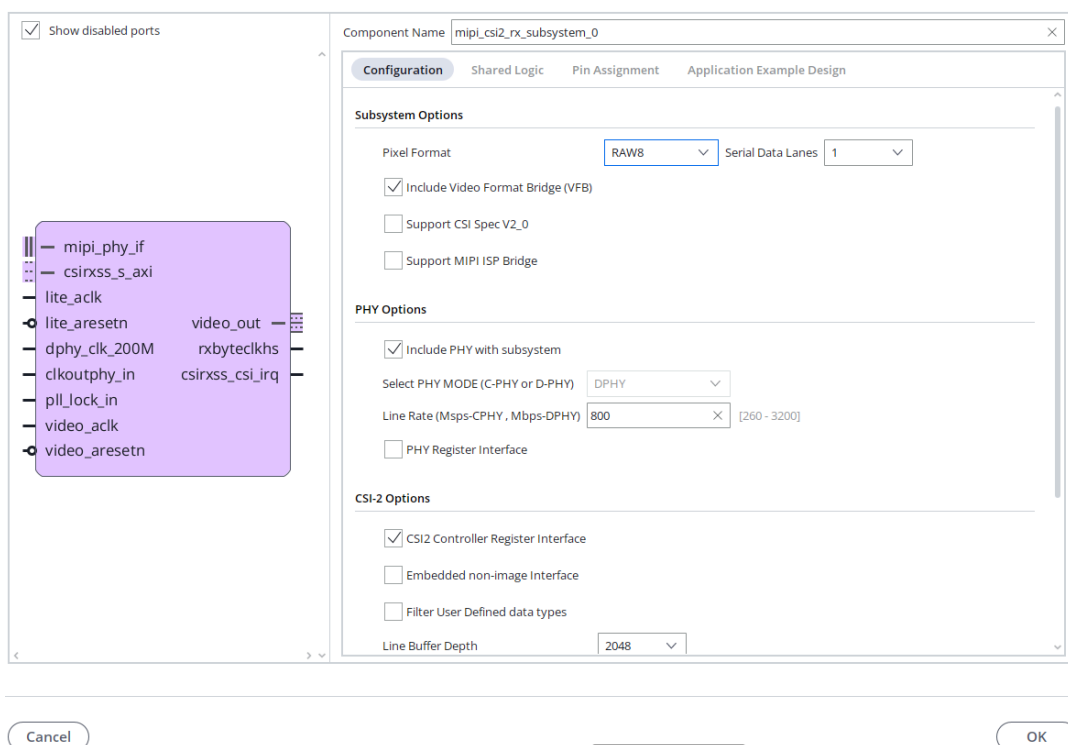
Applicable only for ZCU102 board with FMC_HPC0 connector selected as LI-IMX274MIPI-FMC V1.0 during board selection for a 4-lane MIPI CSI-2 RX subsystem.

This selection automatically configures the MIPI CSI-2 RX subsystem to support an IMX274 camera sensor which can be connected to an EV FMC card. For more information, refer to the [LI-IMX274MIPI-FMC](#) product page.

Configuration Tab

The Configuration tab page provides core-related configuration parameters. The following figure shows the subsystem configuration screen.

Figure: Subsystem Customization Screen – Configuration



- **Pixel Format**

Select the Data Type (pixel format) per the CSI-2 protocol (RAW6, RAW7, RAW8, RAW10, RAW12, RAW14, RGB888, RGB666, RGB565, RGB555, RGB444, YUV422 8-bit, YUV422_10bit, YUV420_8bit). This selection is not considered by the subsystem when Include Video Format Bridge is disabled.

- **YUV420 word count**

Select the maximum Y-line word count in bytes when YUV420 pixel format is selected.

 **Note:** Ensure the Y-line word count of the incoming data is at least eight bytes less than the selected value to avoid the YUV420 WC Error interrupt.

- **Serial Data Lanes**

Select the maximum number of D-PHY lanes for this subsystem instance. Values are 1, 2, 3, or 4.

The maximum number of C-PHY lane for this subsystem instance is 1, 2, or 3.

- **Include Video Format Bridge (VFB)**

Include or exclude the Video Format Bridge core in the subsystem is present. When this option is disabled, the Pixel Format selection is not considered by the subsystem. You can configure the stream data width (TDATA) when this option is disabled.

- **Support MIPI ISP Bridge**

Visible and enabled when the Include Video Format Bridge (VFB)" option is selected.

- **Enable dynamic line rate switching**

For non-Versal AI Edge Series Gen 2 and non-Versal Prime Series Gen 2 devices, this option is disabled by default. The PHY Register Interface must be activated for the Dynamic Line Rate feature to function properly. To enable, set PHY MODE to CPHY, PHY Register interface ON and support level to Include Shared Logic in core- 1.

- **Support CSI Spec V2_0**

Select to enable CSI V2.0 features (RAW16, RAW20 support, and VCX feature support). In Versal Premium Series Gen 2 devices, this option is enabled by default.

- **Support VCX Feature**

Option to include or exclude VCX feature. In Versal Premium Series Gen 2 devices, this option is enabled by default.

- **Line Rate (Mb/s)**

Enter a line rate value in megabits per second (Mb/s) within the valid range: 80–4500 Mb/s, based on the device selected. The Vivado IDE automatically limits the line rates based on the selected device. For details about family/device specific line rate support, refer to the data sheet for your device.

- **Select PHY MODE**

Specify the PHY type as either CPHY and DPHY.

 **Note:** Applicable only for Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices.

- **D-PHY/RX PHY Register Interface**

Select to enable the register interface for the MIPI D-PHY/RX PHY core.

- **Select IO Bank Type**

Specify the IO type as XP5IO or HPIO.

 **Note:** Applicable only for Spartan UltraScale+ devices.

- **Enable Deskew Detection**

Select to enable the deskew sequence detection and center alignment of clock and data lanes in the MIPI D-PHY core.

 **Note:** Applicable only for UltraScale+ devices with line rates above 1500 Mb/s.

 **Note:** The MIPI D-PHY RX IP has specific periodic deskew length requirements. The minimum required length of the periodic pattern is 213 UI.

- **Calibration Mode**

Select the calibration for 7 series MIPI D-PHY RX subsystem.

- **None**

The Calibration Mode does not add the IDELAY2 primitive.

- **Fixed**

The Calibration Mode sets IDELAYE2 TAP value set in the IDELAY Tap Value.

- **Auto**

The Calibration Mode adds the IDELAYE2 primitive and tap value is configured by D-PHY RX IP based on received traffic and calibration algorithm.

- **External IDELAY tap loading**

Enables the loading of IDELAY tap values through external ports for calibration in Fixed mode.

 **Note:** Clock these ports through `dphy_clk_200M`.

- **IDELAY Tap Value**

Select the IDELAY TAP value used for calibration in Fixed mode. Values 1–31.

- **Include IDELAYCTRL in core**

Select to include IDELAYCTRL in core. Only available in FIXED and AUTO calibration modes.

- For multiple MIPI CSI-2 RX IP cores that are sharing single IO bank, select Include the IDELAYCTRL option in the IP for the auto calibration mode. Only one IDELAYCTRL is available per a single clock region.
- If multiple MIPI CSI-2 RX cores exist in single clock region, select this option for only one MIPI CSI-2 RX IP core.
- For the rest of MIPI CSI-2 RX cores, this option needs to be deselected.

 **Note:** Applicable only for 7 series MIPI CSI-2 RX IP configurations.

 **Note:** For 7 series in AUTO Mode, when there are multiple instances of DPHY and they share the IDELAY control Ready from one DPHY Instance to other DPHY Instance. The DPHY Instance which shares the IDELAY Controller Ready cannot have the Enable 300 MHz clock for IDELAYCTRL set to true.

 **Note:** For 7 series, the IP core is tested and validated at a maximum frequency of 1250 Mb/s; 1500 Mb/s is available in the GUI only for testing.

- **IDELAY_GROUP Name**

Select the IDELAY_GROUP Name for the IDELAYCTRL. All core instances in the same bank sharing IDELAYCTRL must have the same name for this parameter. Select a unique name per bank.

 **Note:** Available only for 7 series configurations.

- **Enable 300 MHz clock for IDELAYCTRL**

Select to enable external 300 MHz clock port. Only available in AUTO calibration mode. This 300 MHz port is used for connecting to IDELAYCTRL. When you disable this option, IDELAYCTRL uses 200 MHz clock (dphy_clk_200M).

- **CSI2 Controller Register Interface**

Deselect to remove the MIPI CSI-2 RX Controller register interface. Disabling this option improves resource count. When the CSI-2 RX Controller register interface is disabled, the core allows enabling/disabling the core through external ports. Also, some key status information is made available as external ports of the core. Refer to [Port Descriptions](#) for details of ports available when this option is deselected.

 **Note:** For Versal Premium Series Gen 2 devices, the CSI2 Controller Register Interface is included by default. You cannot disable this option.

- **Embedded non-image Interface**

Select to process and offload embedded non-image CSI-2 packets (with data type code 0x12) using a separate AXI4-Stream interface. If deselected, such packets are not processed and are ignored by the CSI-2 RX controller.

- **Filter User Defined data types**

Select to Filter user defined data types (0x30-0x37) and do not output on Image interface (unsupported ErrId ISR[8] is not set even if filtering is enabled). If deselected, such packets are processed and presented on image interface.

- **Line Buffer Depth**

Depth of internal RAM used to accommodate throttling on the output video interface. Values are 128, 256, 512, 1024, 2048, 4096, 8192, or 16384. Set this to the lowest value possible (for example, 128), and it does not cause line buffer full condition.

 **Note:** There is no throttling allowed on the input to the PPI. Lowest value only applicable for non-Versal AI Edge Series Gen 2 and non-Versal Prime Series Gen 2 devices. For the line buffer depth calculation for Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices, refer [Line Buffer Depth Calculation for Versal AI Edge Series Gen2 and Versal Prime Series Gen 2 Devices](#).

- **Pixels Per Clock**

Select the number of pixels to output per clock on output interface. Values are 1 (single pixel), 2 (dual pixel), or 4 (quad pixel). In Versal Premium Series Gen 2 devices, the clocks on the output interface are 1 (single pixel), 2 (dual pixel), 4 (quad pixel) or 8 (octa pixel).

- **TUSER Width**

Width of the sideband signal [TUSER] to report information like the line number, frame number, ECC, and CRC.

- **Allowed VC**

For non-Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices, select the VC values to use while processing the packets. Values are All, 0-15. For Versal devices, select the VC values to be used to while processing the packets. Values are All, 0, 1, 2, or 3.

- **Enable CRC**

When set, CRC computation is performed on the packet payload and any errors are reported.

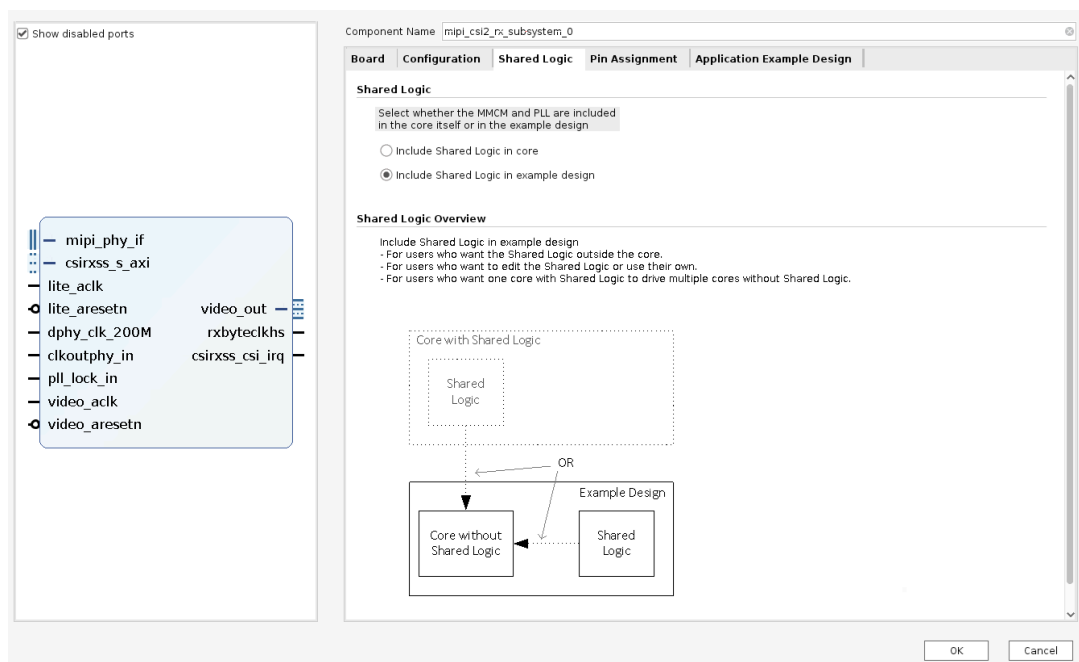
• Enable Active Lanes

When set, the core supports the dynamic configuration of the number of active lanes from the maximum number of lanes selected during core generation using the parameter `Serial Data Lanes`. For example, when `Serial Data Lanes` is set to 3, the number of active lanes can be programmed using the protocol configuration register to be 1, 2, or 3. The core reports an error when the active lanes setting is greater than the serial lanes setting through the interrupt status register, bit-21.

Shared Logic Tab

The Shared Logic tab page provides shared logic inclusion parameters. The following figure shows the subsystem shared logic configuration screen.

Figure: Subsystem Customization Screen – Shared Logic



• Shared Logic

Select whether the PLL are included in the core or in the example design. Values:

- Include Shared Logic in core
- Include Shared Logic in example design
- MMCM Sharing Across the banks (when you select Include Shared Logic in the core)
- Select to use MMCM externally for line rates > 1500

Figure: Internal MMCM

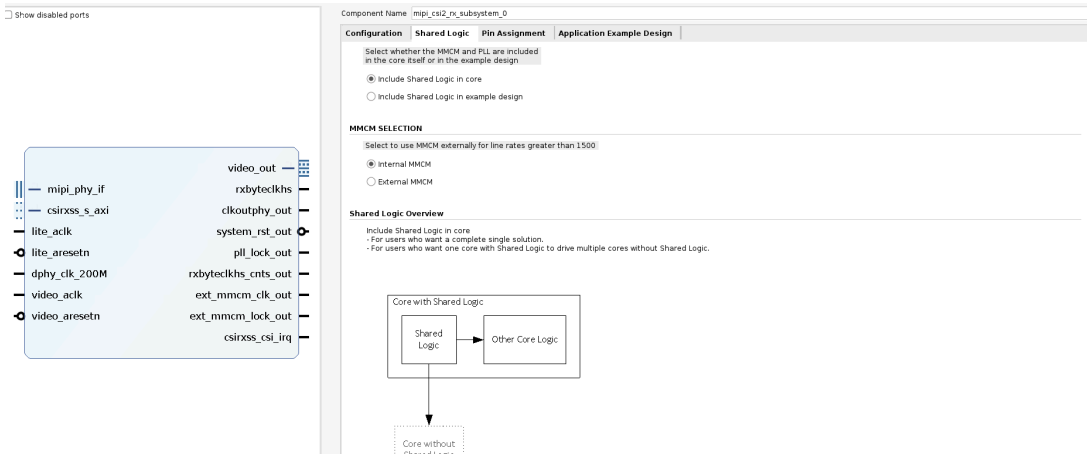
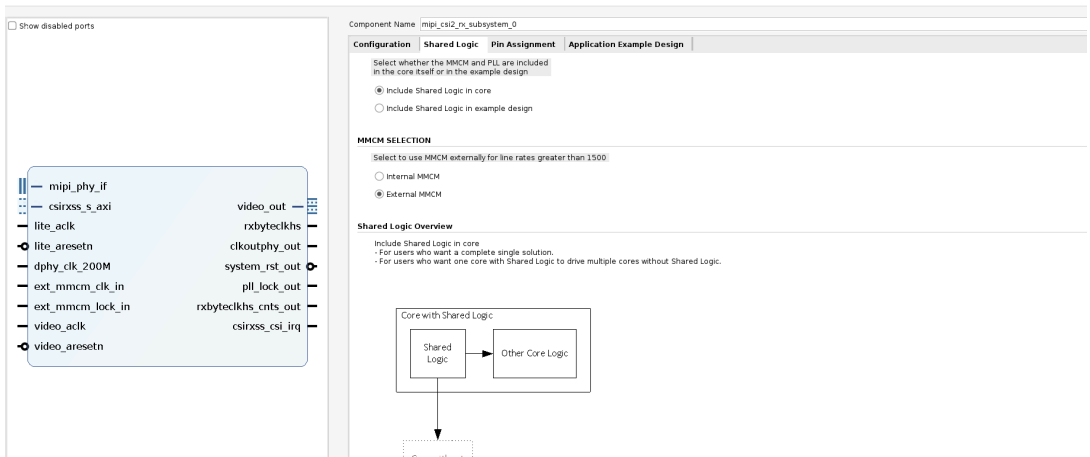


Figure: External MMCM

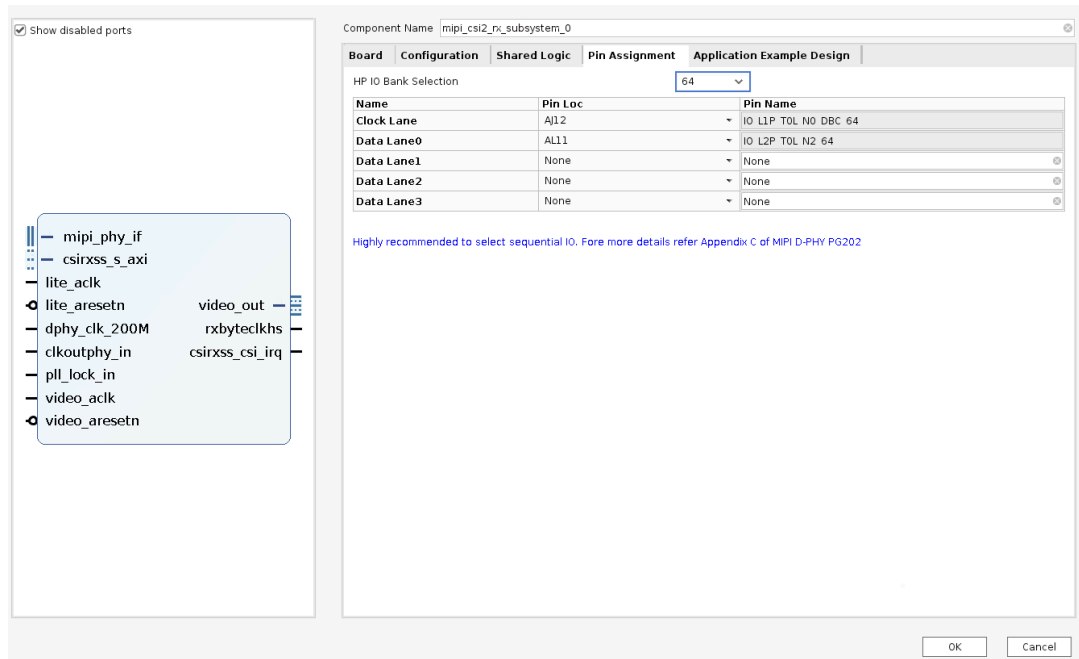


Pin Assignment Tab

The Pin Assignment tab page allows to select pins. The following figure shows the subsystem pin assignment configuration screen.

Note: This tab is not available for 7 series device configurations.

Figure: Subsystem Customization Screen – Pin Assignment



- **HP IO Bank Selection**

Select the HP I/O bank for clock lane and data lane implementation.

- **Clock Lane**

Select the LOC for clock lane. Determines the I/O byte group within the selected HP I/O bank.

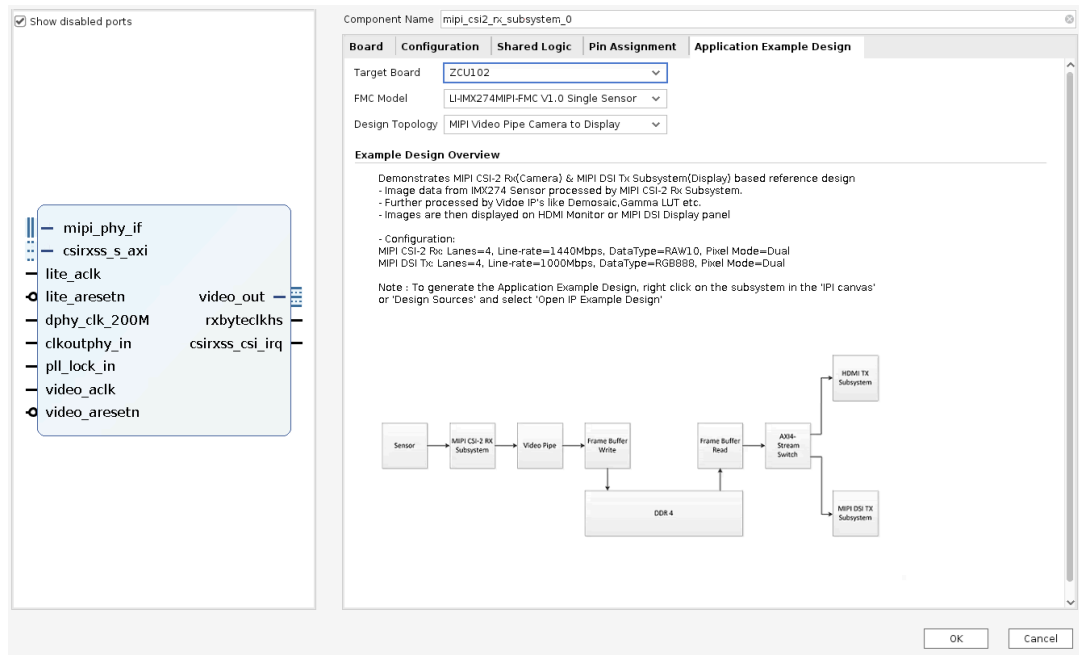
- **Data Lane 0/1/2/3**

Displays the Data lanes 0, 1, 2, and 3 LOC based on the clock lane selection.

Application Example Design Tab

The Application Example Design tab page provides example design parameters. The following figure shows the subsystem application example design configuration screen.

Figure: Subsystem Customization Screen – Application Example Design



• Target Board

Target board on which the Application example design to be built. Supported values:

- ZCU102
- SP701
- VCK190
- VEK280
- SCU200

• FMC Model

FMC Model to connect the MIPI camera sensor and MIPI display. Supported value is LI-IMX274MIPI-FMC V1.0 Single Sensor.

• Design Topology

Application example design configuration type. Select MIPI_Video_Pipe_Camera_to_Display to view the flow from the camera receive path to the display path either on the HDMI monitor or MIPI DSI display.

User Parameters

The following table shows the relationship between the fields in the Vivado IDE and the User Parameters (which can be viewed in the Tcl Console).

Table: Vivado IDE Parameter to User Parameter Relationship

Vivado IDE Parameter	User Parameter	Default Value
Pixel Format	CMN_PXL_FORMAT	RAW8
YUV420 word count	YUV420_BUF_DPTH	128
PHY MODE (C-PHY/D-PHY)	C_PHY_MODE	DPHY
Serial Data Lanes	CMN_NUM_LANES	1
Allowed VC	CMN_VC	All
Pixels Per Clock	CMN_NUM_PIXELS	1
Include video Format Bridge (VFB)	CMN_INC_VFB	true
Support MIPI ISP Bridge	C_SPRT_ISP_BRIDGE	false
Enable dynamic line rate switching	C_EN_DYNAMIC_LINE_RATE	false
Enable Deskew Detection ¹	C_RCVE_DESKEW_SEQ	false
Enable Alternate Deskew Detection ²	C_RCVE_ALT_DESKEW_SEQ	false
Enable Preamble Deskew Detection ²	C_RCVE_PREAMBLE_SEQ	false
Support CSI Spec V2_0	C_EN_CSI_V2_0	false
Support VCX Feature	C_EN_VCX	false
Line Rate (Mb/s)	DPY_LINE_RATE	1000
D-PHY Register Interface	DPY_EN_REG_IF	false
Calibration Mode	C_CAL_MODE	None
Select IO Bank Type	C_IO_TYPE	HPIO
IDELAY Tap Value	C_IDLY_TAP	1

Vivado IDE Parameter	User Parameter	Default Value
External IDELAY tap loading	C_EN_EXT_TAP	false
Include IDELAYCTRL in Core	C_SHARE_IDLYCTRL	false
Enable 300 MHZ Clock for IDELAYCTRL	C_EN_CLK300M	false
Embedded non-image Interface	CSI_EMB_NON_IMG	false
Line Buffer Depth	CSI_BUF_DEPTH	2048
Enable CRC	C_CSI_EN_CRC	true
Enable Active Lanes	C_CSI_EN_ACTIVELANES	false
Shared Logic	Support Level	0
HP IO Bank Selection	HP_IO_BANK_SELECTION	Value based on part selected.
Clock Lane	CLK_LANE_IO_LOC	Value based on part selected
Data Lane0	DATA_LANE0_IO_LOC	Value based on part selected
Data Lane1	DATA_LANE1_IO_LOC	Value based on part selected
Data Lane2	DATA_LANE2_IO_LOC	Value based on part selected
Data Lane3	DATA_LANE3_IO_LOC	Value based on part selected
HS_SETTLE Parameter (ns) ³	C_HS_SETTLE_NS	135ns + 10UI
Filter User Defined data types	C_CSI_FILTER_USERDATA TYPE	false
Target Board	C_EXDES_BOARD	ZCU102

Vivado IDE Parameter	User Parameter	Default Value
FMC Model	C_EXDES_FMC	LI-IMX274MIPI-FMC V1.0 Single Sensor
Design Topology	C_EXDES_CONFIG	MIPI_Video_Pipe_Camera _to_Display
TDATA Width	AXIS_TDATA_WIDTH	32
TDEST Width	AXIS_TDEST_WIDTH	4
TUSER Width (CSI-2 options)	AXIS_TUSER_WIDTH	96
TUSER Width (VFB options)	VFB_TU_WIDTH	1
Internal MMCM	C_EXT_MMCM	0
1. Supported when the line rate exceeds 1500 Mb/s and PHY MODE is set to DPHY. Ensure these conditions are met before enabling these features. 2. Supported only on Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices when the line rate exceeds 2500 Mb/s and PHY MODE is set to DPHY. Ensure these conditions are met before enabling these features. 3. Hidden parameter, sets the HS_SETTLE value. Set through the Tcl flow.		

Output Generation

For details, refer to the Vivado Design Suite User Guide: Designing with IP ([UG896](#)).

Constraining the Subsystem

This section contains information about constraining the subsystem in the Vivado Design Suite.

Required Constraints

The XDC constraints are delivered when the subsystem is generated.

Device, Package, and Speed Grade Selections

The maximum possible line rate per lane is dependent on device selected.

- For details about family/device specific line rate support, refer to UltraScale Architecture SelectIO Resources User Guide ([UG571](#)).
- For details on the upper line rate limits, refer to the respective AMD 7 series FPGA family device data sheet.
- For more details, refer to the Versal Adaptive SoC SelectIO Resources Architecture Manual ([AM010](#)).

Clock Frequencies

Refer to Clocking.

Related Information

[Clocking](#)

Clock Management

The MIPI CSI-2 RX Subsystem generates the required clock constraints when generated using out-of-context mode with <component_name>_fixed_ooc.xdc. You can use these or update as required for other clock constraints.

Clock Placement

This section is not applicable for this subsystem.

Banking

The MIPI CSI-2 RX Subsystem MIPI D-PHY sub-core provides a Pin Assignment tab in the Vivado IDE to select the HP I/O bank. The clock lane and data lane(s) are implemented on the selected I/O bank BITSlice(s).

 **Note:** This tab is not available for AMD 7 series FPGA configurations.

 **Note:** When placing multiple cores, follow banking rules for the target architecture. For UltraScale+ designs, multiple cores cannot target the same nibble (upper or lower). For more details, refer to UltraScale Architecture SelectIO Resources User Guide ([UG571](#)).

Transceiver Placement

This section is not applicable for this subsystem.

I/O Standard and Placement

MIPI standard serial I/O ports must use MIPI_DPHY_DCI for the I/O standard in the XDC file for the UltraScale+ family. The LOC and I/O standards must be specified in the XDC file for all input and output ports of the design. The MIPI CSI-2 RX Subsystem and MIPI D-PHY sub-core generate the I/O pin LOC for the pins that are selected during IP customization. No I/O pin LOCs are provided for AMD 7 series FPGA designs. You need to manually select the clock capable I/O for AMD 7 series FPGA RX clock lane and restrict the I/O selection within the I/O bank.

It is recommended to select the I/O bank with the VRP pin connected for UltraScale+ MIPI CSI-2 RX Subsystem configurations. If the VRP pin is present in other I/O banks in the same I/O column of the device, use the following DCI_CASCADE XDC constraint. For example, I/O bank 65 has a VPR pin, and the D-PHY TX IP is using the IO bank 66.

```
set_property DCI_CASCADE {66} [get_iobanks 65]
```

Simulation

Simulation supported example design is not available for MIPI CSI-2 RX Subsystem. However, you can generate a MIPI CSI-2 TX Subsystem simulation example design to analyze the simulation behavior of the MIPI CSI-2 RX Subsystem. MIPI CSI-2 RX Subsystem provides an application example design, which can be implemented on the hardware. Refer to Application Example Design for details.

For comprehensive information about Vivado simulation components, and information about using supported third-party tools, refer to the Vivado Design Suite User Guide: Logic Simulation ([UG900](#)).

Related Information

[Application Example Design](#)

Synthesis and Implementation

For details about synthesis and implementation, refer to the Vivado Design Suite User Guide: Designing with IP ([UG896](#)).

Application Example Design

This chapter contains step-by-step instructions for generating an MIPI CSI-2 RX subsystem application example design from the MIPI CSI-2 RX subsystem using the AMD Vivado™ flow.

The application example design is supported on multiple AMD evaluation boards with varying hardware configurations.

- For the SCU200 platform, separate example designs are provided based on the selected I/O interface (XP5IO or HPIO).
- For the VEK385 platform, separate configurations are provided for D-PHY and C-PHY modes.

Table: Hardware Details of the Application Example Design

Topology	Hardware	Processor	Lanes, Line Rate, and Data Type
----------	----------	-----------	---------------------------------

Topology	Hardware	Processor	Lanes, Line Rate, and Data Type
MIPI Video Pipe Camera to Display	• ZCU102 Rev 1.0 • AUO display panel (B101UAN01.7_H/W 1A) • LI-IMX274 MIPI-FMC camera sensor module • HDMI monitor supporting 4K@60 fps with at least 12 bpc color depth	AMD Zynq™ MPSoC	4 Lanes 1440 Mb/s Lane RAW10
MIPI Video Pipe Camera to Display	• SP701 Rev 1.1 • AUO display panel (B10UAN01.7_H/W 1A) • PCAM-5C camera sensor module • HDMI monitor	MicroBlaze™	2 Lanes 420 Mb/s Lane RAW10

Topology	Hardware	Processor	Lanes, Line Rate, and Data Type
MIPI Video Pipe Camera to Display	• VCK190/VEK 280 • AUO display panel (B10UAN01.7_H/W 1A) • LI-IMX274 MIPI-FMC camera sensor module • HDMI monitor supporting 4K@60 fps with at least 12 bpc color depth	AMD Versal™ adaptive SoC Control, Interfaces, and Processing System (CIPS)	4 Lanes 1440 Mb/s Lane RAW10
MIPI Video Pipe Camera to Display	• VEK385 (REV-B) • OmniVision OS12D40 MIPI-FMC camera sensor module • IMX715 MIPI-FMC camera sensor module • HDMI monitor	AMD Versal™ adaptive SoC	4 Lanes 1485 Mb/s Lane RAW10

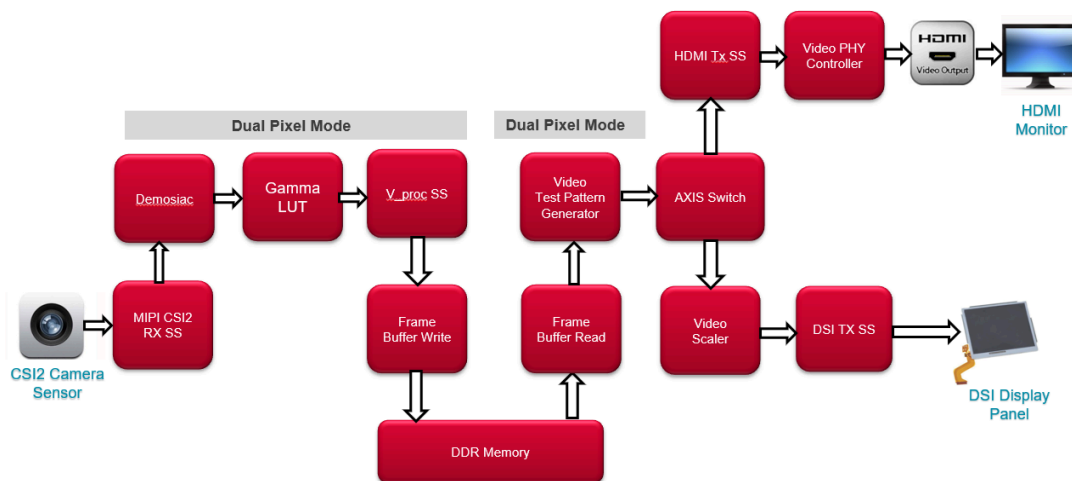
Topology	Hardware	Processor	Lanes, Line Rate, and Data Type
MIPI Video Pipe Camera to Display	• VEK385 (REV-B) • OmniVision OS12D40 MIPI-FMC camera sensor module • HDMI monitor	AMD Versal™ adaptive SoC	3 Lanes 1600 MSPS RAW10
MIPI Video Pipe Camera to Display	• SCU200 • ALINX FL1404 MIPI-FMC module • ALINX AN5020 camera sensor module • HDMI monitor	MicroBlaze™	4 Lanes 1088 Mb/s RAW10
MIPI Video Pipe Camera to Display	• SCU200 • Sony IMX219 camera sensor module • HDMI monitor	MicroBlaze™	4 Lanes 720 Mb/s 4 pixels per clock RAW10

ZCU102 Application Example Design Overview

The Application Example Design demonstrates the use of the MIPI CSI-2 RX Subsystem and MIPI DSI TX Subsystem on an AMD Zynq™ UltraScale+™ ZCU102 board. On the capture path, the system receives images captured by an IMX274 image sensor. Processed images display on either the HDMI monitor or the MIPI DSI display.

The following figures shows the block diagram of the MIPI CSI-2 RX Subsystem application example design.

Figure: MIPI CSI-2 RX Subsystem Application Example Design Block Diagram



The MIPI CSI-2 RX Subsystem decodes, processes video data and presents on AXI4-Stream data with two pixels data per clock. The RAW video data is then converted into RGB data using the Demosaic IP, V_Gamma_Lut, V_Proc_SS CSC IPs, two pixels at a time.

RGB data is then fed to the Video Test Pattern Generator IP (V-TPG). The TPG is available in the design to act as an alternate source of video in case no MIPI CSI-2 video source is present. The TPG (in pass-through mode) sends video packets across the AXI4-Stream data in dual pixel per beat mode to an AXI4-Stream broadcaster.

The broadcaster broadcasts the stream to the MIPI DSI TX Subsystem or HDMI TX Subsystem to be displayed. The HDMI TX Subsystem is available as an alternative if a MIPI DSI-compliant display panel is not available. Using the GPIO IP, one of the destination video paths is selected. The GPIO enables the `TREADY` signal in the

selected path. If you select the MIPI DSI TX Subsystem path, the video passes through a video processing subsystem configured as a scaler. This is required because the MIPI DSI Panel works on a fixed resolution of 1920x1200. All videos must either be up scaled (480p, 720p, 1080p) or downscaled (4K) to 1920x1200 resolution for the MIPI DSI display panel.

The entire system runs at 150 MHz video clock frequency.

You must have the hardware evaluation license for the following IPs to build the complete design:

- MIPI CSI-2 RX Subsystem
- MIPI DSI TX Subsystem
- HDMI TX Subsystem
- Test pattern generator

ZCU102 Setup Details

This section lists the prerequisites and setup required for a ZCU102-based application example design.

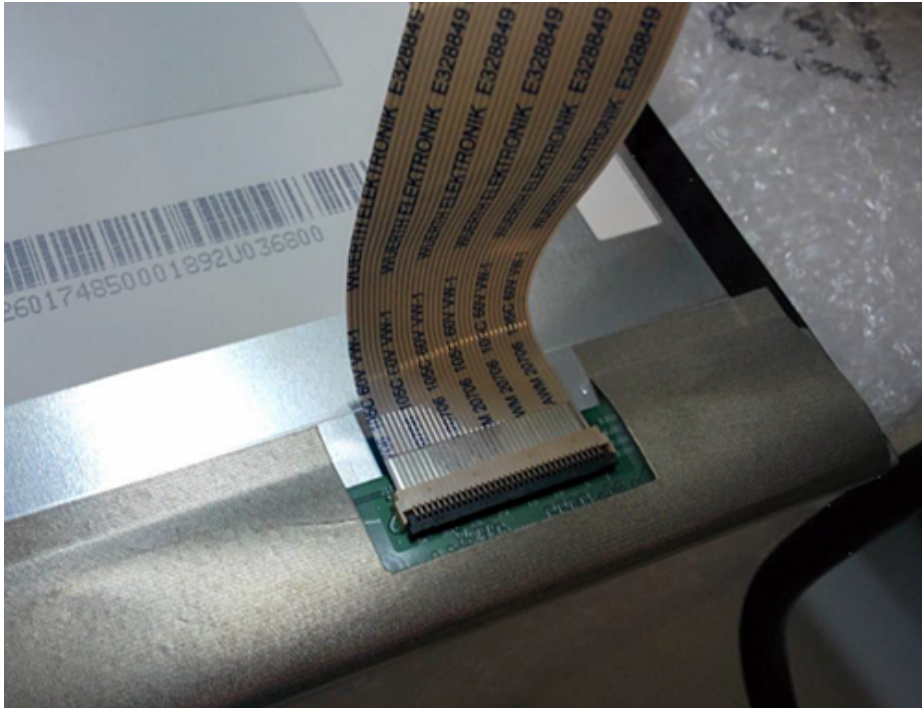
ZCU102 Prerequisites

Before working on the rest of instructions in this section, ensure that you have the following hardware available:

- AMD Zynq™ UltraScale+™ ZCU102 Rev 1.0 board and power supply
- JTAG USB Platform cable or USB cable Type A to micro – B
- USB cable Type A to micro–B for USB–UART
- HDMI cable
- HDMI monitor supporting 4K@30 fps with at least 12 bpc color depth
- AUO DSI Display panel (B101UAN01.7_H/W 1A) with ribbon cable
- LI-IMX274MIPI–FMC camera sensor module
- Host PC (to program and communicate with the program via UART)

ZCU102 Hardware Setup

1. Connect the ribbon cable to the AUO display panel.

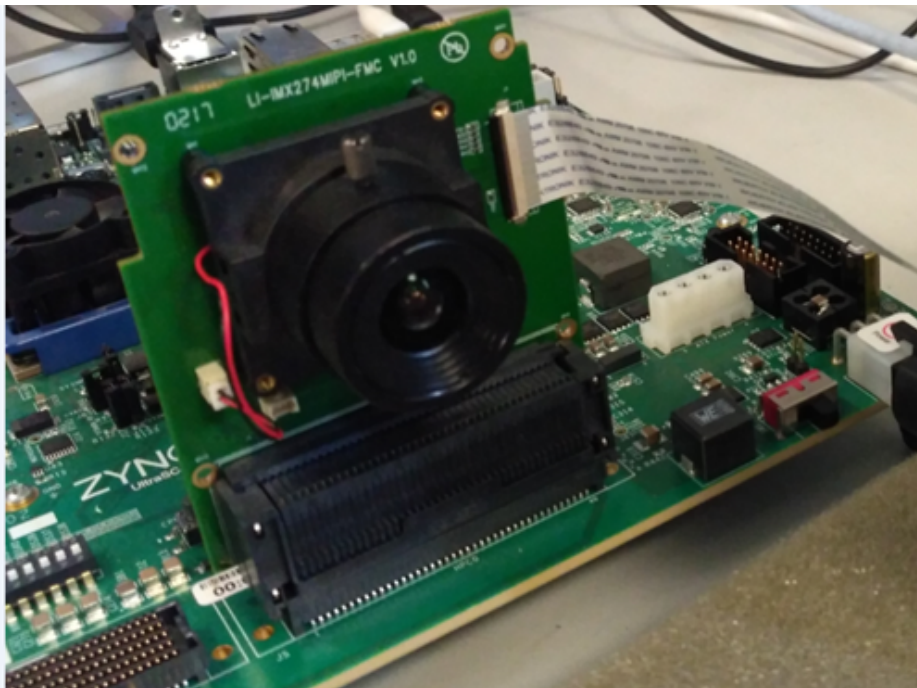


2. Connect the other end of the ribbon cable to the LI-IMX274MIPI-FMC camera sensor module.



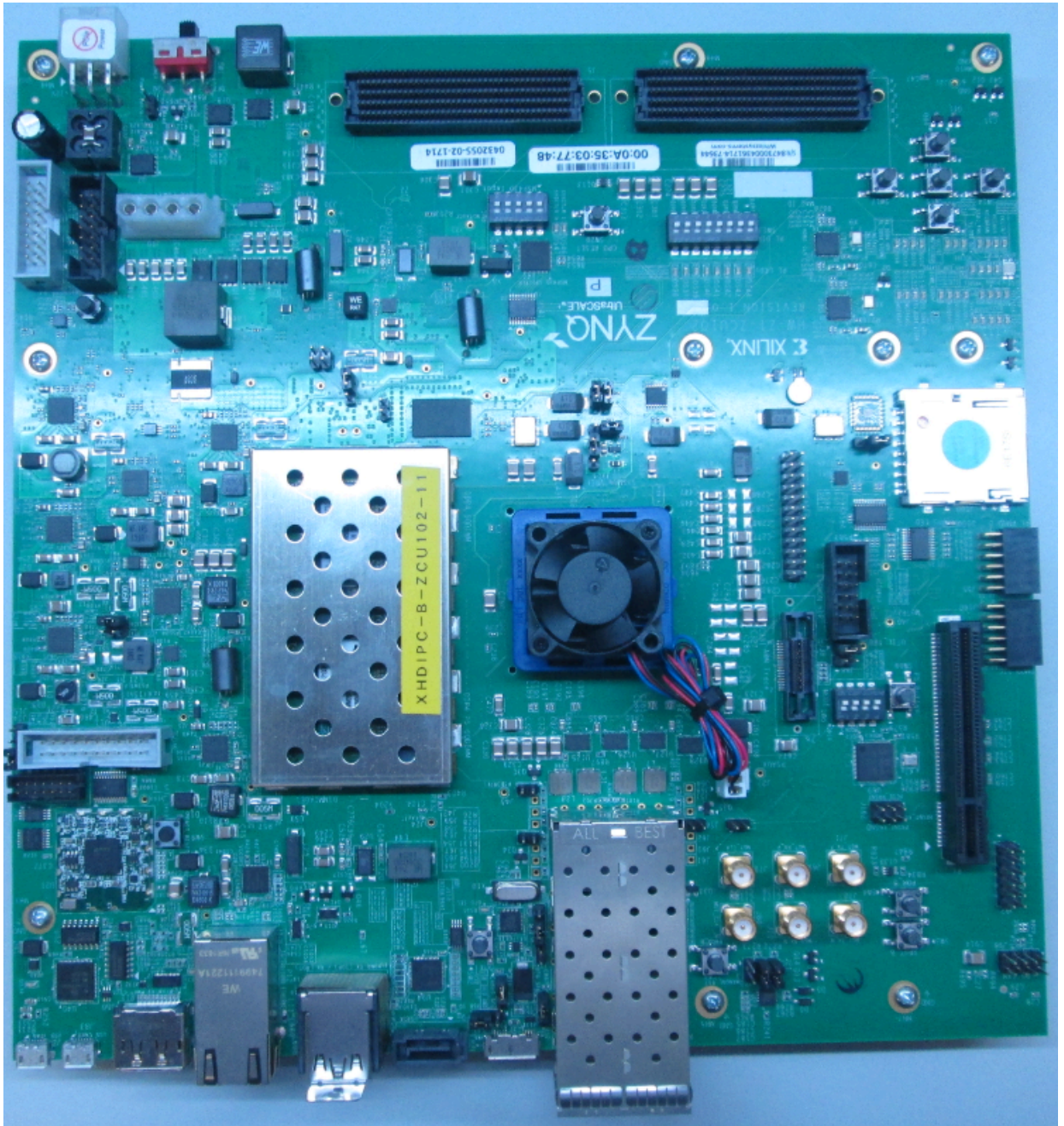
3. Set up the hardware connections:

- a. Mount the LI-IMX274MIPI-FMC camera sensor module on the ZCU102 board HPC0 FMC slot.
- b. Connect the HDMI cable to the ZCU102 HDMI port (top port).



- c. Connect the other end of the HDMI cable to the HDMI monitor.
- d. Switch on the HDMI monitor, and select HDMI as the input source.
- e. Connect USB-UART type A to micro USB cable from the host PC to the UART micro USB port of the board.
- f. Connect the USB-JTAG programming cable from host PC to JTAG micro USB port of the board.
- g. Ensure the board switches and jumpers are in position as shown in the following figure. Ensure that all SW6 switches are set to the ON position to allow programming from JTAG.
- h. Connect the USB UART and JTAG programming cables to the Windows host computer where xsdb and hw_server is running.

4. Connect the power supply cable and turn on the ZCU102 board.



5. Start a Hyper Terminal program on the host PC with the following settings:

- Baud rate: 115200
- Data Bits: 8
- Parity: None
- Stop Bits: 1
- Flow Control: None

🔧 **Note:** Ensure that VADJ is set to 1.2V. Refer to the ZCU102 board documentation for details.

SP701 Application Example Design Overview

The Application Example Design demonstrates the usage of the MIPI CSI-2 RX subsystem and MIPI DSI TX subsystem on an AMD Spartan™ 7 SP701 board. On the capture path, the system receives images captured by pcam-5c image sensor. Processed images display on either the HDMI monitor or MIPI DSI Display.

The MIPI CSI-2 RX subsystem decodes, processes video data and presents on AXI4-Stream data with two pixels data per clock. The RAW video data is then converted into RGB data using the Demosaic IP two pixels at a time. The AXI4-Stream data is sent to the AXI Switch that broadcast the stream to MIPI DSI TX subsystem or the AXI4-Stream to Video Out subsystem depending on the user selection.

The Video Test Pattern Generator IP (V-TPG) is available in the HDMI Path to act as an alternate source of video in case no MIPI CSI-2 video source is present. The TPG (in pass-through mode) sends video packets across to the AXI4-Stream to Video Out IP.

If you choose the MIPI DSI TX subsystem path, the video passes through a video processing subsystem configured as a scaler. This is required as the MIPI DSI Panel works on a fixed resolution of 1920x1200. The video must be up scaled (1080p) to 1920x1200 resolution for the MIPI DSI display panel.

SP701 Setup Details

This section lists the prerequisites and setup required for a SP701 board-based application example design.

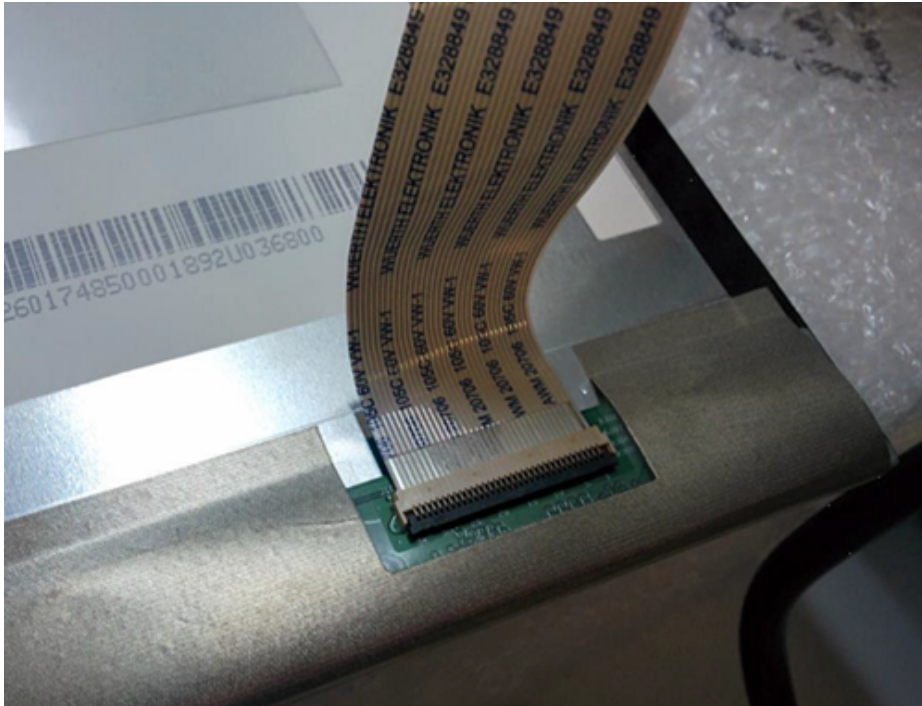
SP701 Prerequisites

Before working on the rest of instructions in this section, ensure that you have the following hardware available:

- SP701 board and power supply
- JTAG USB Platform cable or USB cable Type A to micro – B
- HDMI cable
- HDMI monitor
- AUO DSI display panel (B101UAN01.7_H/W 1A) with ribbon cable
- Pcam-5C camera sensor module
- Host PC (to program and communicate with the program through UART)

SP701 Hardware Setup

1. Connect the ribbon cable to the AUO display panel.



2. Connect the other end of the ribbon cable to the Pcam-5C camera sensor module.



3. Set up the hardware connections:

- a. Connect PCAM 5C camera sensor module to the SP701 board MIPI CSI slot.
- b. Connect the HDMI cable to the SP701 HDMI OUT port (refer to the following figure).
- c. Connect the other end of the HDMI cable to the HDMI monitor.
- d. Switch on the HDMI monitor, and select HDMI as the input source.
- e. Connect the USB-JTAG programming cable from the host PC to the board JTAG micro USB port.
- f. Ensure the board switches and jumpers are in position as shown in the following figure.



- g. Ensure that all SW6 switches are set to the ON position to allow programming from JTAG.
 - h. Connect the USB UART and JTAG programming cables to the Windows host computer where xsdb and hw_server is running.
4. Connect the power supply cable and turn on the SP701 board.
5. Start a Hyper Terminal program on the host PC with the following settings:
- Baud rate: 9600
 - Data Bits: 8
 - Parity: None
 - Stop Bits: 1
 - Flow Control: None

VCK190 Application Example Design Overview

The Application Example Design demonstrates the use of the MIPI CSI-2 RX subsystem on a VCK190 board. On the capture path, the system receives images captured by an IMX274 image sensor. Processed images display on the HDMI monitor.

The MIPI CSI-2 RX subsystem decodes, processes video data, and converts to AXI4-Stream data with four pixels data per clock. The RAW video data is then converted into RGB data using the Demosaic IP, V_Gamma_Lut, and V_Proc_SS CSC IPs, four pixels at a time and sent to the HDMI Video Out IP.

 **Note:** Ensure that you select the VCK190 board in the project setup in the Vivado Design Suite. To do this, when you get to the Default Part screen, select the Boards tab, and select Install/Update Boards. This takes you to the Xhub stores, where you can right-click the VCK190 board, and select Install. This installs the VCK190 board support files.

VCK190 Setup Details

This section lists the prerequisites and setup required for VCK190 board-based application example design.

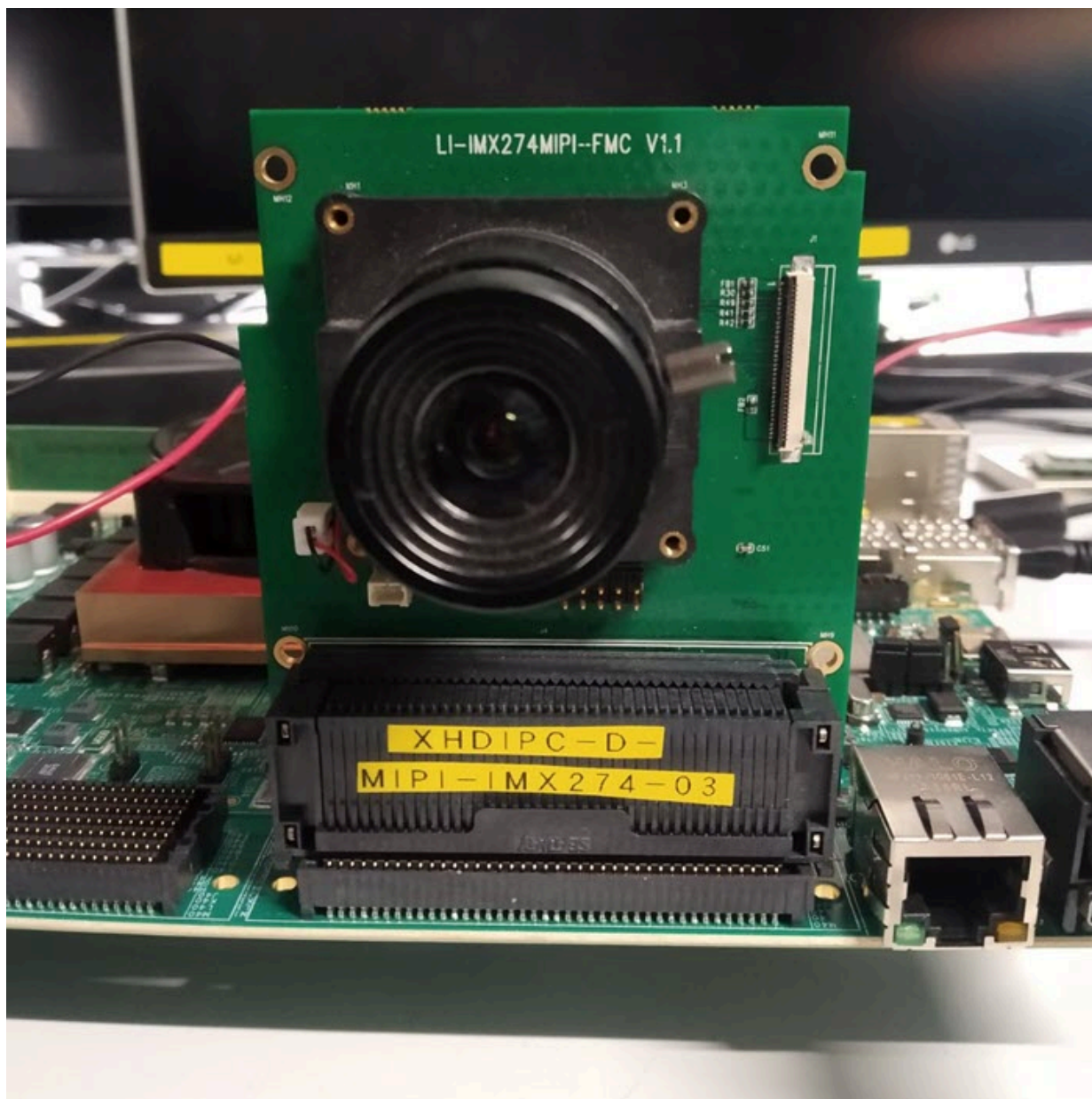
VCK190 Prerequisites

Before working on the rest of instructions in this section, ensure that you have the following hardware available:

- VCK190 board and power supply
- JTAG USB Platform cable or USB cable Type A to Type C
- HDMI cable
- HDMI monitor
- LI-IMX274MIPI-FMC camera sensor module
- SD card
- Host PC (to program and communicate with the program through UART)

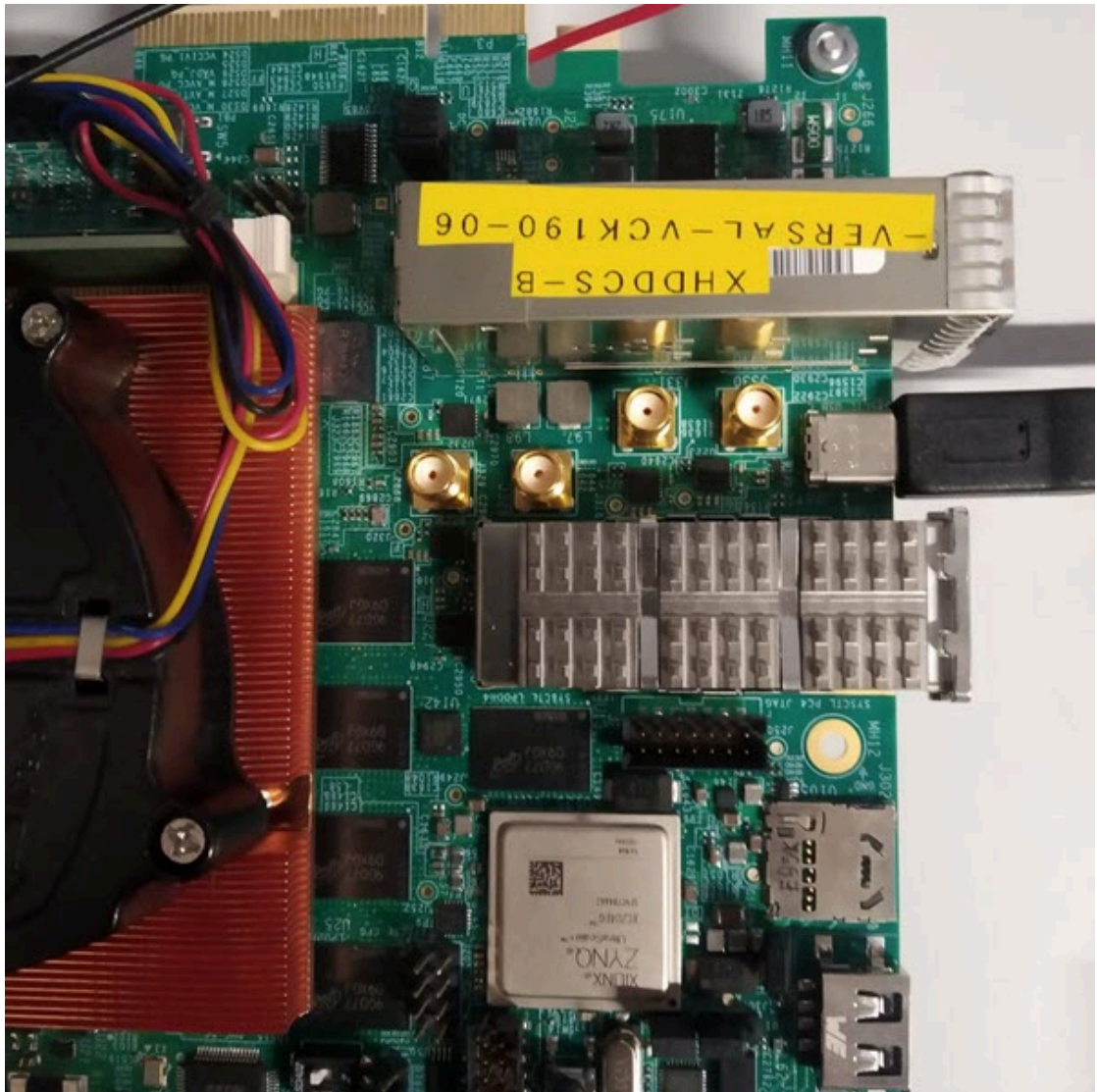
VCK190 Hardware Setup

1. Set up the hardware connections as shown in the following figure.



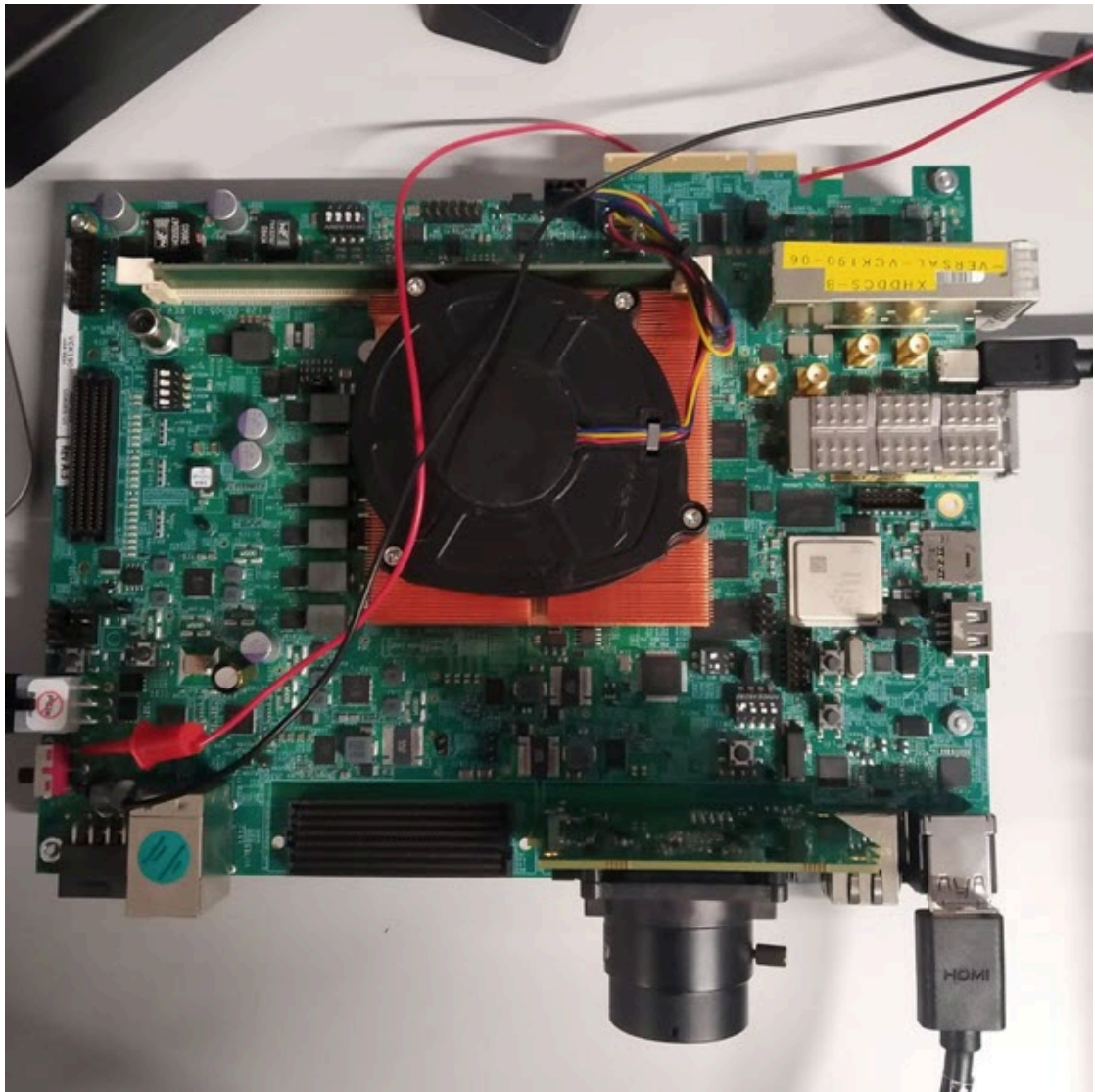
- a. Mount the LI-IMX274MIPI-FMC camera sensor module on the VCK190 board FMCP1 slot.

- b. Connect the HDMI cable to the VCK190 HDMI port as shown in the following figure.



- c. Connect the other end of the HDMI cable to the HDMI monitor.
- d. Switch on the HDMI monitor, and select HDMI as the input source.
- e. Connect the USB-JTAG programming cable from host PC to JTAG Type C USB port of the board.

- f. Ensure that the SD card, board switches, and jumpers are in position as shown in the following figure.



- g. Connect the USB UART and JTAG programming cables to the Windows host computer where xsdb and hw_server is running.
2. Connect the power supply cable and turn on the VCK190 board.
 3. Start a Hyper Terminal program on the host PC with the following settings:
 - Baud rate: 115200
 - Data Bits: 9
 - Parity: None
 - Stop Bits: 1
 - Flow Control: None

VEK385 D-PHY Application Example Design Overview

The Application Example Design demonstrates the use of the MIPI CSI-2 RX subsystem on a VEK385 board. On the capture path, the system receives images captured by an IMX715 image sensor. Processed images display on the HDMI monitor. The MIPI CSI-2 RX subsystem decodes, processes the video data, and converts to AXI4-Stream data with four pixels data per clock. The RAW video data is then converted into RGB data using the Demosaic IP, V_Gamma_Lut, and V_Proc_SS CSC IPs, four pixels at a time and sent to the HDMI Video Out IP.

Note: Ensure that you select the VEK385 (REV-B) board in the Vivado Design Suite project setup. To do this, on the Default Part screen, click the Boards tab, and select Install/Update Boards. This takes you to the Xhub stores, where you can right-click the VEK385 (REV-B) board, and select Install. This installs the VEK385 (REV-B) board support files.

VEK385 D-PHY Setup Details

This section lists the prerequisites and setup required for a VEK385 (REV-B) D-PHY board based application example design.

VEK385 D-PHY Prerequisites

Before working on the rest of instructions in this section, ensure that you have the following hardware available:

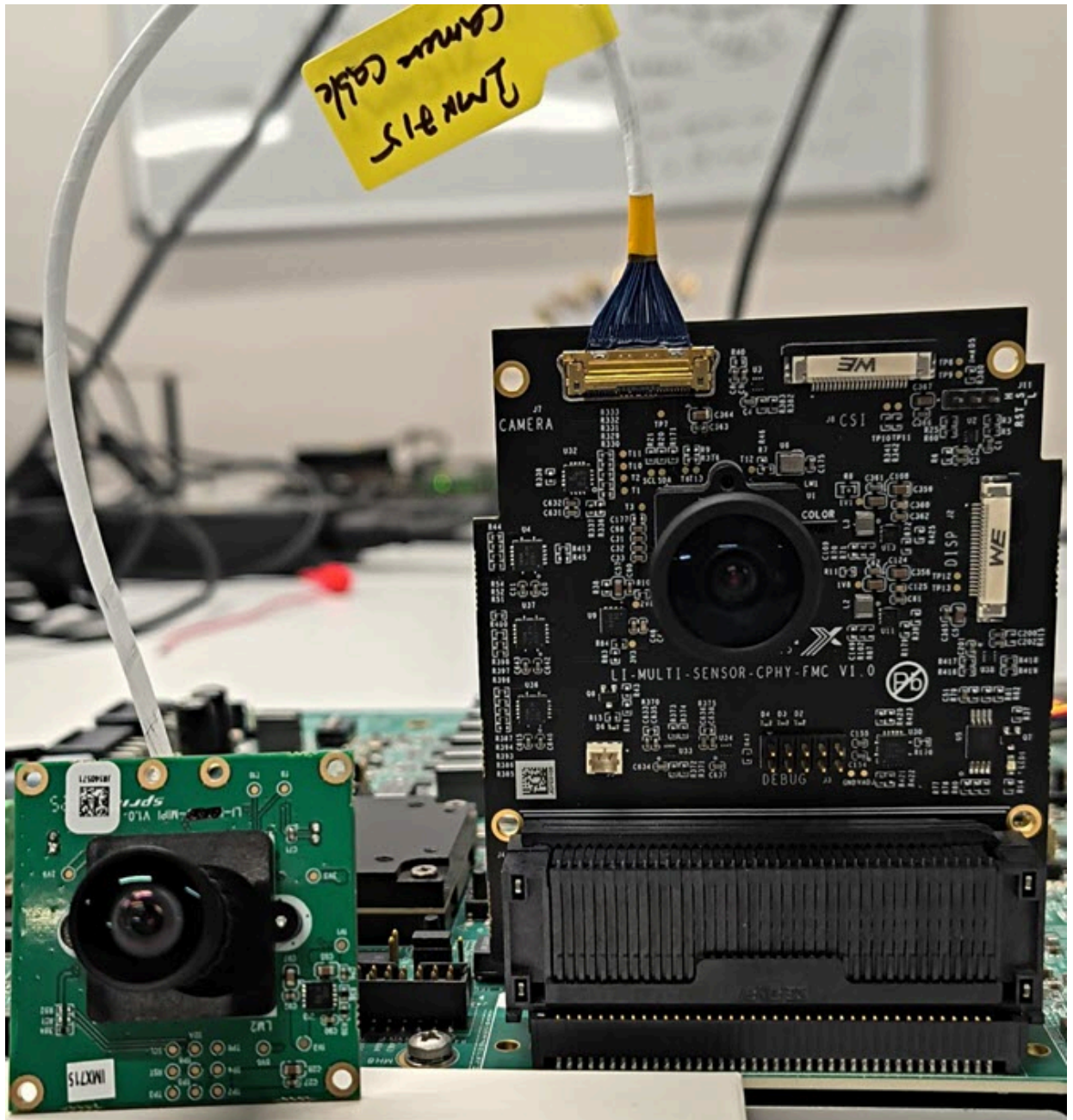
- VEK385 (REV-B) board and power supply
- JTAG USB platform cable or USB cable Type A to Type C
- HDMI cable
- HDMI monitor
- OmniVision OS12D40 MIPI-FMC Camera Sensor Module (LI-MULTI-SENSOR-CPHY-FMC_V1.0)

 **Note:** Contact marketing for C-PHY FMC details.

- LI-IMX715-MIPI D-PHY Camera Module with IPEX Connector
(<https://leopardimaging.com/product/robotics-cameras/cis-2-mipi-modules/i-pex-mipi-camera-modules/rolling-shutter-mipi-cameras/8-4mp-imx715/li-imx715-mipi-m12/>)
- Micro Coax I-PEX Cable: 200 mm (Part: FAW-1233-02)
- SD card
- Host PC (to program and communicate with the program through UART)

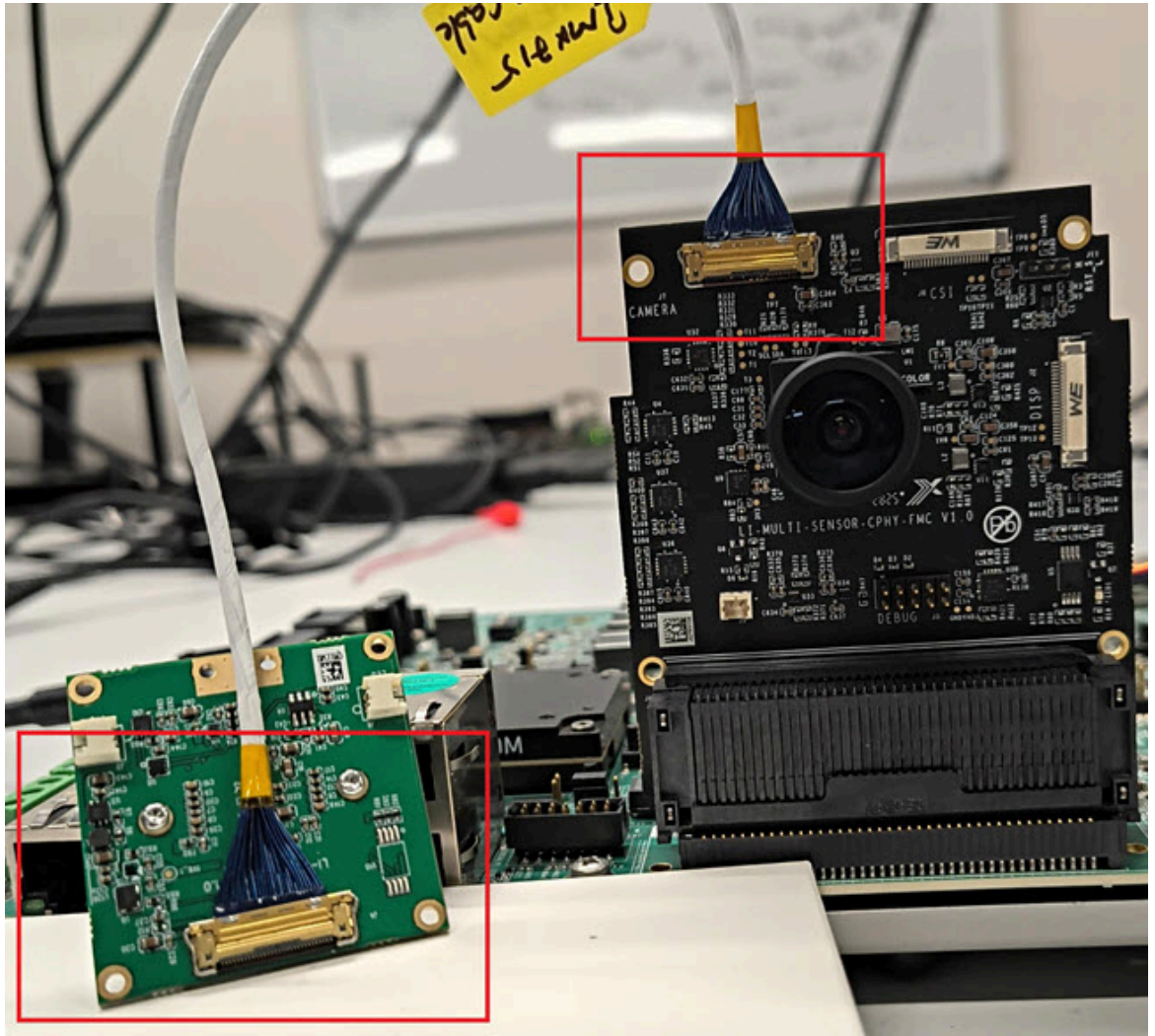
VEK385 D-PHY Hardware Setup

1. Set up the hardware connections as shown in the following figure.



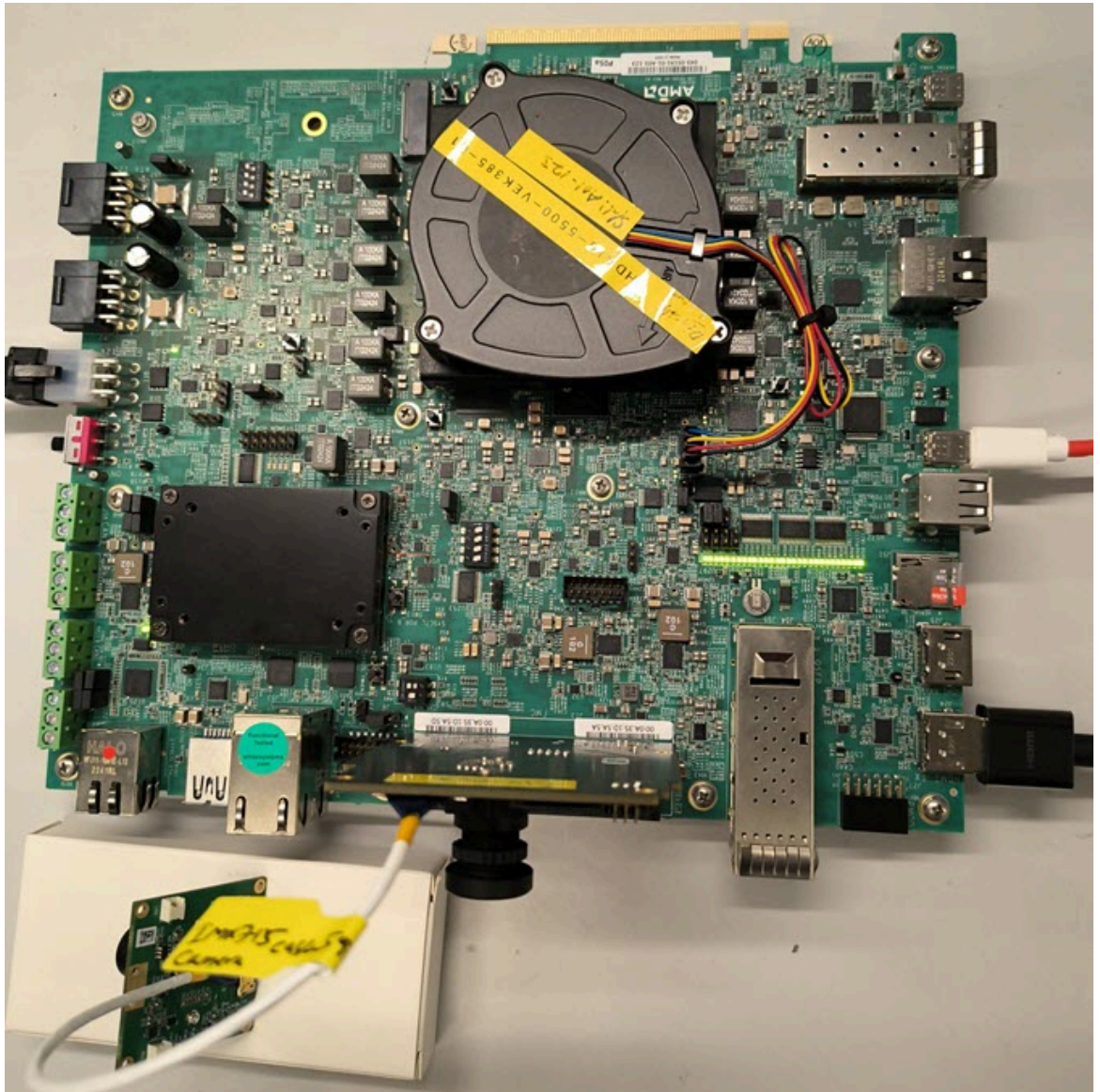
- a. Mount the OmniVision OS12D40 MIPI-FMC camera sensor module on the VEK385 board FMC slot.
- b. Connect the IMX715 camera sensor module to the OmniVision OS12D40 MIPI-FMC camera sensor module through the J7 camera port using the Micro-Coax Cable (I-PEX).

- c. Connect the HDMI cable to the VEK385 HDMI port.



- d. Connect the other end of the HDMI cable to the HDMI monitor.
- e. Switch on the HDMI monitor, and select HDMI as input source.
- f. Connect the USB-JTAG programming cable from host PC to JTAG type-C USB port of board.

- g. Ensure that the SD card, board switches and jumpers are in position as shown in the following figure. Connect the USB UART and JTAG programming cables to the Windows host computer where xsdb and hw_server is running.



2. Connect the power supply cable and turn on the VEK385 board.
3. Start a Hyper Terminal program on the host PC with the following settings:
 - Baud rate: 115200
 - Data Bits: 8
 - Parity: None
 - Stop Bits: 1
 - Flow Control: None

🔧 **Note:** Ensure that Vadj is set to 1.2V. Refer to the VEK385 board documentation for details.

VEK385 C-PHY Application Example Design Overview

The Application Example Design demonstrates the use of the MIPI CSI-2 RX subsystem on a VEK385 board. On the capture path, the system receives images captured by an OMNIVISION OS12D40 image sensor. Processed images display on the HDMI monitor. The MIPI CSI-2 RX subsystem decodes, processes video data, and converts to AXI4-Stream data with four pixels data per clock. The RAW video data is then converted into RGB data using the Demosaic IP, V_Gamma_Lut, and V_Proc_SS CSC IPs, four pixels at a time and sent to the HDMI Video Out IP.

Note: Ensure that you have selected the VEK385 (REV-B) board in the project setup in the Vivado Design Suite. To do this, on the Default Part screen, select the Boards tab, and select Install/Update Boards. This takes you to the Xhub stores, where you can right-click the VEK385 (REV-B) board, and select Install. This installs the VEK385 (REV-B) board support files.

VEK385 C-PHY Setup Details

This section lists the prerequisites and setup required for a VEK385 (REV-B) C-PHY board based application example design.

VEK385 C-PHY Prerequisites

Before working on the rest of instructions in this section, ensure that you have the following hardware available:

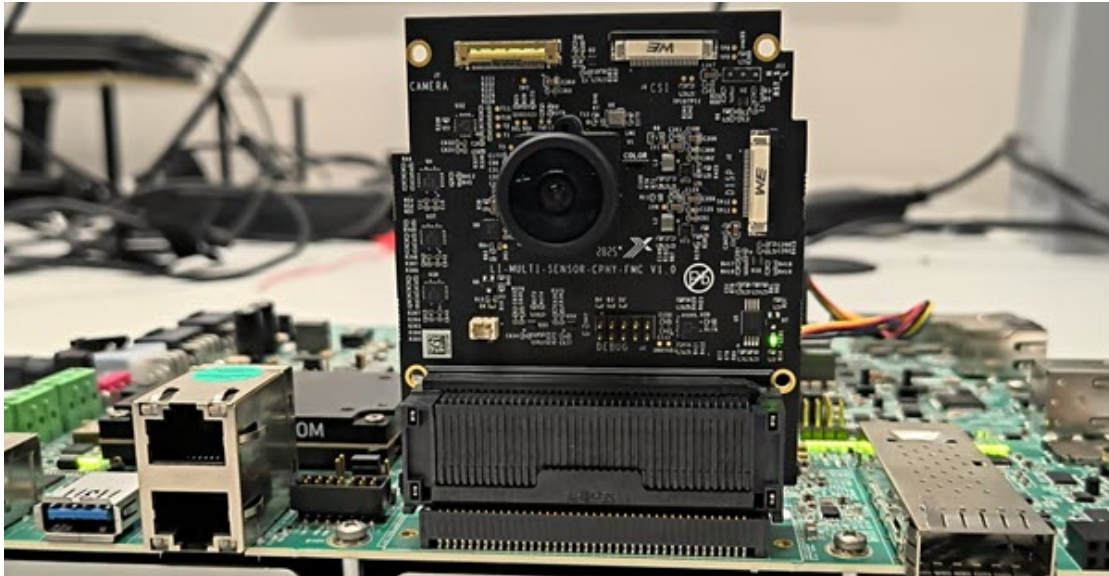
- VEK385 (REV-B) board and power supply
- JTAG USB Platform cable or USB cable Type A to Type C
- HDMI cable
- HDMI monitor
- OMNIVISION OS12D40 MIPI-FMC camera sensor module (LI-MULTI-SENSOR-CPHY-FMC_V1.0)

Note: Contact marketing for C-PHY FMC details.

- SD card
- Host PC (to program and communicate with the program through UART)

VEK385 C-PHY Hardware Setup

1. Set up the hardware connections as shown in the following figure.



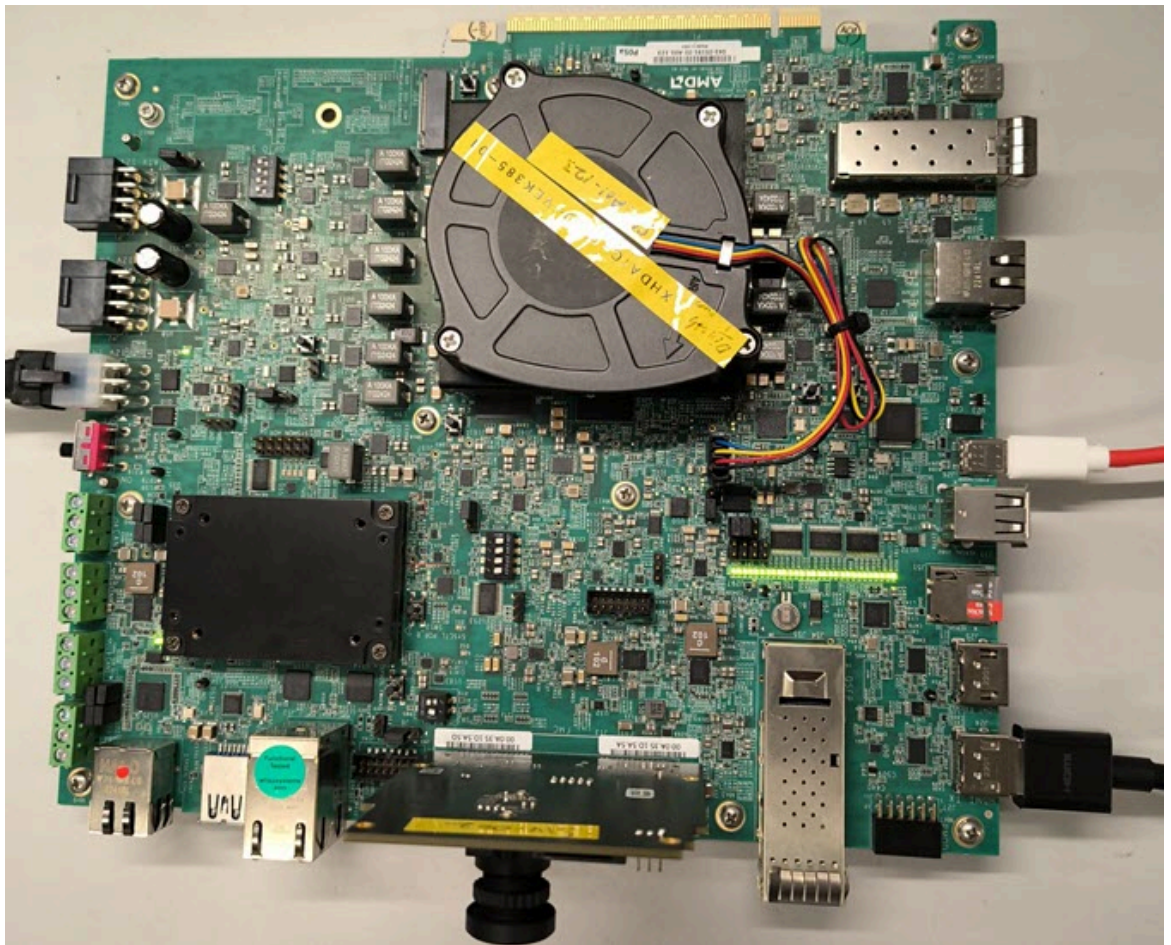
- a. Mount the OMNIVISION OS12D40 MIPI-FMC camera sensor module on the VEK385 board FMC slot.

- b. Connect the HDMI cable to the VEK385 HDMI port.



- c. Connect the other end of the HDMI cable to the HDMI monitor.
d. Switch on the HDMI monitor, and select HDMI as input source.
e. Connect the USB-JTAG programming cable from host PC to JTAG type-C USB port of the board.

- f. Ensure that the SD card, board switches and jumpers are in position as shown in the following figure. Connect the USB UART and JTAG programming cables to the Windows host computer where xsdb and hw_server is running.



2. Connect the power supply cable and turn on the VEK385 board.
3. Start a Hyper Terminal program on the host PC with the following settings:
 - Baud rate: 115200
 - Data Bits: 8
 - Parity: None
 - Stop Bits: 1
 - Flow Control: None

🔧 **Note:** Ensure that VADJ is set to 1.2V. Refer to the VEK385 board documentation for details.

VEK280 Application Example Design Overview

The Application Example Design demonstrates the use of the MIPI CSI-2 RX subsystem on a VEK280 board. On the capture path, the system receives images captured by an IMX274 image sensor. Processed images display on the HDMI monitor. The MIPI CSI-2 RX subsystem decodes, processes video data, and converts

to AXI4–Stream data with four pixels data per clock. The RAW video data is then converted into RGB data using the Demosaic IP, V_Gamma_Lut and V_Proc_SS CSC IPs, four pixels at a time and sent to the HDMI Video Out IP.

 **Note:** Ensure that you select the Versal VEK280(REV–A) board in the project setup in the Vivado Design Suite. To do this, on the Default Part screen, select the Boards tab, and select Install/Update Boards. This takes you to the Xhub stores, where you can right click on the VEK280 (REV–A) board, and select Install. This installs the VEK280 (REV–A) board support files.

VEK280 Setup Details

This section lists the prerequisites and setup required for VEK280 (REV–A) board–based application example design.

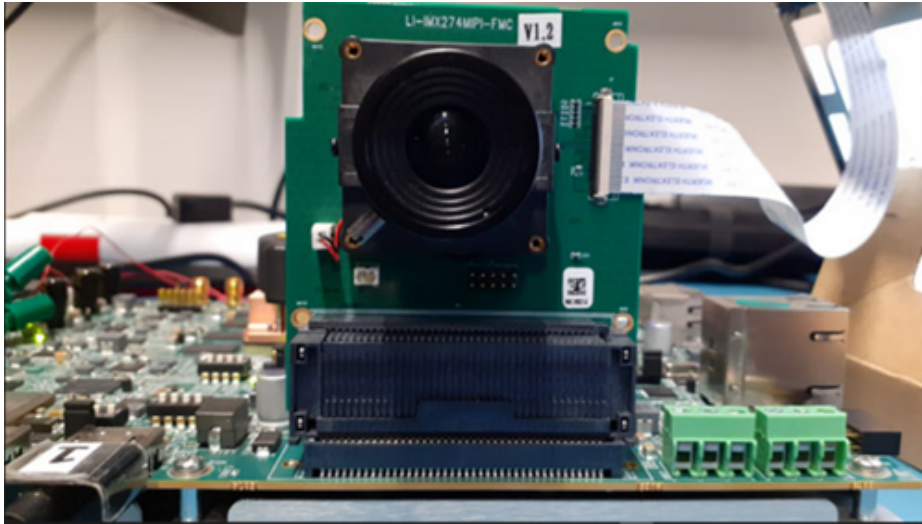
VEK280 Prerequisites

Before working on the rest of instructions in this section, ensure that you have the following hardware available:

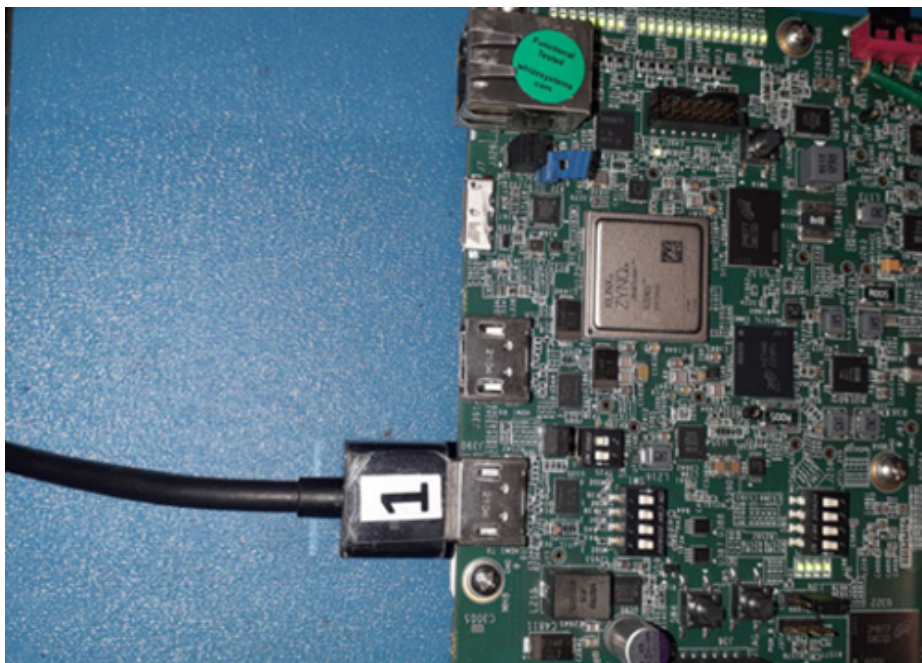
- VEK280 (REV–A) board and power supply
- JTAG USB platform cable or USB cable Type A to Type C
- HDMI cable
- HDMI monitor
- LI–IMX274MIPI–FMC camera sensor module
- SD card
- Host PC (to program and communicate with the program through UART)

VEK280 Hardware Setup

1. Set up the hardware connections as shown in the following figure.



- a. Mount the LI-IMX274MIPI-FMC camera sensor module on the VEK280 board FMC slot.
- b. Connect the HDMI cable to the VEK280 HDMI port.



- c. Connect the other end of the HDMI cable to the HDMI monitor.
- d. Switch on the HDMI monitor, and select HDMI as the input source.
- e. Connect the USB-JTAG programming cable from host PC to JTAG Type-C USB port of board.

- f. Ensure that the SD card, board switches, and jumpers are in position as shown in the following figure. Connect the USB UART and JTAG programming cables to the Windows host computer where xsdb and hw_server is running.



2. Connect the power supply cable and turn on the VCK190 board.
3. Start a Hyper Terminal program on the host PC with the following settings:
 - Baud rate: 115200
 - Data Bits: 8
 - Parity: None
 - Stop Bits: 1
 - Flow Control: None

SCU200 XP5IO Application Example Design Overview

The Application Example Design demonstrates the use of the MIPI CSI-2 RX subsystem on a SCU200 board. On the capture path, the system receives images captured by an ALINX AN5020 camera sensor module. Processed images display on the HDMI monitor.

The MIPI CSI-2 RX subsystem decodes, processes video data, and converts to AXI4-Stream data with two pixels data per clock. The RAW video data is then converted into RGB data using the Demosaic IP, V_Gamma_Lut, and V_Proc_SS CSC IPs, four pixels at a time and sent to the HDMI Video Out IP.

 **Note:** Ensure that you select the Spartan UltraScale+ SCU200 board in the project setup in the Vivado Design Suite. To do this, on the Default Part screen, select the Boards tab, and select Install/Update Boards. This takes you to the Xhub stores, where you can right-click the SCU200 board, and select Install. This installs the SCU200 board support files.

SCU200 XP5IO Setup Details

This section lists the prerequisites and setup required for a SCU200 XP5IO board-based application example design.

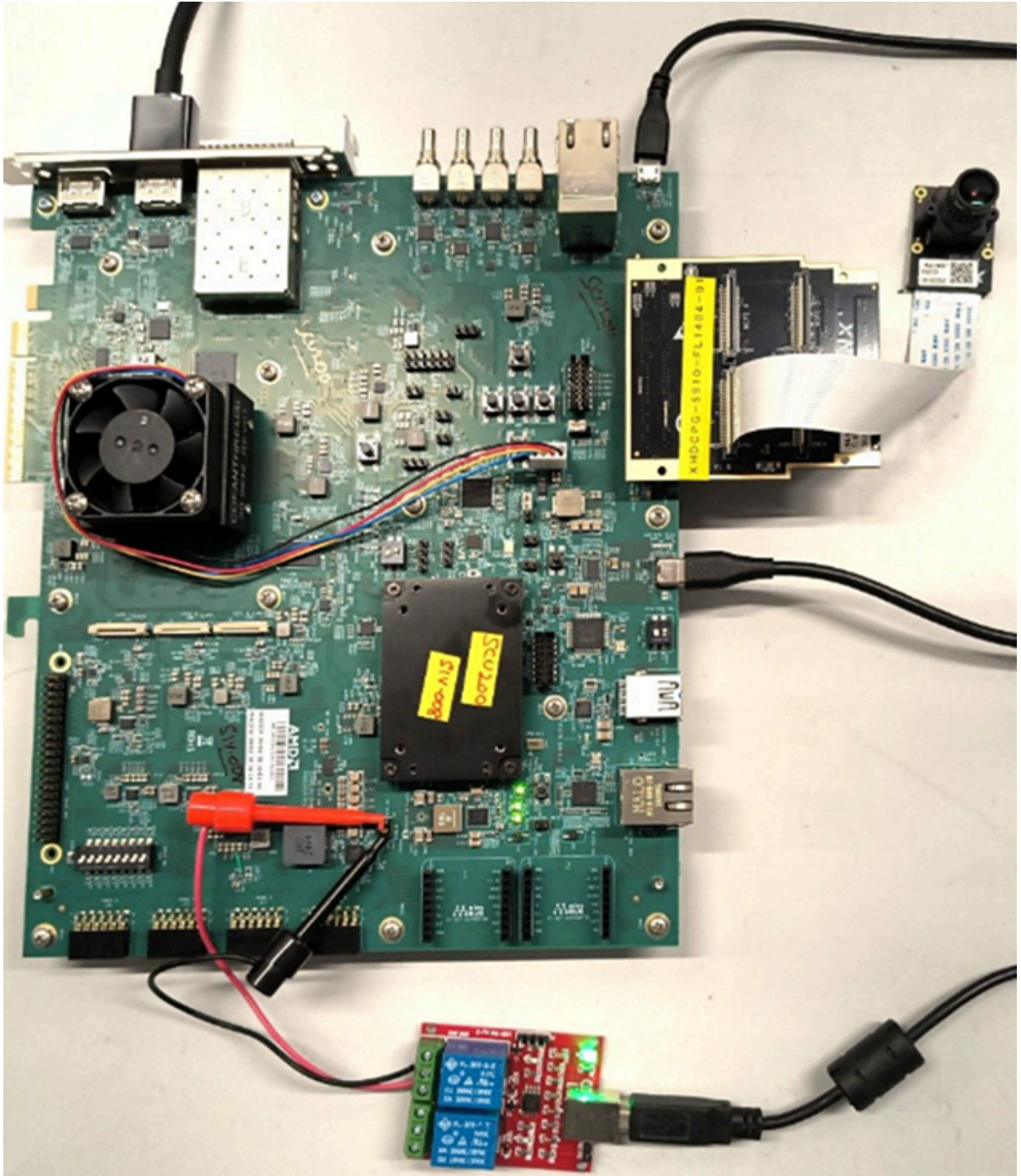
SCU200 XP5IO Prerequisites

Before working on the rest of instructions in this section, ensure that you have the following hardware available:

- SCU200 board and power supply
- JTAG USB Platform cable or USB cable to Type A
- HDMI cable
- HDMI monitor
- ALINX FL1404 4-channel MIPI FMC module
(<https://www.en.alinx.com/Product/FMC-Cards/Video/FL1404.html>)
- ALINX AN5020 camera sensor module
(<https://www.en.alinx.com/Product/Add-on-Modules/AN5020.html>)
- Host PC (to program and communicate with the program through UART)

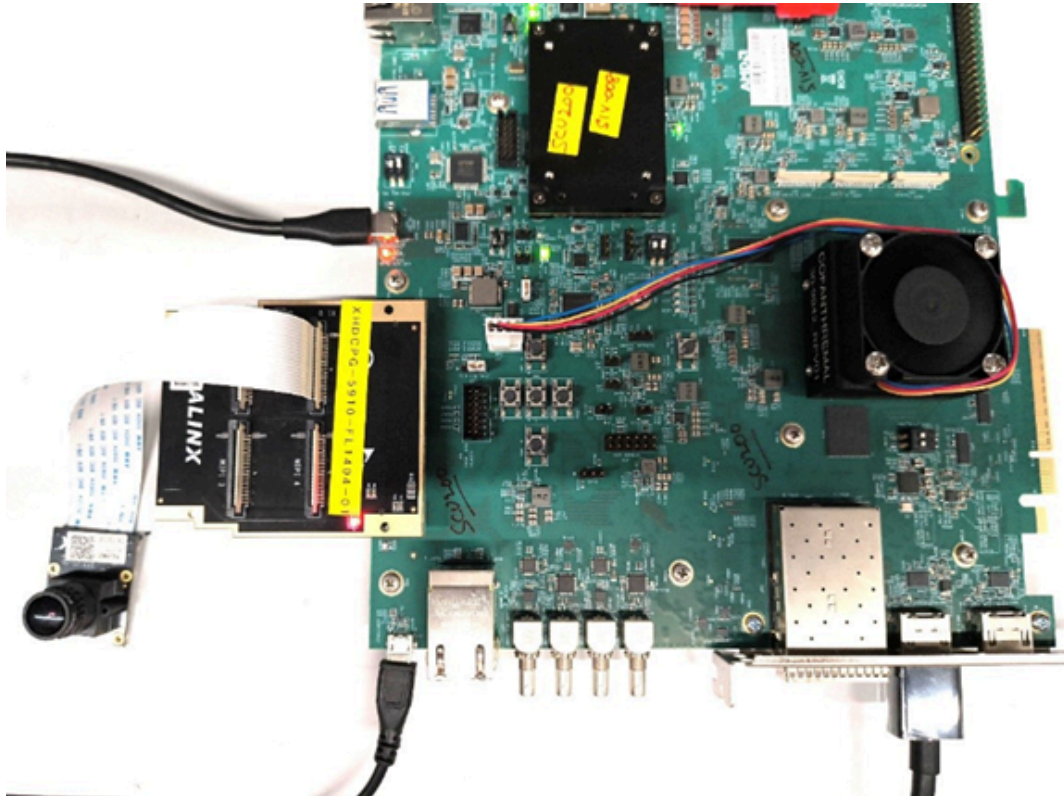
SCU200 XP5IO Hardware Setup

1. Set up the hardware connections as shown in the following figure.



- a. Insert the ALINX FL1404 4-channel MIPI FMC Module on the SCU200 FMC slot, and connect the ALINX AN5020Camera sensor module to the Slot-1 of FL1404 using 20-pin ribbon flex cable.

- b. Connect the HDMI cable to the SCU200 HDMI port.



- c. Connect the other end of the HDMI cable to the HDMI monitor.
d. Switch on the HDMI monitor, and select HDMI as the input source.
e. Connect the USB-JTAG programming cable from host PC to JTAG type-A USB port of the board.
f. Ensure that the SD card, board switches and jumpers are in position.
Connect the USB UART and JTAG programming cables to the Windows host computer where xsdb and hw_server is running.
2. Connect the power supply cable and turn on the SCU200 board
3. Start a Hyper Terminal program on the host PC with the following settings:
- Baud rate: 115200
 - Data Bits: 8
 - Parity: None
 - Stop Bits: 1
 - Flow Control: Non

SCU200 HPIO Application Example Design Overview

The Application Example Design demonstrates the use of the MIPI CSI-2 RX subsystem on a SCU200 board. On the capture path, the system receives images

captured by an Sony IMX219 camera sensor module. Processed images display on the HDMI monitor.

The MIPI CSI-2 RX subsystem decodes, processes video data, and converts to AXI4-Stream data with two pixels data per clock. The RAW video data is then converted into RGB data using the Demosaic IP, V_Gamma_Lut and V_Proc_SS CSC IPs, four pixels at a time and sent to the HDMI Video Out IP.

Note: Ensure that you have selected the SCU200 board in the project setup in the Vivado Design Suite. To do this, on the Default Part screen, select the Boards tab, and select Install/Update Boards. This takes you to the Xhub stores, where you can right-click the SCU200 board, and select Install. This installs the SCU200 board support files.

SCU200 HPIO Setup Details

This section lists the prerequisites and setup required for a SCU200 HPIO board-based application example design.

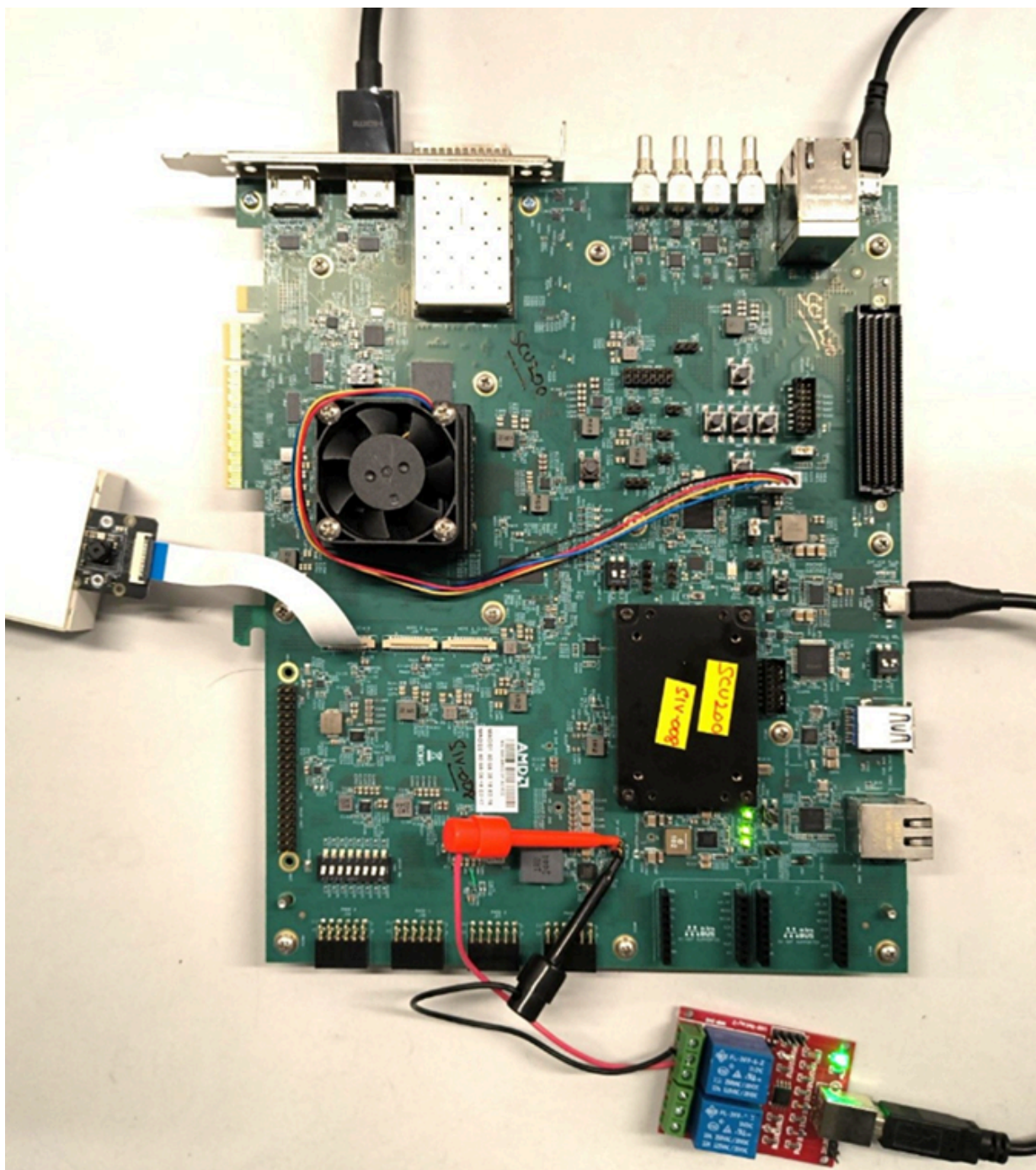
SCU200 HPIO Prerequisites

Before working on the rest of instructions in this section, ensure that you have the following hardware available:

- SCU200 board and power supply
- JTAG USB Platform cable or USB cable to Type A
- HDMI cable
- HDMI monitor
- Raspberry Pi camera module 2 (IMX219)
(<https://www.raspberrypi.com/products/camera-module-v2/>)
- Host PC (to program and communicate with the program through UART)

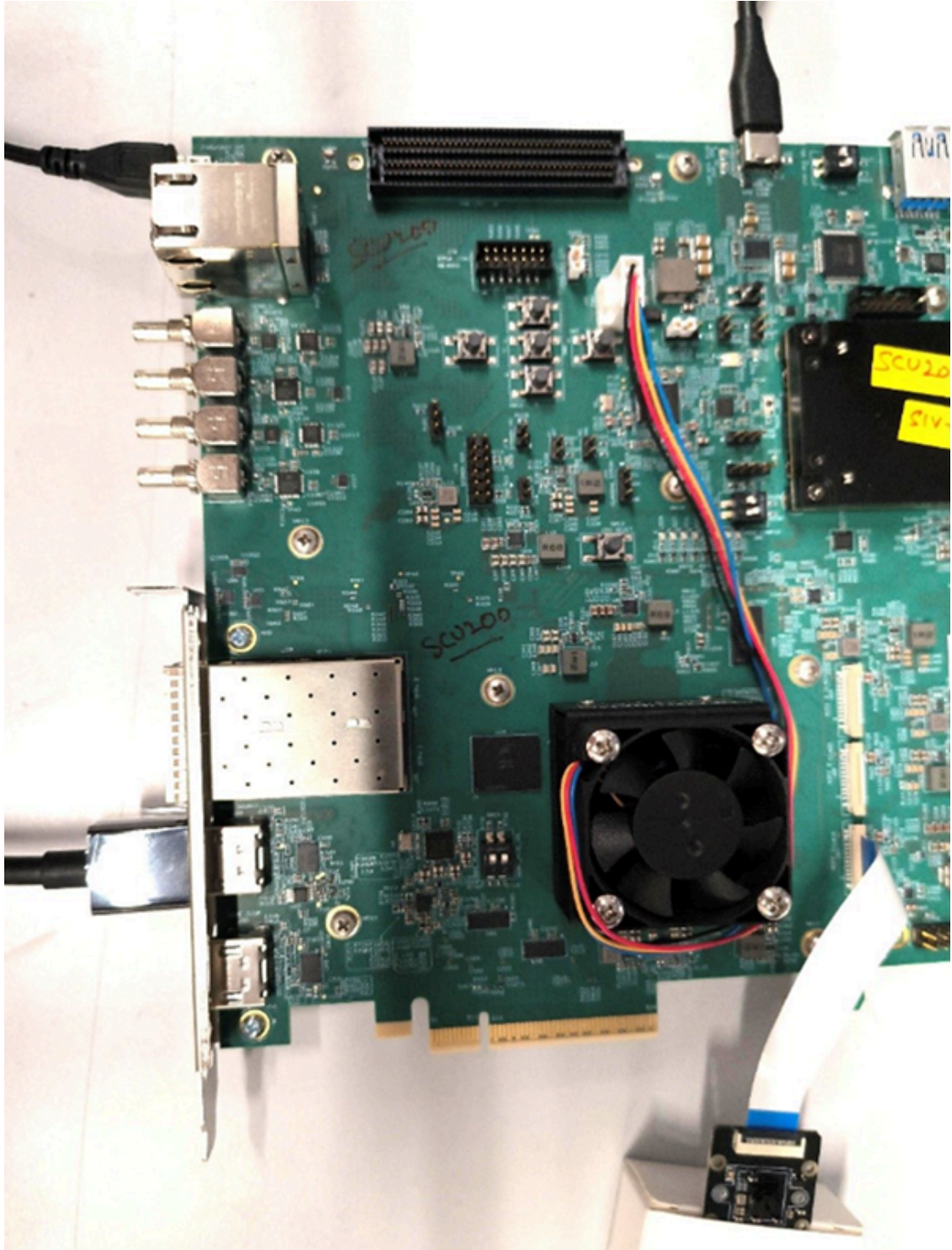
SCU200 HPIO Hardware Setup

1. Set up the hardware connections as shown in the following figure.



- a. Connect the SONY IMX219 camera module to the J31 connector of the SCU200 using a 22-pin flex cable.

- b. Connect the HDMI cable to the SCU200 HDMI port.



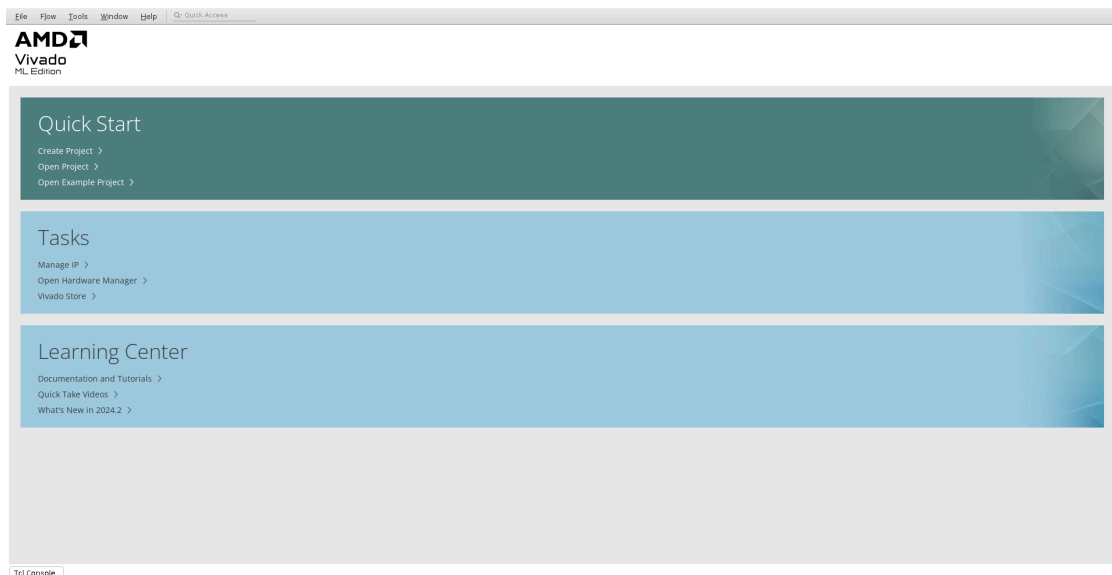
- c. Connect the other end of the HDMI cable to the HDMI monitor.
d. Switch on the HDMI monitor, and select HDMI as the input source.
e. Connect the USB-JTAG programming cable from host PC to JTAG type-A USB port of board.
f. Ensure that the SD card, board switches, and jumpers are in position. Connect the USB UART and JTAG programming cables to the Windows host computer where xsdb and hw_server is running.
2. Connect the power supply cable and turn on the SCU200 board.

3. Start a Hyper Terminal program on the host PC with the following settings:
 - Baud rate: 115200
 - Data Bits: 8
 - Parity: None
 - Stop Bits: 1
 - Flow Control: None

Implementing the Example Design

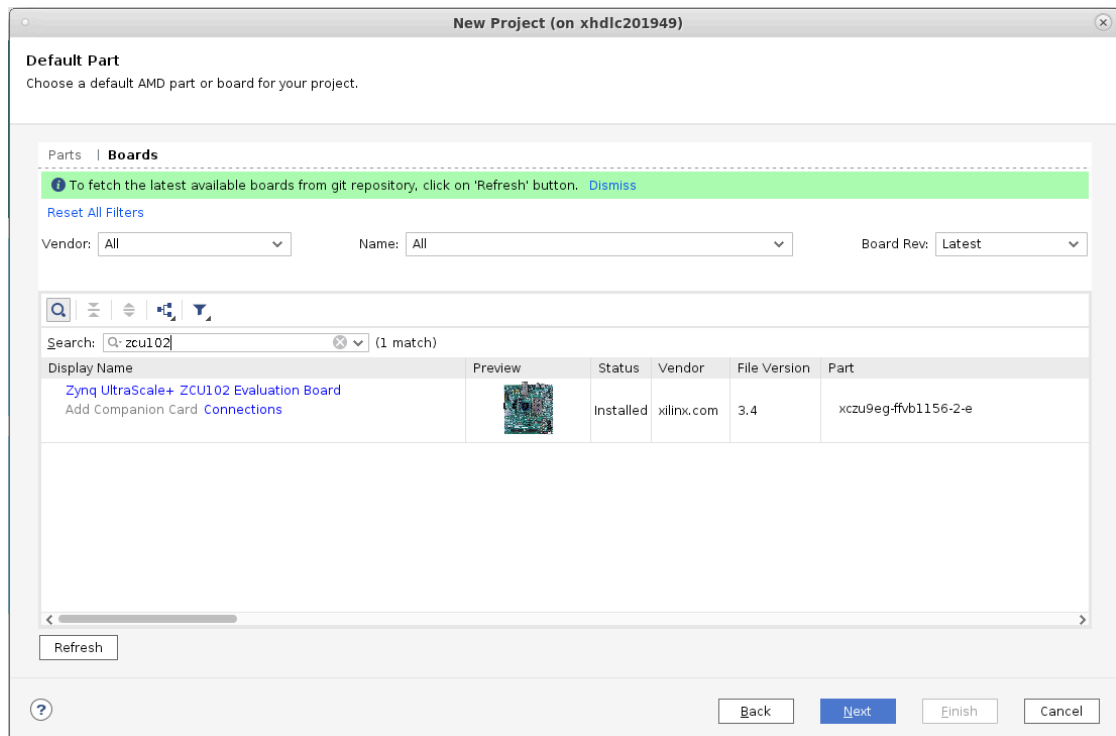
1. Open the Vivado Design Suite.

The Vivado IDE Getting Started page contains links to open or create projects and to view documentation.

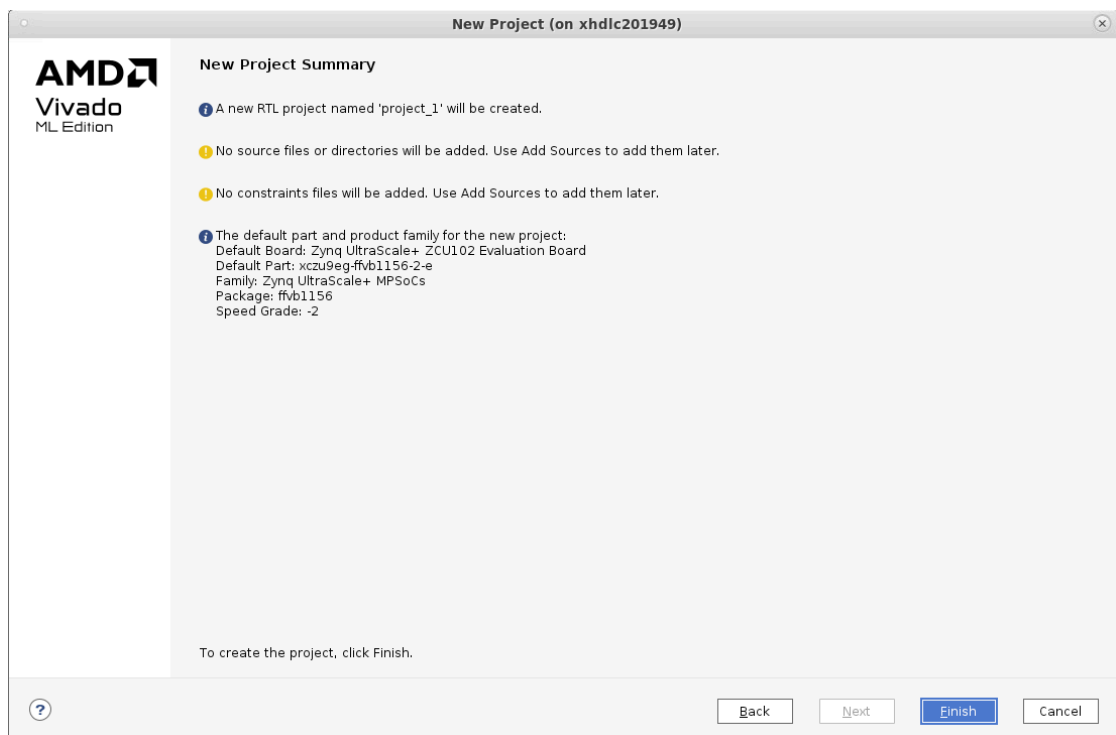


2. In the Getting Started page, click Create Project to start the New Project wizard.
3. In the Project Name page, name the new project and enter the project location. Make sure to select the Create project subdirectory option, and click Next.
4. In the Project Type page, specify the type of project to create as RTL Project, make sure to deselect the Do not specify sources at this time option, and click Next.
5. In the Add Sources page, click Next.
6. (Optional) In the Add Existing IP dialog box, click Next.
7. (Optional) In the Add Constraints dialog box, click Next.

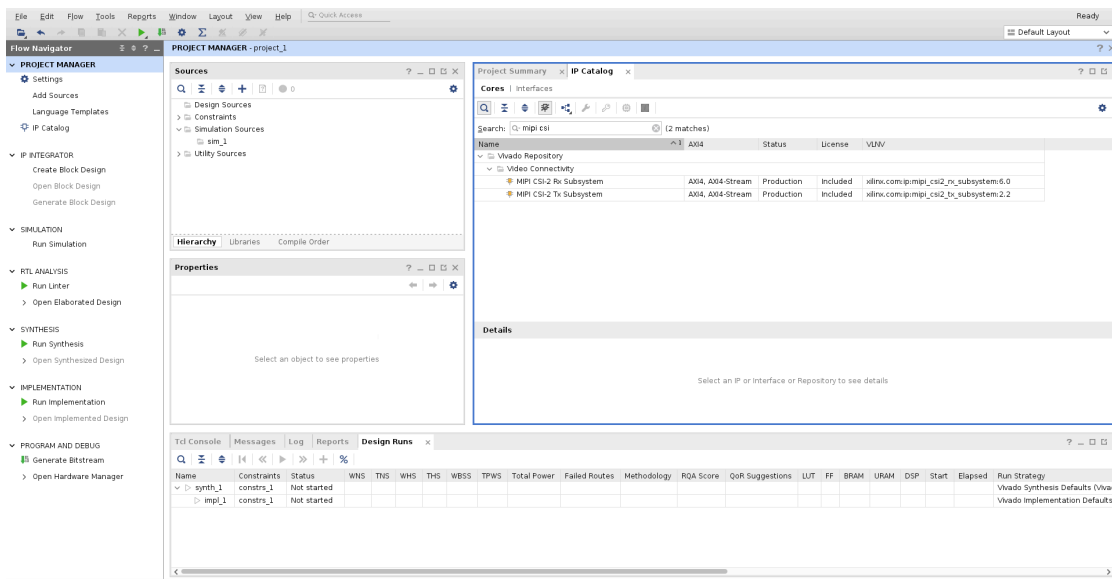
- In the Default Part dialog box, click Boards to specify the board for the target device (ZCU102, SP701, SCU200, VEK385, VEK280 and VCK190 boards are supported). Then click Next.



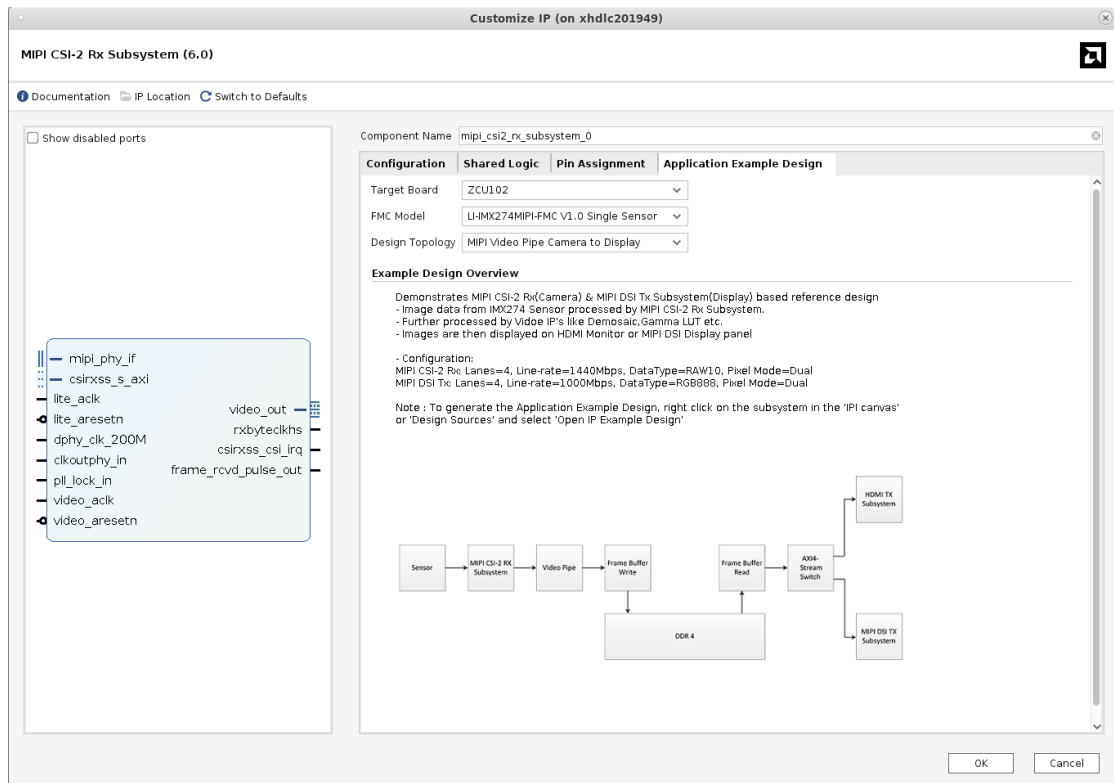
- Review the New Project Summary page. Verify that the data appears as expected, per the steps above, and click Finish.



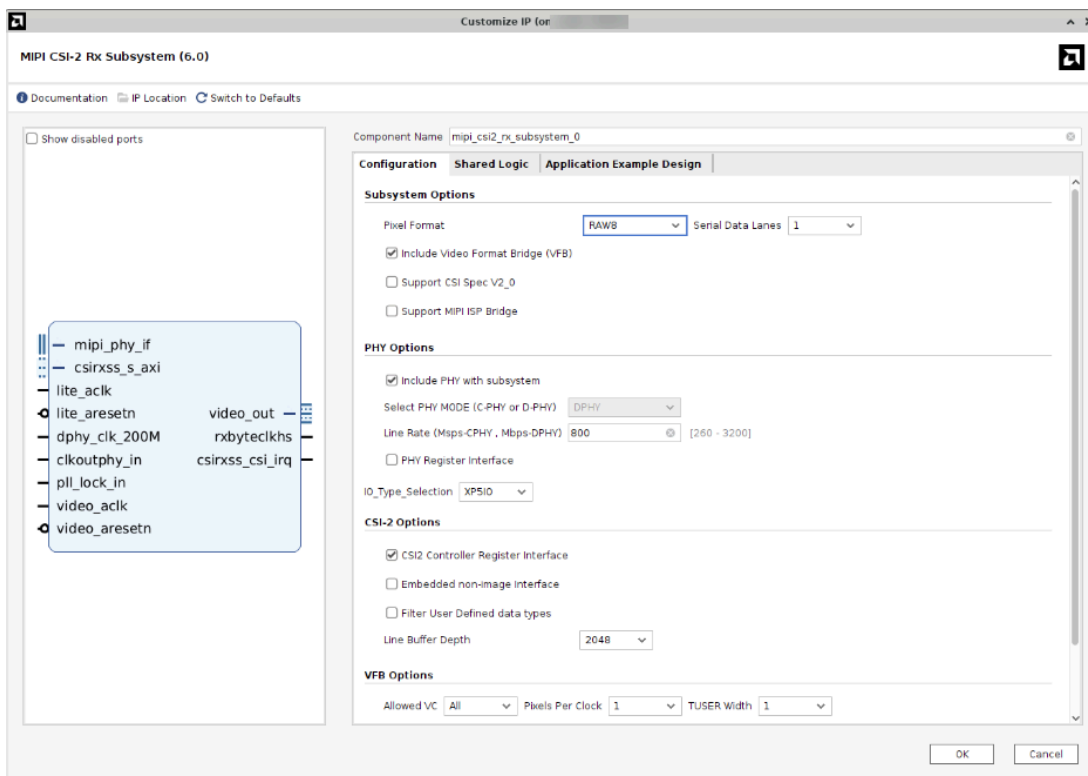
10. Click IP Catalog, select MIPI CSI-2 RX Subsystem under Video Connectivity, and double-click it.



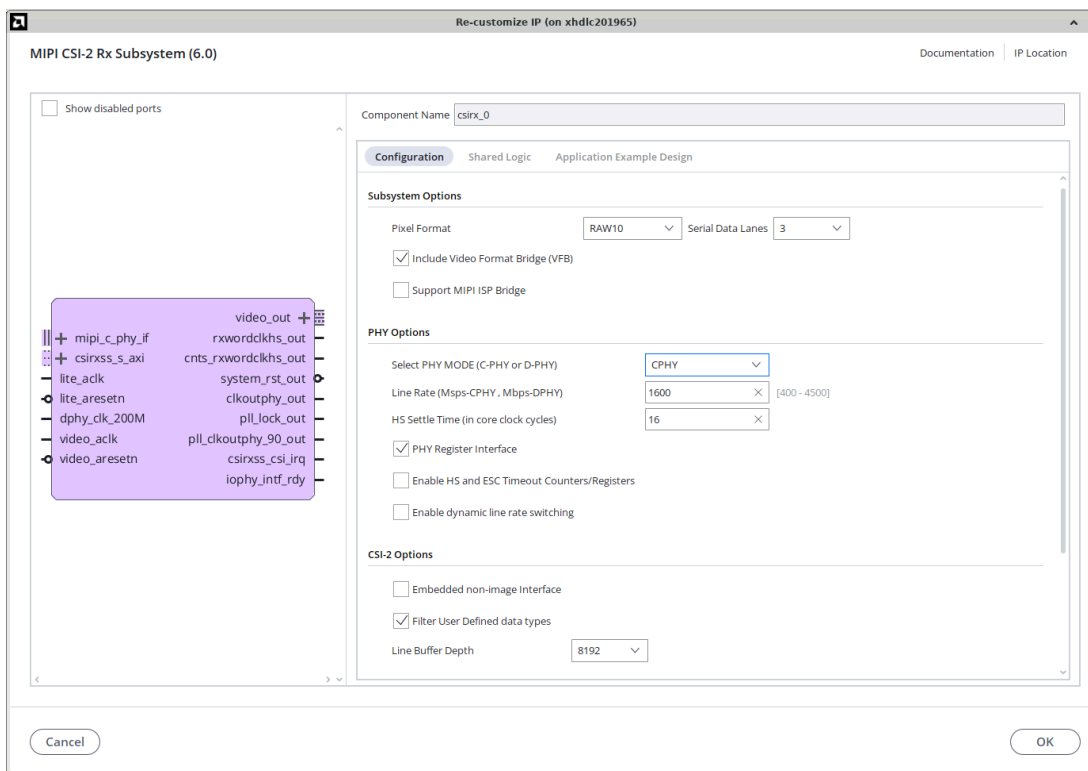
11. You can rename the IP component name. Configure the MIPI CSI-2 RX Subsystem Application Example Design tab to select the ZCU102, SP701, SCU200, VEK385, VCK190, or VEK280 board-based design, then click OK.



Note: For SCU200 board-based designs, ensure that you configure the IO_Type_selection parameter to either XP5IO or HPIO, based on the required I/O type as shown in the following screenshot.

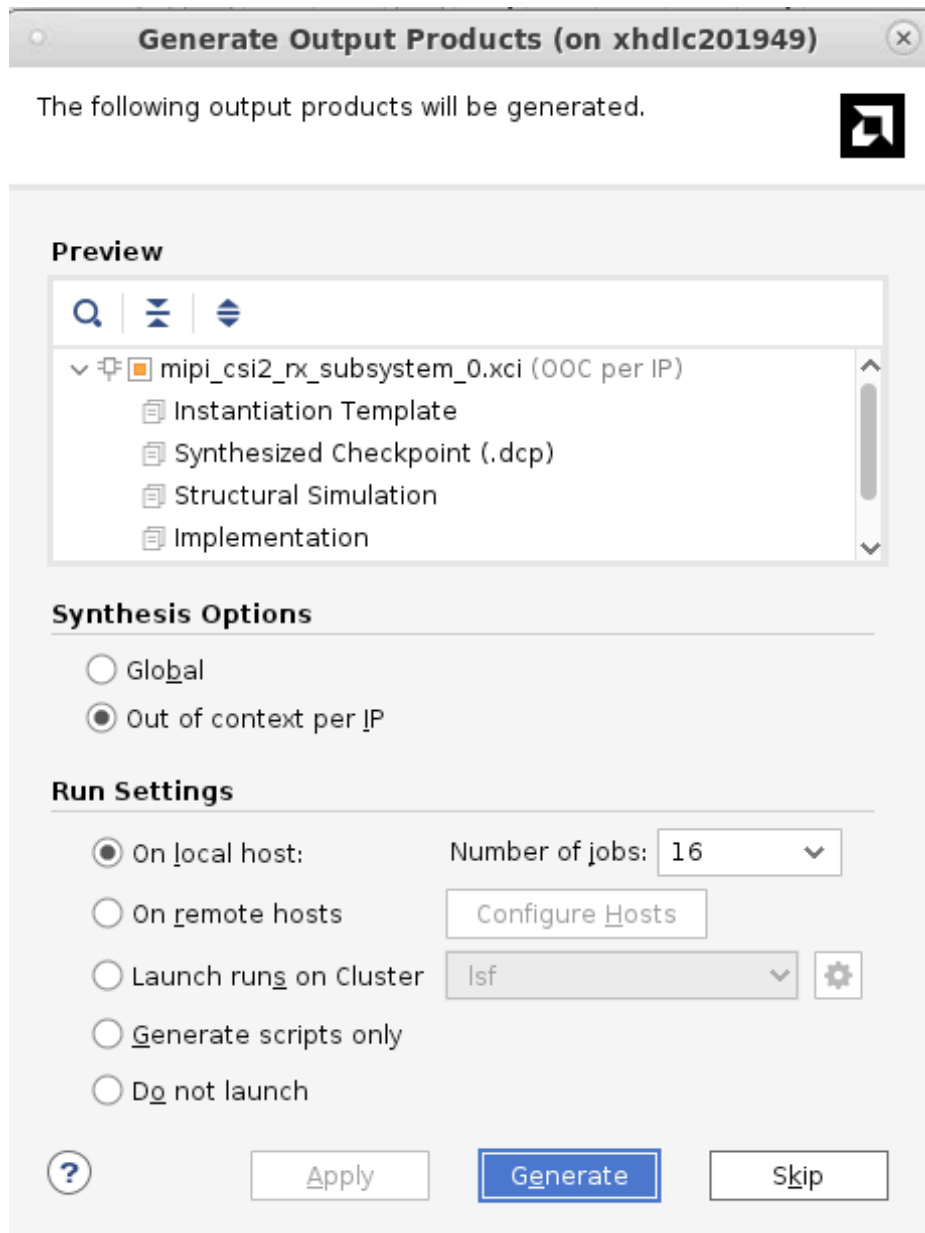


Note: For VEK385 board-based designs, ensure you configure the PHY_MODE parameter to either CPHY or DPHY as shown in the following screenshot.



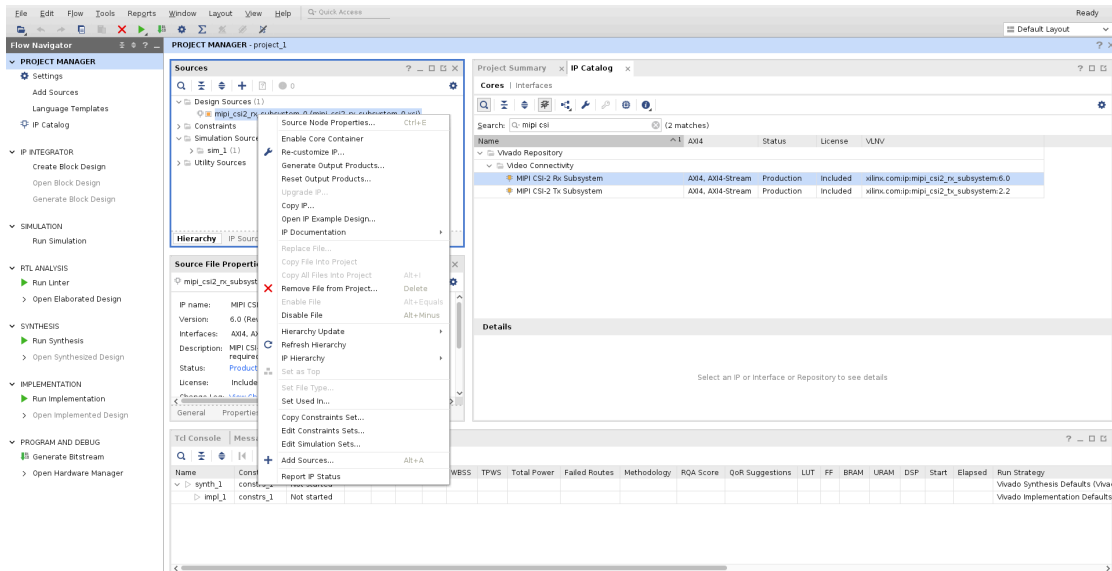
The Generate Output Products dialog box displays.

12. Click Generate. You can optionally click Skip if you want to skip generating the output products.



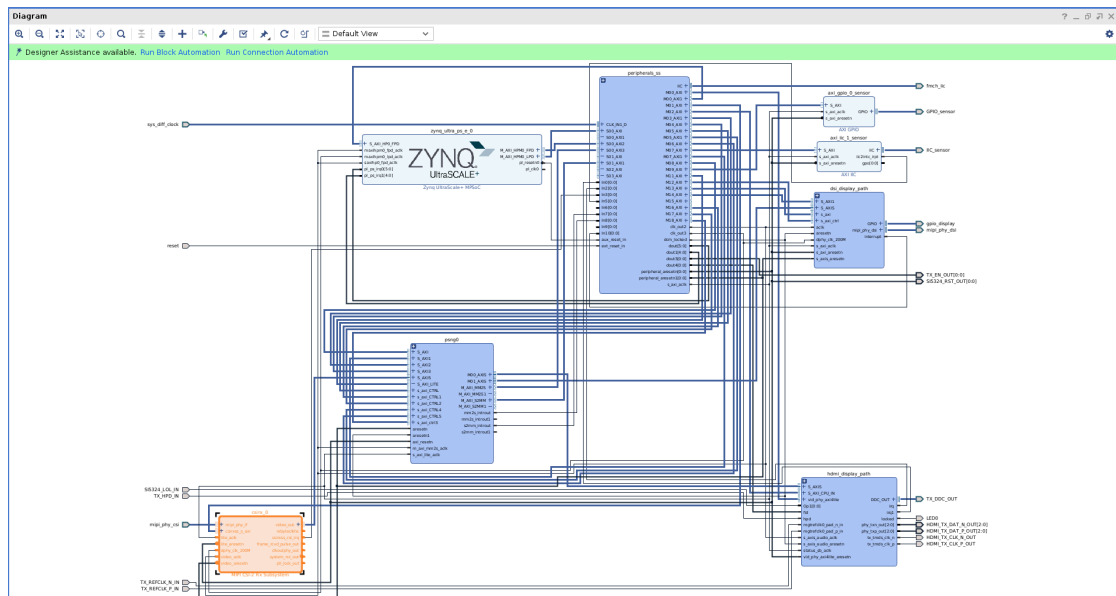
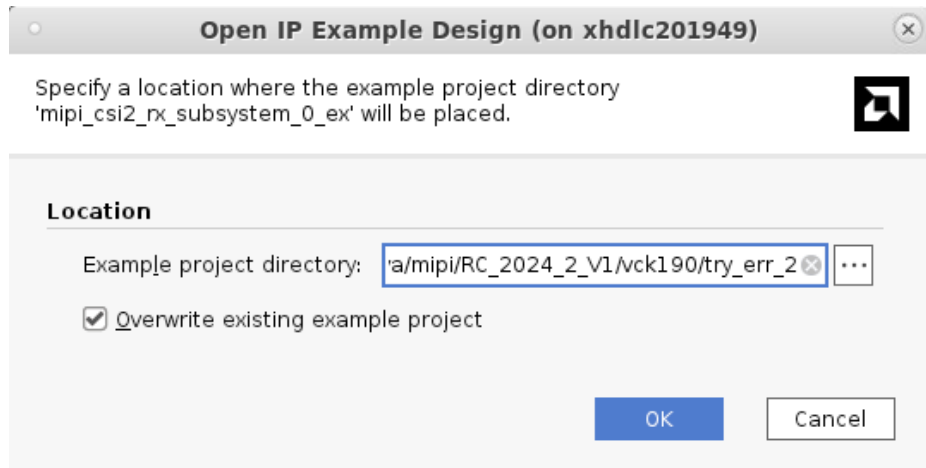
- Right-click the MIPI CSI-2 RX Subsystem component under Design source, and click Open IP Example Design.

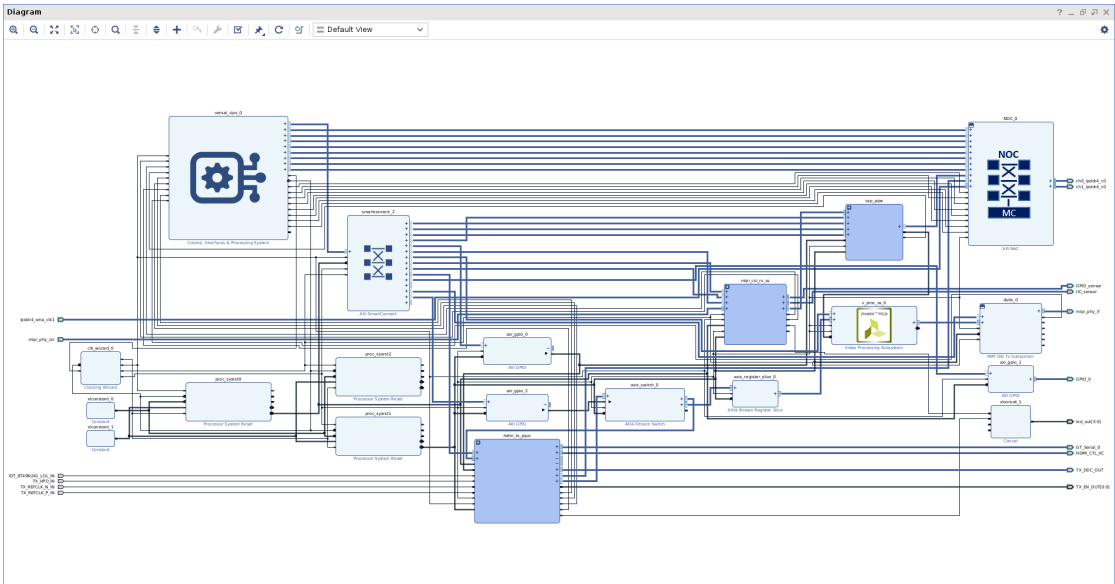
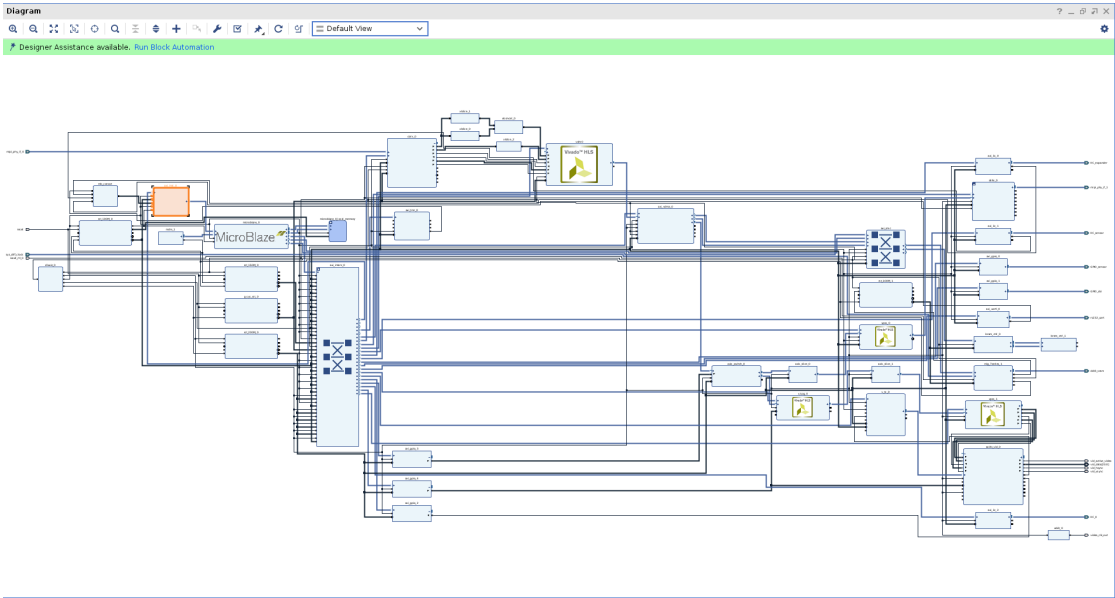
Note: Because this step involves the generation of the complete system including multiple subsystems, it takes some time to completely build the design.



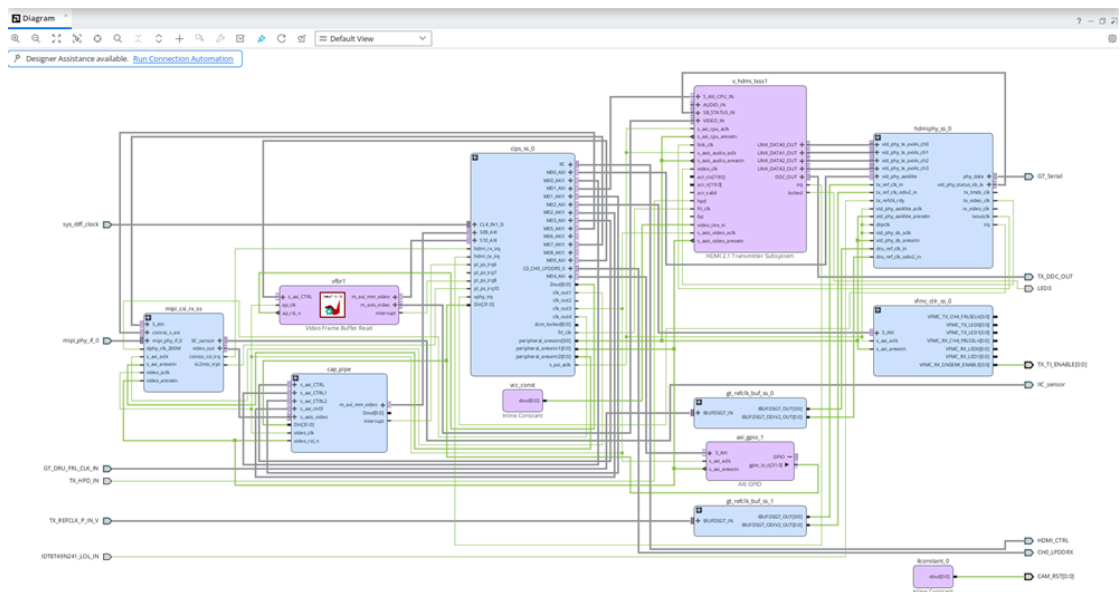
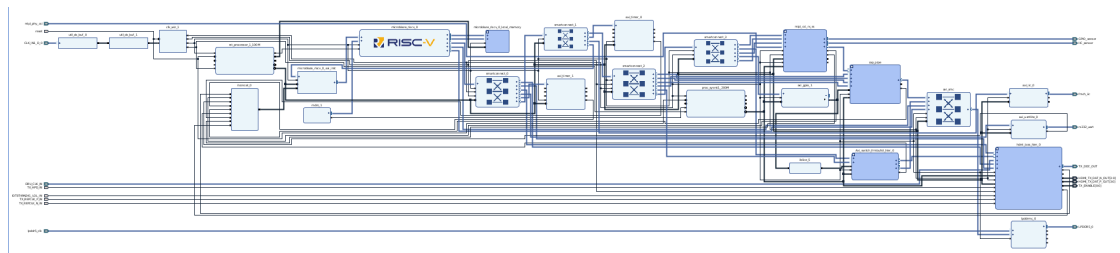
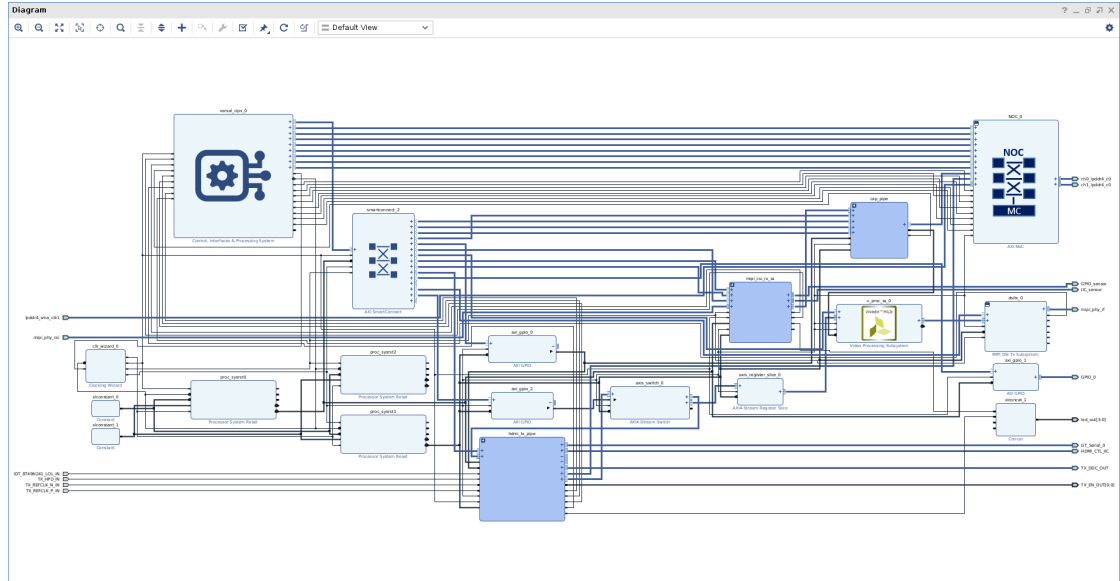
14. Choose the target project location, then click OK.

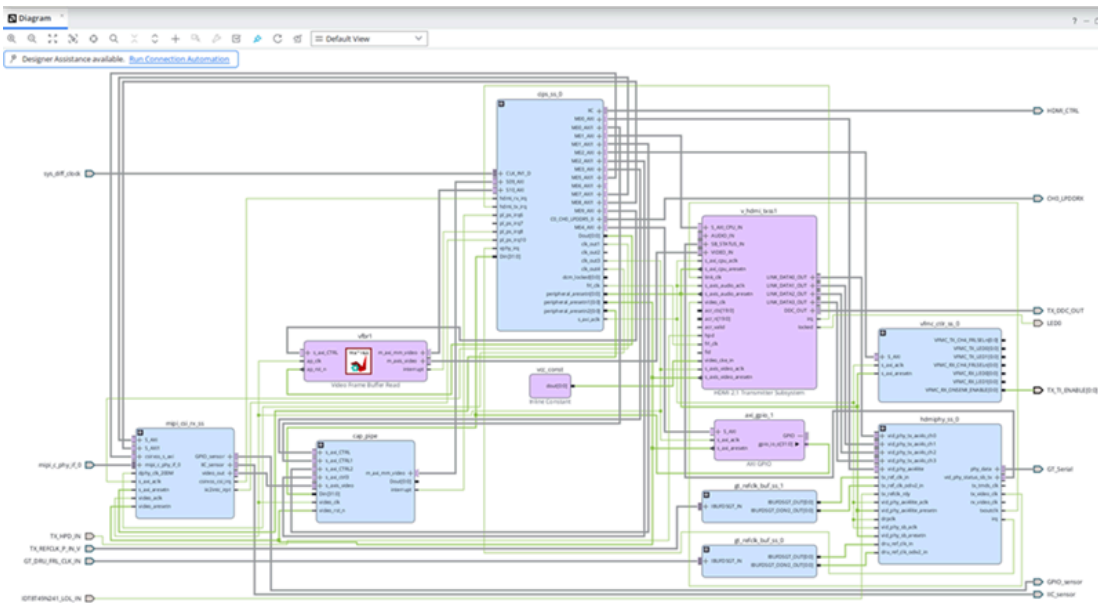
The overall system IP integrator block diagram of the ZCU102, SP701, SCU200, VEK385, VEK280, or VCK190 board-based application example design is generated depending on the GUI option selected. You can choose to Run Synthesis, Implementation, or Generate Bitstream (Generate Device Image for the VEK280, VEK385, or VCK190 board).





15. Select File > Export > Export hardware (.xsa).



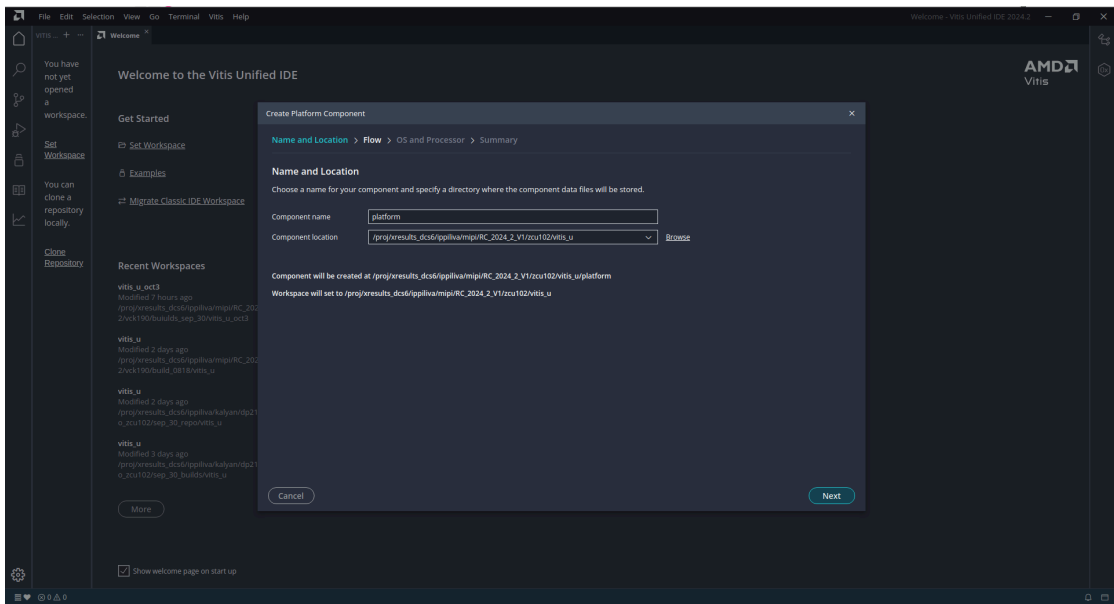


16. Select Tools > Launch Vitis IDE. Browse to your project location.

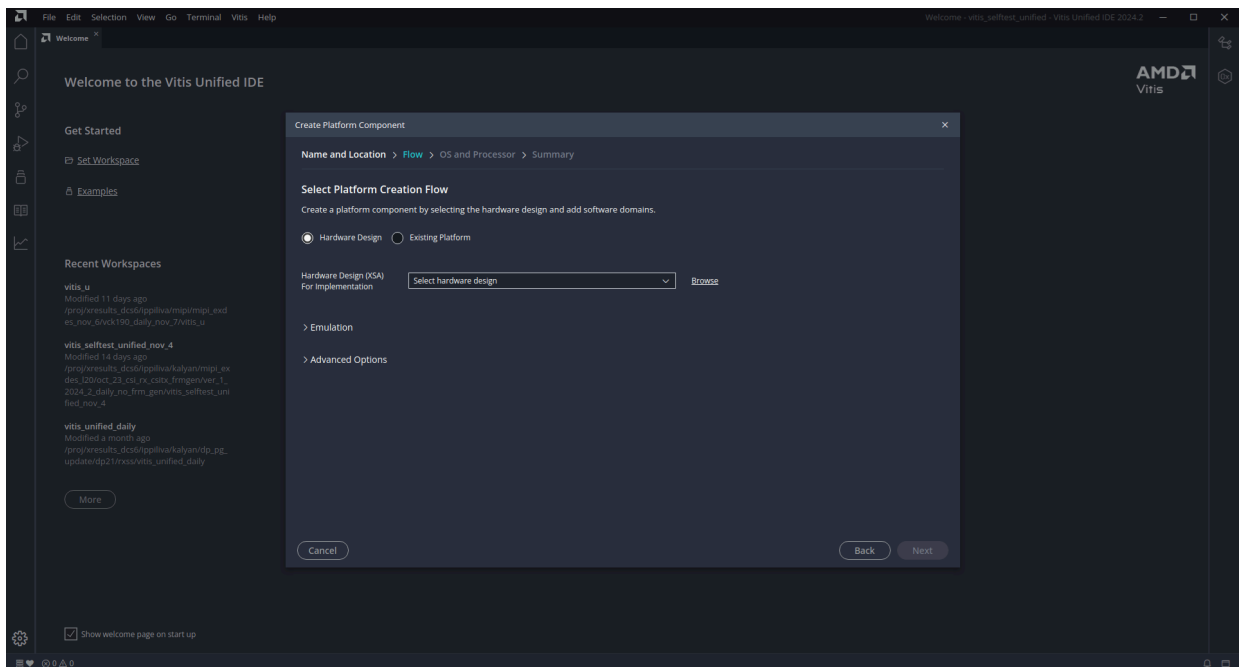
Build a Software Application Using the Unified Vitis Flow

1. To create a new platform project using the generated .xsa file, click on Create Platform Component under Embedded Development tab in the Unified Vitis IDE window, and execute the following steps.

- Set the component name, set the location to create the platform component, and click Next.



- Click Browse, select the generated XSA from the design, and click Next.



4. Click Next.

The screenshot shows the 'Create Platform Component' dialog box with the 'OS and Processor' step selected in the breadcrumb navigation. The title bar reads 'Create Platform Component'. The breadcrumb path is 'Name and Location > Flow > OS and Processor > Summary'. The main heading is 'Select Operating System and Processor'. Below it, a text block states: 'Specify the details for the initial domain to be added to the platform component. The platform can be modified later to add new domains or change settings by opening the platform editor.' There are three dropdown menus: 'Operating system', 'Processor', and 'Compiler'. At the bottom, there are three buttons: 'Cancel', 'Back', and 'Next'.

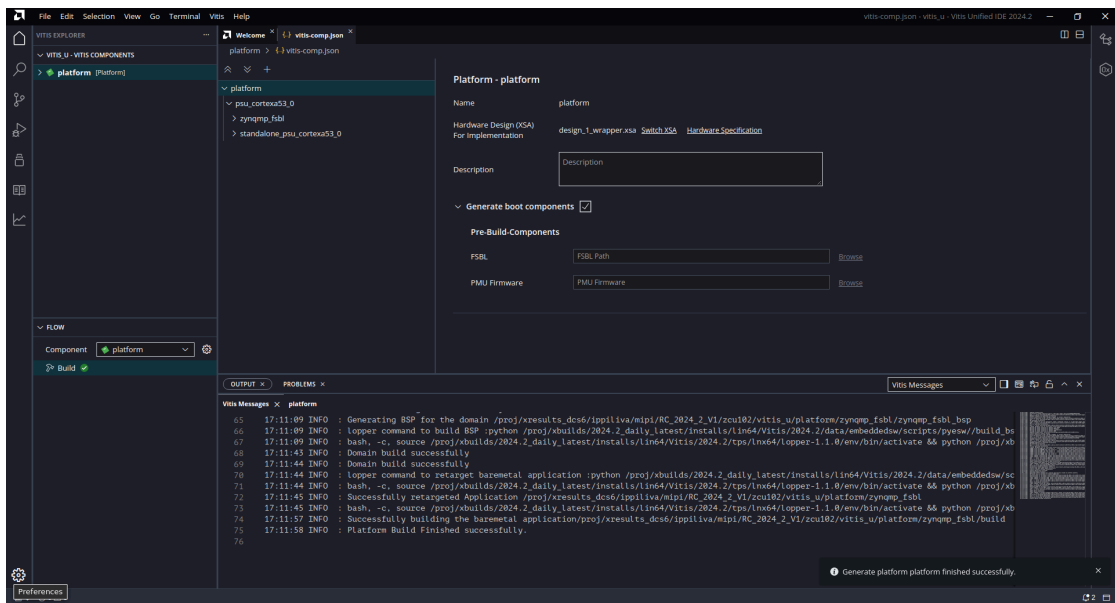
5. Click Finish.

The screenshot shows the 'Create Platform Component' dialog box with the 'Summary' step selected in the breadcrumb navigation. The title bar reads 'Create Platform Component'. The breadcrumb path is 'Name and Location > Flow > OS and Processor > Summary'. The main heading is 'Summary'. Below it, a text block states: 'The following new Platform component will be created.' There is a table with the following details:

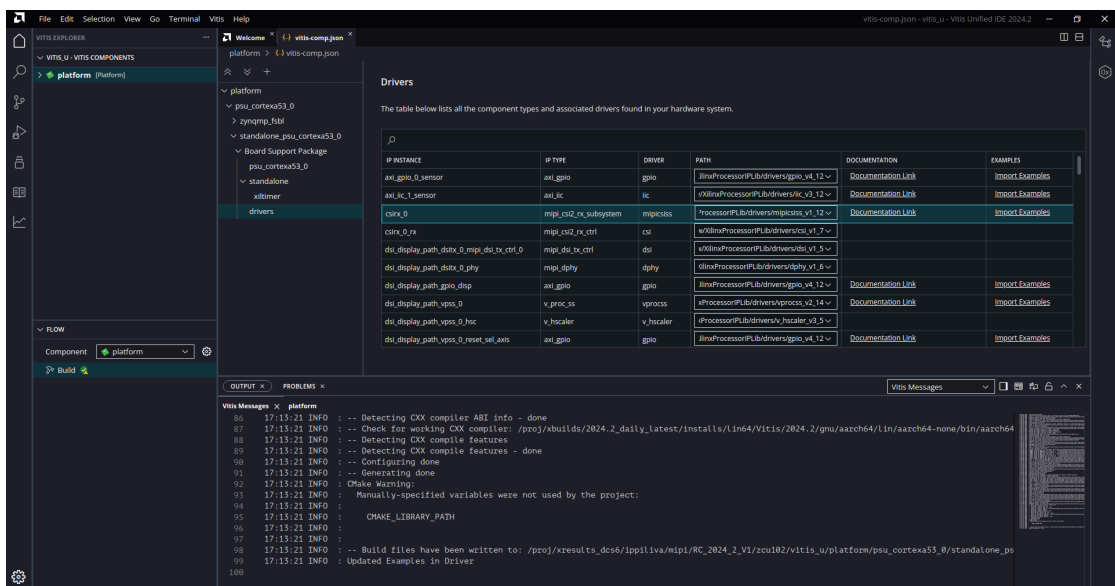
Name	platform
Workspace	
Component directory	
Compiler	GCC
Operating system	standalone
Processor	

Below the table, there is a note with a yellow warning icon: 'Note: A domain with selected operating system and processor will be added to the platform. The platform can be modified later to add new domains or change settings.' At the bottom, there are three buttons: 'Cancel', 'Back', and 'Finish'.

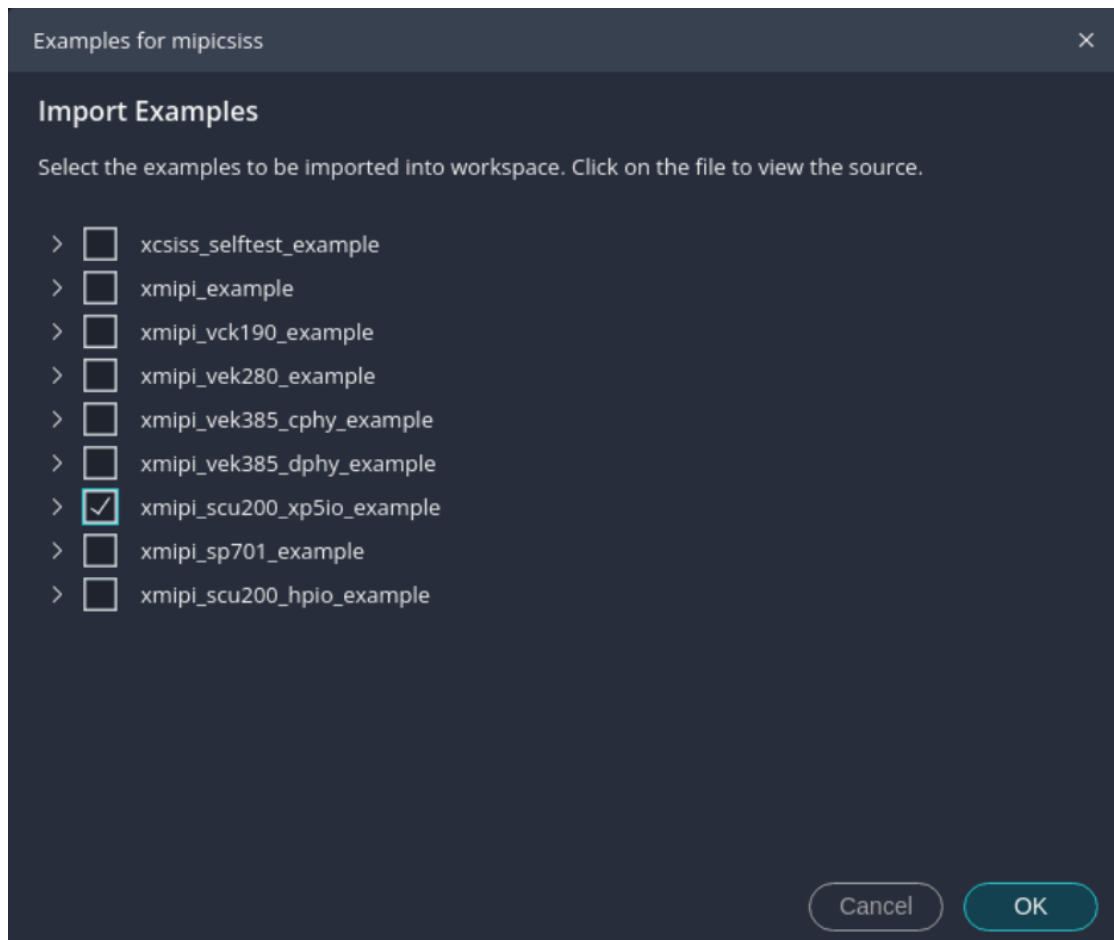
6. Select the platform in the VITIS COMPONENTS window, and click Build (hammer icon).



7. When the platform is created, click Board support package > drivers > mipicsiss driver, and then click Import Examples.

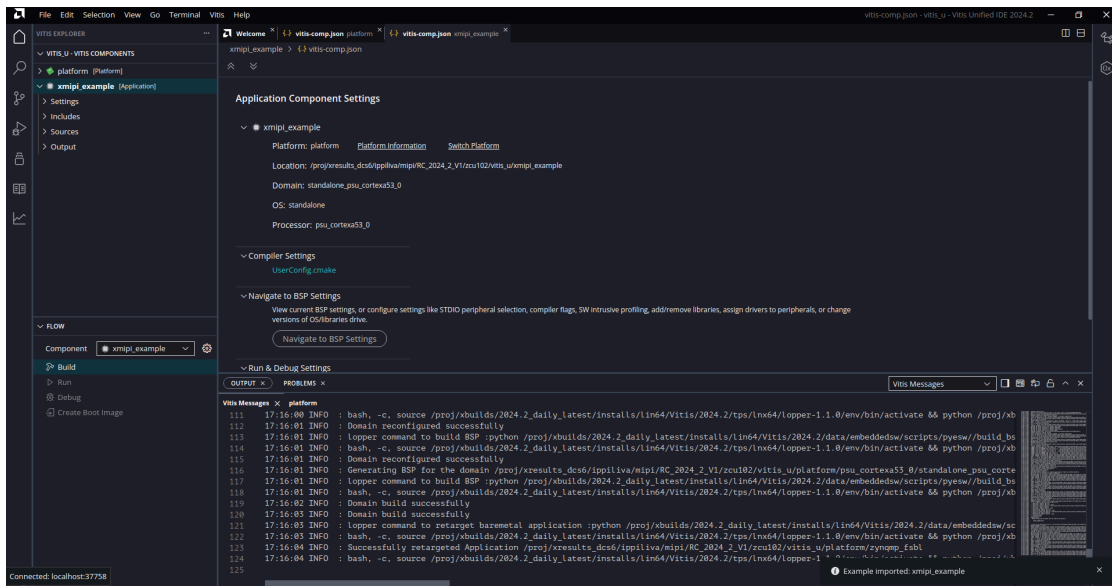


8. Next, import the example and select the application corresponding to your hardware.



- a. For the SP701 board-based example design, select application xmipi_sp701_example.
- b. For the ZCU102 board-based example design, select application xmipi_example.
- c. For the VCK190 board-based example design, select application xmipi_vck190_example.
- d. For the VEK280 board-based example design, select application xmipi_vek280_example.
- e. For the SCU200 XP5IO-based example design, select application xmipi_scu200_xp5io_example.
- f. For the SCU200 HPIO-based example design, select application xmipi_scu200_hpio_example.
- g. For the VEK385 D-PHY-based example design, select application xmipi_vek385_dphy_example.
- h. For the VEK385 C-PHY-based example design, select application xmipi_sek385_cphy_example

9. Click the imported example application, and then click Build.



The design is built and ready to use.

Running the Design on Hardware

Important: Before running the design on the Hardware, build the project and generate the required bit/elf files. Refer to [Implementing the Example Design](#) for more details.

1. Connect the JTAG cable and USB-UART cable to the board.
2. Go to the Application Example Design Project directory.

```
Example: cd ./<IP instance name>_ex
```

3. Launch the Xilinx System Debugger by selecting Start > All Programs > Xilinx Design Tools > Vivado > Vivado Tcl Shell.
4. Invoke Xilinx System Debugger.

```
(xsdb)Vivado% xsdb
```

5. Establish connections to debug targets.

```
xsdb% connect
```

6. List all available JTAG targets

```
xsdb% targets
```

Note: The target number for the PSU, PL, RPU, and Cortex-R5F might be different. Run the targets and ensure that they are using the correct target number.

7. Download the bitstream to the FPGA.

```
xsdb% fpga -file ./<ip_instance_name>_ex.runs/impl_1/design_1_wrapper.bit
```

Note: For Versal AI Edge Series Gen 2, both the boot.pdi and pld.pdi files must be downloaded.

```
xsdb% dev p -file
./<ip_instance_name>_ex.runs/impl_1/design_1_wrapper_boot.pdi
xsdb% dev p -file
./<ip_instance_name>_ex.runs/impl_1/design_1_wrapper_pld.pdi
```

8. Set and reset the corresponding target processor.

```
xsdb % targets -set -filter {name =~ "Cortex-A53 #0"}/{name =~ "Cortex-A72
#0"}/{name =~ "Cortex-A78 #0"}
xsdb % rst -processor
```

9. Download and start the application:

```
xsdb % dow -force
vitis_workspace>/<platform_project>/zynqmp_fsbl/fsbl_a53.elf
```

Note: Skip the preceding step for SP701, SCU200, or VEK280/VCK190/VEK385-based example designs.

```
xsdb % dow -force
<vitis_workspace>/xmipi_example_1/Debug/xmipi_example_1.elf
For VEK385 - DPHY
xsdb % dow -force
<vitis_workspace>/xmipi_example_1/Debug/xmipi_vek385_dphy_example.elf
For VEK385 - CPHY
xsdb % dow -force
<vitis_workspace>/xmipi_example_1/Debug/xmipi_vek385_cphy_example.elf
```

Run the application using the `con` command in xsdb prompt.

Note: The corresponding application files are as follows:

- For SP701 board-based example designs, xmipi_sp701_1.elf
- For VEK280 board-based example designs, xmipi_vek280_1.elf
- For VCK190 board-based example designs, xmipi_vck190_1.elf
- For SCU200.XP5IO- based example designs, xmipi_scu200_xp5io_example.elf
- For SCU200.HPIO- based example designs, xmipi_scu200_hpio_example.elf
- For VEK385_DPHY- based example designs, xmipi_vek385_dphy_example.elf
- For VEK385_CPHY- based example designs, xmipi_vek385_cphy_example.elf

Note: Configure the target processor based on the board design:

- For the SP701 /SCU200 board-based example design, set the target processor as MicroBlaze.
 - For the VCK190 board based example design, set the target processor as Cortex-A72.
 - For the VEK385 board based example design, set the target processor as Cortex-A78.
-

10. For the ZCU102 board-based example design, to observe the results, start a Hyper Terminal program on the host PC and configure its serial port (Interface 0) to 115200 baud rate with the default configuration. Ensure that the UART cable is connected to the board and the PC. The UART console displays a menu in the console. You are prompted for design related inputs.

- a. Initially, the application asks you if the camera sensor and display panel are connected. Enter either y or n.

Note: If you enter n, it is assumed that the camera or the DSI panel or both are not available. The system displays the camera sensor as Disconnected, and/or the DSI Display panel as a Disconnected error message on the console.

- b. Under the Main Menu, select the video source, display device, and resolution details.

- Press s to select the Sensor as video source and show live sensor data capture.
- Press t to select the Video Test Pattern generator as the video source and shows rainbow pattern on screen.
- Press h to switch the display to HDMI monitor if not already displayed.
- Press d to switch the display to DSI panel.
- Press r to bring up the resolution menu.

Note: Selecting an invalid option prompts an Unknown option error message on the console. All resolutions support only four lanes. The supported lane and other pipeline configurations are listed under the Current Pipe Configuration section displayed on the console.

11. For the SP701 board-based example design, to observe the results, start a Hyper Terminal program on the host PC and configure its serial port (Interface 0) to 9600 baud rate with the default configuration. Ensure that the UART cable is connected to the board and the PC. The UART console displays a menu in the console. Under the Main Menu, you are prompted to select the preferred output.

- a. Press 1 to select the display to DSI panel.
- b. Press 2 to switch the display to HDMI monitor.

12. For the VCK190 board-based example design, to observe the results, start a Hyper Terminal program on the host PC and configure its serial port (Interface 0) to 115200 baud rate with the default configuration. Ensure that the UART cable is connected to the board and the PC. The UART console displays a menu in the console. Under the Main Menu, you are prompted to select the preferred output.

```

-----
  Uersal MIPI CSI RX Design Example  -----
  <c> 2019 by Xilinx, Inc.  -----
Build Apr 30 2020 - 04:50:13
-----

  MAIN MENU  -----
0 - 1920x1080p60
=> Configures Sensor for 1920x1080 60fps.
1 - 3840x2160p60
=> Configures Sensor for 3840x2160 60fps.

Enter Selection ->

```

- 0: (1920x1080p60)
- 1:(3840x2160p60)

13. For the VEK385 board-based example design, to observe the results, start a Hyper Terminal program on the host PC and configure its serial port (Interface 0) to 115200 baud rate with the default configuration. Ensure that the UART cable is connected to the board and the PC. The UART console displays a menu in the console. Under the Main Menu, you are prompted to select the preferred output.

D-PHY:

```

-----
-- Universal MIPI CSI RX Design Example --
-- (c) 2019 by Xilinx, Inc. -----
Build May 25 2026 - 16:39:14
Please answer the following questions about the hardware setup.

*Init IIC is Done

*Init CSI is Done

-----

-- HDMI 2.1 SS + HdmiPhy URR Example v1.0 --
-- (c) 2019 by Xilinx, Inc. -----
Build May 25 2026 - 16:39:14
-----
Initializing IIC and clock sources.
Initializing HDMI Video Transmitter.
TX: FRL Base: 0xB0020180
Initializing Video Phy with Video Transmitter
HDMI Video Transmitter system setup successful.
ID1_8149N241 device ID 0x606
success
UFMC active HDMI TX <T1> Mezz <R3> Detected
Initializing Example design controller.

Inside Exdes_ConfigureIpgEnableInput ...
VERBOSITY is disabled : 0
Set TX stream to HDMI TMDS, sink is HDMI

EDID Parsing Pass
System timer configured to generate (and count)pulses at 100ns intervals.
HDMI TX Configuration Done.
Tx Transmit is 1
TMDS Clock:148500000
Failed to select TX Ref clock.
HdmiPhyHdmiTxReadyCallback:178.
Inside Exdes_ConfigureIpgEnableInput ...
Tx stream is up in colorbar

Colorbar :
Color Format:      RGB
Color Depth:      8
Pixels Per Clock: 4
Mode:             Progressive
DSC Status:       Uncompressed
Frame Rate:       60Hz
Resolution:       1920x1080@60Hz
Pixel Clock:      148500 kHz
TX Mode:          TMDS

XU_Ix_VideoSetupAndStart Success

-- MAIN MENU --
h - Select Display Device : HDMI

```

- C-PHY:

```

-----
  Universal MIPI CSI RX Design Example
  (c) 2019 by Xilinx, Inc.
-----
Build May 25 2026 - 11:02:06
Please answer the following questions about the hardware setup.

*Init IIC is Done

*Init CSI is Done

-----

  HDMI 2.1 SS + HdmiPhy URR Example v1.0
  (c) 2019 by Xilinx, Inc.
-----
Build May 25 2026 - 11:02:06
Initializing IIC and clock sources.
Initializing HDMI Video Transmitter.
TX: FRL Base: 0x00020180
Initializing Video Phy with Video Transmitter
HDMI Video Transmitter system setup successful.
IDT_BT49N241 device ID 0x606
success
UFMC Active HDMI TX (T1) Mezz (R3) Detected
Initializing Example design controller.
Inside Exdes_ConfigureTpgEnableInput ...
VERBOSITY is disabled : 0
Set TX stream to HDMI TMDS, sink is HDMI

EDID Parsing Pass
System timer configured to generate (and count)pulses at 100ns intervals.
HDMI TX Configuration Done.
Tx Transmit is 1
TMDS Clock:148500000
Failed to select TX Ref clock.
HdmiPhyHdmiTxReadyCallback,178.
Inside Exdes_ConfigureTpgEnableInput ...
Tx stream is up in colorbar

Colorbar :
  Color Format:      RGB
  Color Depth:      8
  Pixels Per Clock:  4
  Mode:              Progressive
  DSC Status:        Uncompressed
  Frame Rate:        60Hz
  Resolution:         1920x1080@60Hz
  Pixel Clock:        148500 kHz
  TX Mode:           TMDS

XV_Ix_VideoSetupAndStart Success

-----
  MAIN MENU
-----
h - Select Display Device : HDMI
r - Change the video resolution 2K/4K.
p - 1920x1080p60 -> Configures Sensor for 1920x1080 60fps.
q - 3840x2160p60 -> Configures Sensor for 3840x2160 60fps.

Enter Selection ->

```

Note: Selecting an invalid option prompts an Unknown option error message on the console. All resolutions support only four lanes. The supported lane and other pipeline configurations are listed under the Current Pipe Configuration section displayed on the console.

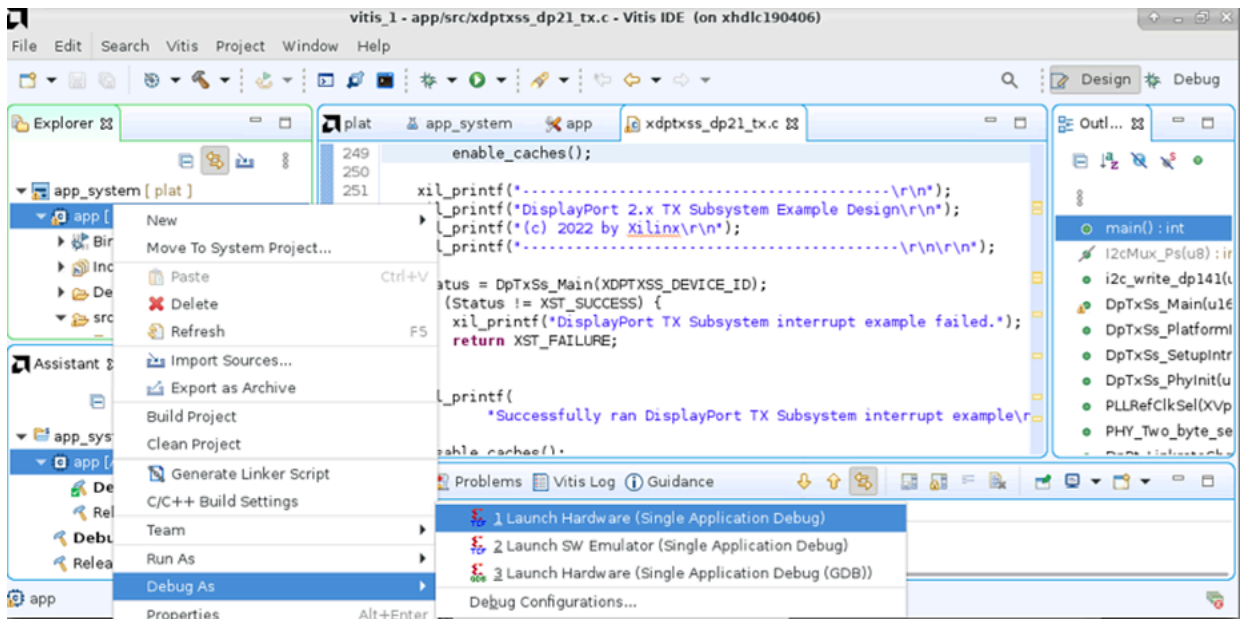
- For the SCU200 XP5IO/HPIO based example designs, to observe the results, start a Hyper Terminal program on the host PC and configure its serial port (Interface 0) to 115200 baud rate with the default configuration. Ensure that the UART cable is connected to the board and the PC.
- The UART console displays a menu in the console. Under the Main Menu, you are prompted to select the preferred output. Press h to select the display to HDMI monitor.

Vitis Debugger Flow to Program FPGA Instead of

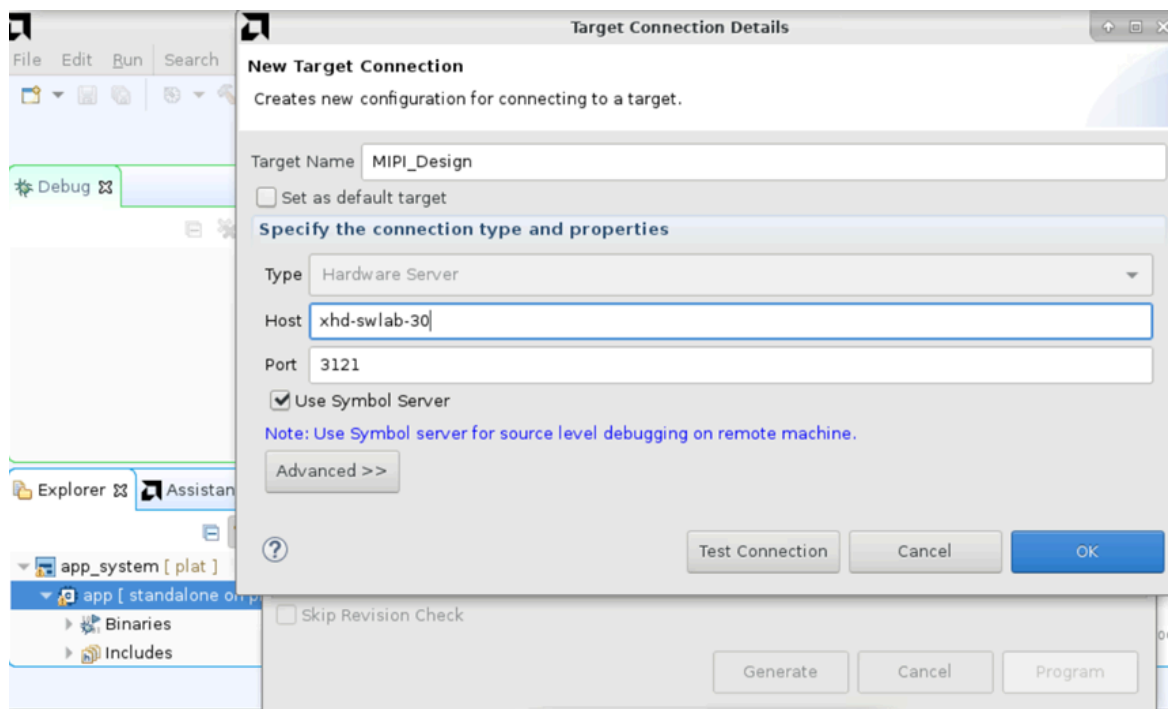
XSDB Console

1. Click the Debug as option in AMD Vitis™ .

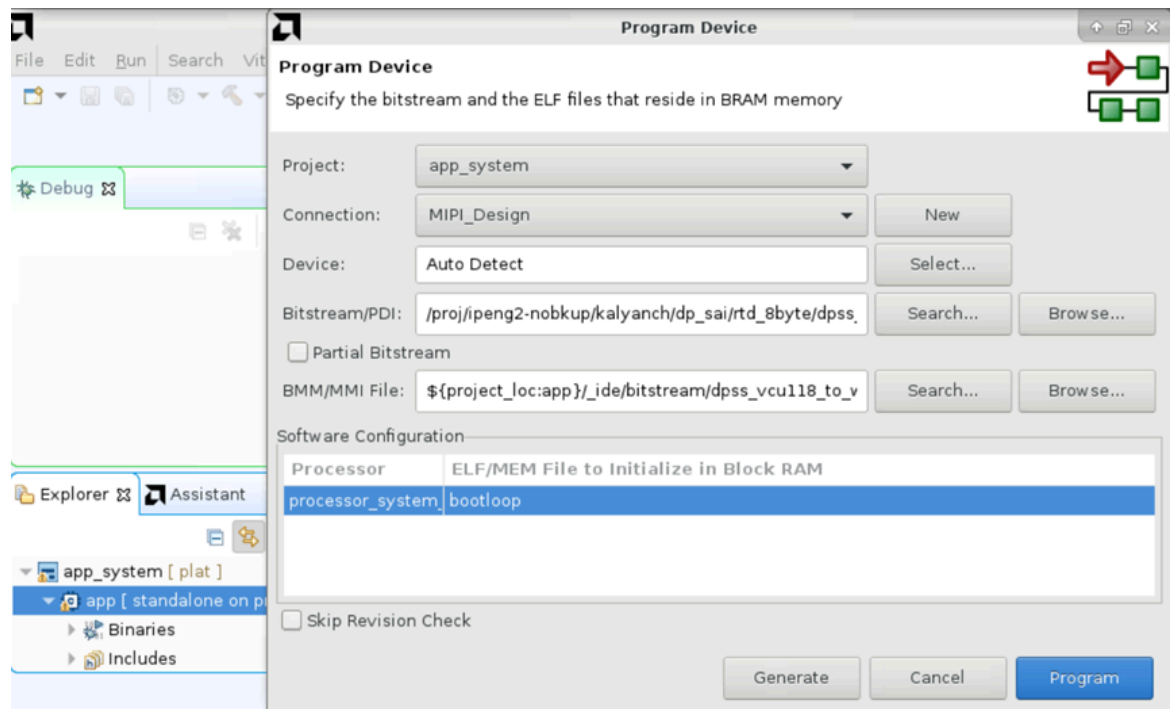
2. Select Launch Hardware (Single Application Debug).



- On the Main tab, ensure that your connection is correct and the right project is selected.
- On the Target Setup tab, ensure:
 - Hardware Platform is correct
 - Bitstream File is correct
 - FSBL is selected
 - Reset Entire System is selected
 - Program FPGA is selected



c. Click Debug and wait for the FPGA to program device.



d. After completion, click Play to start the software.

Verification, Compliance, and Interoperability

The MIPI CSI-2 RX Subsystem has been verified using both simulation and hardware testing. A highly parameterizable transaction-based simulation test suite has been used to verify the subsystem. The tests include:

- Different lane combinations and line rates
- High-Speed Data reception with short/long packets, different virtual channels and different data types.
- All possible interleaving cases (data type and virtual channel)
- All possible output pixel, data type combinations.
- Recovery from error conditions
- Register read and write access

Hardware Validation

The MIPI CSI-2 RX subsystem is tested in hardware for functionality, performance, and reliability using AMD evaluation platforms. The MIPI CSI-2 RX Subsystem verification test suites for all possible modules are continuously being updated to increase test coverage across the range of possible parameters for each individual module.

A series of MIPI CSI-2 RX Subsystem test scenarios are validated using the following AMD development boards. These boards permit the prototyping of system designs

where the MIPI CSI-2 RX subsystem processes different short/long packets received on serial lines.

Table: AMD Development Board

Target Family	Evaluation Board	Characterization Board
AMD Zynq™ UltraScale+™ MPSoC	ZCU102	N/A
Zynq 7000 SoC	ZC702	N/A
AMD Spartan™ 7 FPGA	SP701	N/A
AMD Versal™ Adaptive SoC	VCK190	N/A
Versal Adaptive SoC	VEK280	N/A
Versal Adaptive SoC	VEK385	N/A

AMD 7 series FPGAs do not have a native MIPI IOB support. You need to target either the HR bank I/O or the HP bank I/O for the MIPI IP implementation. For more information on MIPI IOB compliant solution and guidance, refer to D-PHY Solutions ([XAPP894](#)).

A series of interoperability test scenarios listed in the following tables are validated using different core configurations and resolutions. All ZCU102 designs use the native MIPI I/O available in the AMD UltraScale+™ FPGA.

 **Note:** Sony IMX274 tested in continuous & non-continuous clock mode. All other sensors operating in continuous clock mode.

Table: Interoperability Testing UltraScale+ Device

Sensor	Board/Device	Tested Configuration	Resolution
Omnivision OV13850	ZCU102/xczu9eg -ffvb1156-2-e	1200 Mb/s	480p@60 fps
		1, 2, 4 Lanes	720p@60 fps
		RAW8, RAW10, RAW12	1080p@60 fps
			4k@30 fps

Sensor	Board/Device	Tested Configuration	Resolution
Sony IMX274	ZCU102/xczu9eg-ffvb1156-2-e	1440 Mb/s 4 Lanes RAW10, RAW12	All supported modes by sensor
Sony IMX224	ZCU102/xczu9eg-ffvb1156-2-e	149 Mb/s, 594 Mb/s 1, 2, 4 Lanes RAW10, RAW12	All-pixel (QVGA) and Window cropping modes
ON Semi AR0330	ZCU102/xczu9eg-ffvb1156-2-e	490 Mb/s 4 Lanes RAW10	480p@60 fps 720p@60 fps 1080p@60 fps
Sony IMX334	ZCU102/xczu9eg-ffvg1156-2-e	1782 Mb/s 4 Lanes RAW12	4K@30 fps
Sony IMX412	ZCU102/xczu9eg-ffvb1156-2-e	2100 Mb/s 4 Lanes RAW10	4K@60 fps

All AMD 7 series FPGA interop designs use the external Meticom (MC20901) based solution which implements MIPI D-PHY I/O.

Table: Interoperability Testing with AMD 7 Series FPGAs

Sensor	Board/Device	Tested Configuration	Calibration Mode	Resolution
Sony IMX274	ZC702/xc7z020clg484-1	576 Mb/s 4 Lanes RAW10	Auto Enable 300 MHz clock for IDELAYCTRL=false	1080p@60 fps

Sensor	Board/Device	Tested Configuration	Calibration Mode	Resolution
Sony IMX274	ZC702/ xc7z020clg484-1	1152 Mb/s 2 Lanes RAW10	Auto Enable 300 MHz clock for IDELAYCTRL= true	1080p@60 fps
ON Semi AR0330	KC705/xc7k3 25tffg900-2	490 Mb/s 4 Lanes RAW10	None	480p@60 fps 720p@60 fps 1080p@60 fps 2304x1296@ 60 fps
ON Semi AR0330	KC705/xc7k3 25tffg900-2	588 Mb/s 4 Lanes RAW12	None	1080p@60 fps 2304x1296@ 60 fps
ON Semi AR0330	ZC702/ xc7z020clg484-1	490 Mb/s 4 Lanes RAW10	None	480p@60 fps

All AMD 7 series FPGA loopback designs use the XM107 loopback card.

Table: Loopback Testing Using AMD Evaluation Boards

Board/Device	Line Rate	Lanes	Calibration Mode	Clock Selection (C_EN_CLK300M)
AC701 / xc7a200tfbg 676-2	1250	4	Auto	False

Board/Device	Line Rate	Lanes	Calibration Mode	Clock Selection (C_EN_CLK300M)
KC705/ xc7k325tffg900-2	1250	4	Auto	False
VC709/ xc7vx690tffg1761-2	1250	4	Auto	False
ZC702/ xc7z020clg484-1	928	4	Auto	True
ZC706/ xc7z045ffg900-2	1250	4	Auto	True
VCK190/xcvc1902- vsva2197-2MP-e	2496	4	Auto	False

Following board guidelines such as equal trace lengths helps to achieve higher line rates. For PCB guidelines, refer to the UltraScale Architecture PCB Design User Guide ([UG583](#)).

Debugging

This appendix includes details about resources available on the Support website and debugging tools.

Debug Tools

There are many tools available to address MIPI CSI-2 Receiver Subsystem design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The AMD Vivado™ Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in AMD devices.

The Vivado logic analyzer is used with the logic debug IP cores, including:

- ILA 2.0 (and later versions)
- VIO 2.0 (and later versions)

See the Vivado Design Suite User Guide: Programming and Debugging ([UG908](#)).

Hardware Debug

Hardware issues can range from link bring-up to problems seen after hours of testing. This section provides debug steps for common issues. The Vivado debug feature is a valuable resource to use in hardware debug. The signal names mentioned in the following individual sections can be probed using the debug feature for debugging the specific problems.

General Checks

- Ensure MIPI DPHY and MIPI CSI-2 RX Controller cores are in the enable state by reading the registers.
- Ensure Incorrect Lane Configuration is not set in the MIPI CSI-2 RX Controller Interrupt Status register.
- Ensure that the output AXI4-Stream bandwidth is high enough to meet throughput demand.
- Ensure line buffer full condition is not set in the MIPI CSI-2 RX Controller Interrupt Status register. Core setting this bit implies that the input data rate is higher than the output data rate. Consider either decreasing input data rate (D-PHY Line rate) or increase output data rate (select the appropriate output pixel per Clock: Single, Dual, Quad).

Line Buffer Depth Calculation for Versal AI Edge Series Gen2 and Versal Prime Series Gen 2 Devices

For Versal AI Edge Series Gen 2 and Versal Prime Series Gen 2 devices, you need to calculate the CSI buffer depth (CSI_BUF_DEPTH) based on the data type to prevent buffer overflow.

Condition to Avoid Buffer Overflow – (Number of bytes per line buffer) × CSI_BUF_DEPTH > 2 × WC (word count)

Example

RAW7 : $21 \times \text{CSI_BUF_DEPTH} > 2 \times \text{WC}$

RGB888 : $24 \times \text{CSI_BUF_DEPTH} > 2 \times \text{WC}$

Refer the following table for the number of bytes per line buffer calculations.

Table: Number of Bytes Per Line Buffer Calculations

Data Type	Bytes in Line Buffer
RGB888	24
RGB565	16
RGB666	18
RGB555	16
RGB444	16
RAW6	18
RAW7	21
RAW8	16
RAW10	20
RAW12	24
RAW14	28

Data Type	Bytes in Line Buffer
RAW16	16
RAW20	20
RAW24	24
YUV422-8bit	16
YUV422-10bit	20
YUV420-8bit	16

Monitoring Registers

The following MIPI CSI-2 RX Controller registers can be monitored to confirm reception of data packets:

- Packet count in Core Status register
- Data type and Byte count in Image Information registers
- Frame received bit in Interrupt Status register

No packets received by the MIPI CSI-2 Subsystem

- **Possible causes**
 - No packets received at MIPI DPHY level itself
 - Frame end packets not received or not passed the ECC checks at MIPI CSI-2 RX Subsystem level
- **Debug instructions**
 - Verify MIPI DPHY packet count registers. If the packet counts at MIPI DPHY level are not getting reported, debug connections/paths from source to MIPI DPHY Input
 - Verify MIPI CSI-2 RX Controller interrupt status register to see if any ECC errors getting reported. If there is frequent ECC 2-bit error getting reported means some of the packets are not getting processed by MIPI CSI-2 RX Controller.

Packets received by the MIPI CSI-2 Subsystem with PPI Level Errors (like SoT Error, SoT sync Error)

and/or Controller level errors (like ECC 1-bit, ECC 2-bit, CRC)

- **Possible causes**
 - Lane position mismatch between source (sensor) and MIPI CSI-2 RX Subsystem.
 - Noise detected by MIPI DPHY as a valid packet.
- **Debug instructions**
 - Verify that the lane positions of the source (sensor) and the MIPI CSI-2 RX Subsystem are matching. Lane0 holds first byte of the packet, Lane1 holds the next byte and so on.
 - Verify that sensor TX output timing. If it does not meet MIPI specification, adjust the MIPI D-PHY RX HS_SETTLE setting. When subsequently processed by the MIPI CSI-2 RX Controller, these packets are reported as erroneous packets. Increase the HS_SETTLE value either through the MIPI DPHY registers or through the C_HS_SETTLE_NS parameter (hidden) available in MIPI CSI-2 RX Subsystem.

Advanced Debug

For more debug information on MIPI D-PHY, refer to the MIPI D-PHY LogiCORE IP Product Guide ([PG202](#)). To debug further, capture the PPI signals using the Vivado Logic analyzer and confirm the bytes received through source (sensor) are as expected for short and long packets.

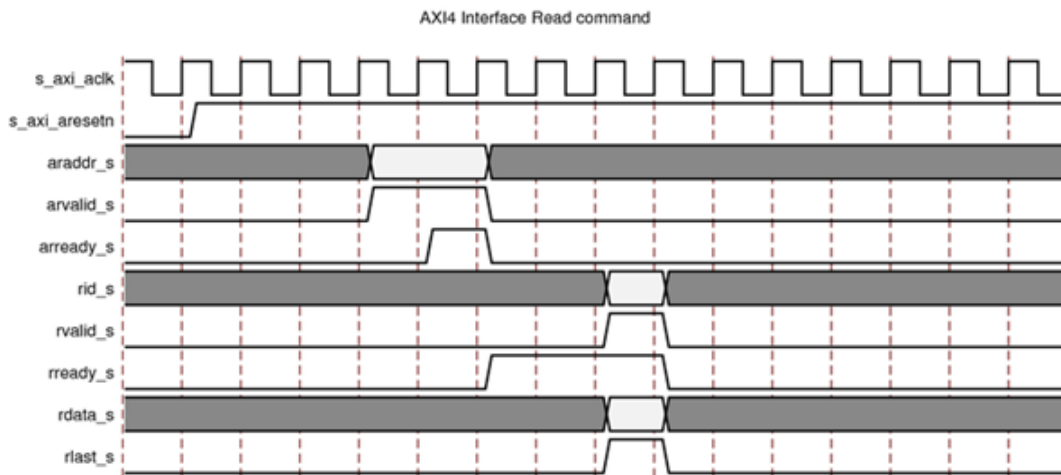
Interface Debug

AXI4-Lite Interfaces

Read from a register that does not have all 0s as a default to verify that the interface is functional. See the following figure for a read timing diagram. Output `s_axi_arready` asserts when the read address is valid, and output `s_axi_rvalid` asserts when the read data/response is valid. If the interface is unresponsive, ensure that the following conditions are met:

- The `lite_aclk` inputs are connected and toggling.
- The interface is not being held in reset, and `lite_aresetn` is an active-Low reset.
- The main subsystem clocks are toggling and that the enables are also asserted.
- If the simulation has been run, verify in simulation and/or a debug feature capture that the waveform is correct for accessing the AXI4-Lite interface.

Figure: AXI4-Lite Timing

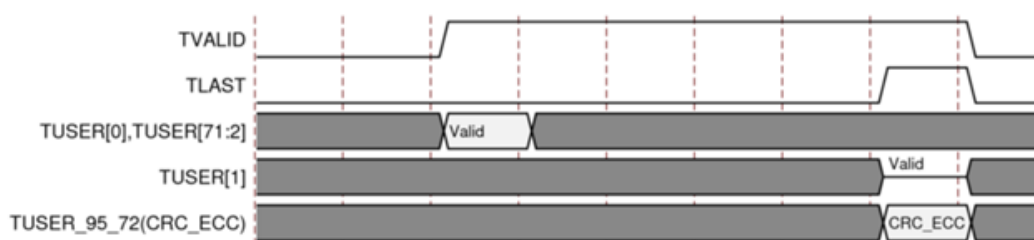


AXI4-Stream Interfaces

If data is not being transmitted or received, check the following conditions:

- If transmit `<interface_name>_tready` is stuck Low following the `<interface_name>_tvalid` input being asserted, the subsystem cannot send data.
- If the receive `<interface_name>_tvalid` is stuck Low, the subsystem is not receiving data.
- Check that the `video_aclrk` and `dphy_clk_200M` inputs are connected and toggling.
- Check subsystem configuration.
- Ensure “Stream line buffer full” condition not getting reported in subsystem Interrupt Status register.
- Sideband Information on AXI4–Stream Interfaces.
- Sideband information such as frame and line number appear on the TUSER signal of the AXI4–Stream interface.
- Start of frame, frame number, line number, word count, and data type need to be sampled by you on the first beat of the transfer.
- Frame Number and Line number is an optional information set by the TX. According to the MIPI CSI-2 specification, TX can send a fixed "0" if these information is not used.
- Packet Error, ECC, and CRC need to be sampled by you on the last beat of the transfer.
- The side band information are optionally sent by the sensor. Refer to the Low Level Protocol section of MIPI CSI-2 standard v2.0 for more details.

Figure: Sideband Information (TUSER) Timing Diagram



Additional Resources and Legal Notices

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, refer to [Support](#).

References

These documents provide supplemental material useful with this guide:

1. MIPI Alliance Standard for Camera Serial Interface CSI-2:
<https://www.mipi.org/specifications/csi-2>
2. AXI4-Stream Video IP and System Design Guide (UG934)
3. MIPI D-PHY LogiCORE IP Product Guide (PG202)
4. AXI Interconnect LogiCORE IP Product Guide (PG059)
5. SmartConnect LogiCORE IP Product Guide (PG247)
6. MIPI Alliance Physical Layer Specifications, D-PHY Specification:
<https://www.mipi.org/current-specifications#D-PHY%20Specification>
7. Vivado Design Suite: AXI Reference Guide (UG1037)
8. Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator (UG994)
9. Vivado Design Suite User Guide: Designing with IP (UG896)
10. Vivado Design Suite User Guide: Getting Started (UG910)
11. Vivado Design Suite User Guide: Logic Simulation (UG900)
12. ISE to Vivado Design Suite Migration Guide (UG911)
13. Vivado Design Suite User Guide: Programming and Debugging (UG908)
14. Vivado Design Suite User Guide: Implementation (UG904)
15. D-PHY Solutions (XAPP894)
16. UltraScale Architecture SelectIO Resources User Guide (UG571)
17. UltraScale Architecture PCB Design User Guide (UG583)
18. LI-IMX274MIPI-FMC product page:
<https://leopardimaging.com/product/platformpartners/amd/li-imx274mipi-fmc/>
19. FMC XM107 Loopback Card User Guide (UG539)
20. Advanced IO Wizard LogiCORE IP Product Guide (PG320)
21. Versal Adaptive SoC SelectIO Resources Architecture Manual (AM010)
22. Vivado Design Suite User Guide: Release Notes, Installation, and Licensing (UG973)
23. MIPI RX C-PHY/D-PHY LogiCORE IP Product Guide (PG435)

Revision History

Section		Revision History
		07/08/2026 Version 6.0
Features		Updated supported data types.
MIPI CSI-2 RX Controller		Added tables.

Section	Revision History
Core Configuration Register	Updated table.
Interrupt Status Register	Updated table.
Interrupt Enable Register	Updated table.
Lane<n> Information Registers	Updated table.
video_aclk Calculation	Added note.
Configuration Tab	Updated parameters and screenshot.
Application Example Design Tab	Updated parameters.
User Parameters	Updated parameters.
Application Example Design	Updated table.
VEK385 D-PHY Application Example Design Overview	Added section.
VEK385 C-PHY Application Example Design Overview	Added section.
SCU200 XP5IO Application Example Design Overview	Added section.
SCU200 HPIO Application Example Design Overview	Added section.
Implementing the Example Design	Updated topic.
Build a Software Application Using the Unified Vitis Flow	Updated topic.
Running the Design on Hardware	Updated topic.
Hardware Validation	Updated table.
Line Buffer Depth Calculation for Versal AI Edge Series Gen2 and Versal Prime Series Gen 2 Devices	Added topic.
11/20/2025 Version 6.0	
MIPI CSI-2 RX Controller	Added a note.

Section	Revision History
YUV420 8-Bit Support	Updated the section.
Port Descriptions	Updated the table.
Interrupt Status Register	Updated the table.
Table 5	Added the table.
Table 9	Added the table.
Interrupt Enable Register	Updated the table.
05/29/2025 Version 6.0	
MIPI RX C-PHY/D-PHY	New section added
MIPI RX PHY Registers	New section added
MIPI CSI-2 RX Controller	Updated the section
Port Descriptions	Updated the section
Interrupt Status Register	Updated the section
Clocking	Updated the section
Implementing the Example Design	Updated the section
11/13/2024 Version 6.0	
Implementing the Example Design	Updated the topic for Unified Vitis flow.
05/30/2024 Version 6.0	
MIPI CSI-2 RX Controller	Updated.
Features	Updated.
Unsupported Features	Updated.
Protocol Configuration Register	Updated.
11/06/2023 Version 5.4	
Vitis Debugger Flow to Program FPGA Instead of XSDB Console	New Section.

Section	Revision History
Port Descriptions	Updated.
VCK190 Hardware Setup	Updated.
VEK280 Hardware Setup	Updated.
05/16/2023 Version 5.3	
VEK280 Setup Details	New section.
MMCM Sharing	Added information in the Shared Logic section.
10/19/2022 Version 5.2	
General Updates	Added new register to allow Dynamic VC selection.
04/26/2022 Version 5.1	
General Updates	Active_lanes port added when CSI-2 Controller Register Interface Disabled.
ZCU102 Application Example Design Overview	Updated to support 4K60 on HDMI output.
11/17/2021 Version 5.1	
Clocking	Updated the examples.
Configuration Tab	Updated Pixel Format and Included Video Format Bridge (VFB) options.
General Updates	Updated MIPI CSI-2 RX Subsystem Application Example Design Block Diagram.
07/14/2021 Version 5.1	
Tclk-Post Requirement	New Section.
Designing with the Subsystem	Added a new note for line rates that are >1500 in.
Configuration Tab	Added a new parameter.

Section	Revision History
Running the Design on Hardware	Updated.
Implementing the Example Design	Updated.
Hardware Validation	Updated.
General Updates	Updated Vivado IDE Parameter to User Parameter Relationship.
01/08/2021 Version 5.1	
General Updates	• Corrections for Versal Adaptive SoC Support. • Added YUV420-8bit support.
Clocking	Revised the examples.
07/16/2020 Version 5.0	
General Updates	Added support for Versal Adaptive SoC.
06/12/2020 Version 5.0	
General Updates	• Updated the register space information to include VCX Frame Error bit in the IER. • Updated the Interoperability testing for UltraScale+ devices. • Example Design for Vitis updated.
11/22/2019 Version 4.1	

Section	Revision History
General Updates	• Updated the register space information to include ISR and IER bits for skewcalhs. • Added Application example design for SP701 board. • Added CSI2 Controller Register Interface GUI option to aid resource optimization. • Removed AXI IIC Option.
07/02/2019 Version 4.0	
General Updates	• Extended line rate support up to 2500 Mbps. • Added support for Deskew sequence detection at MIPI D-PHY for UltraScale+ devices. • Corrected the doc version.
05/22/2019 Version 4.0	
General Updates	• Updated the minimum video clock requirement in chapter 3. • Added MIPI CSI2 RX Subsystem Latency Calculation. • Corrected Data corruption for certain word counts during RAW20 data type reception.
12/05/2018 Version 4.0	

Section	Revision History
General Updates	- Updated video_out_tdest and emb_nonimg_tdest port size. - Updated Table 2–5 to include VCX Frame Error register, Image Information 1, and Image Information 2 Registers for VC4 to VC15. - Updated Table 1–1 to include RAW16, RAW20, and YUV 422 10 bit data types. - Updated Table 4–1 User Parameters. - Included new GUI options for MIPI CSI–2 Standard v2.0 compatibility in the Configuration Tab. - Updated examples in the Pixel Packing for Multiple Data Types section to match the alignment described in AXI4–Stream Video IP and System Design Guide (UG934).
04/04/2018 Version 3.0	
General Updates	- ECC and CRC of long packets are made available on TUSER ports of output stream interfaces. - Added support for additional 7 series devices. - Added dynamic configuration capability for IDELAY Tap values in fixed calibration mode of 7 series.
10/04/2017 Version 3.0	

Section	Revision History
General Updates	- Added Application Example Design to demonstrate a full end-to-end system from capture to display on ZCU102. - Added Board automation support for LI-IMX274MIPI-FMC V1.0 FMC model.
04/05/2017 Version 2.2	
General Updates	- Word Count (WC) corruption limited to current packet. Additional bit in ISR added to report this conditions. - MIPI DPHY v3.1 changes integrated.
11/30/2016 Version 2.1	
General Updates	Added calibration mode parameters for FIXED and AUTO modes.
10/05/2016 Version 2.1	
General Updates	- MIPI D-PHY 3.0 changes integrated - Added 7 series support
04/06/2016 Version 2.0	

Section	Revision History
General Updates	• MIPI D-PHY 2.0 changes integrated • Shared logic support • Video Format Bridge core changes to support RAW8 and User Defined Byte-based Data at all times along with the Vivado IDE selected data type.
11/18/2015 Version 1.0	
Initial Release	N/A

Please Read: Important Legal Notices

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes. THIS INFORMATION IS PROVIDED "AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2015–2026 Advanced Micro Devices, Inc. AMD, the AMD Arrow logo, Artix, Kintex, Spartan, Versal, Virtex, Vitis, Vivado, UltraScale, UltraScale+, Zynq, and combinations thereof are trademarks of Advanced Micro Devices, Inc. AMBA, AMBA Designer, Arm, ARM1176JZ-S, CoreSight, Cortex, PrimeCell, Mali, and MPCore are trademarks of Arm Limited in the US and/or elsewhere. HDMI, HDMI logo, and High-Definition Multimedia Interface are trademarks of HDMI Licensing LLC. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.